CHƯƠNG 2
Dự án Học máy từ Đầu đến Cuối

Trong chương này, bạn sẽ cùng thực hiện một dự án ví dụ từ đầu đến cuối, đóng vai một nhà khoa học dữ liệu (data scientist) vừa mới được tuyển dụng tại một công ty bất động sản. Ví dụ này hoàn toàn mang tính hư cấu; mục tiêu ở đây là minh họa các bước chính của một dự án Machine Learning, chứ không nhằm mục đích cung cấp kiến thức kinh doanh bất động sản. Dưới đây là các bước chính mà chúng ta sẽ cùng trải qua:

1. Quan sát bức tranh tổng thể (Look at the big picture).
2. Thu thập dữ liệu (Get the data).
3. Khám phá và trực quan hóa dữ liệu để thu thập thông tin chuyên sâu (Explore and visualize the data to gain insights).
4. Chuẩn bị dữ liệu cho các thuật toán Machine Learning (Prepare the data for machine learning algorithms).
5. Lựa chọn mô hình và huấn luyện (Select a model and train it).
6. Tinh chỉnh mô hình của bạn (Fine-tune your model).
7. Trình bày giải pháp của bạn (Present your solution).
8. Triển khai, giám sát và bảo trì hệ thống của bạn (Launch, monitor, and maintain your system).

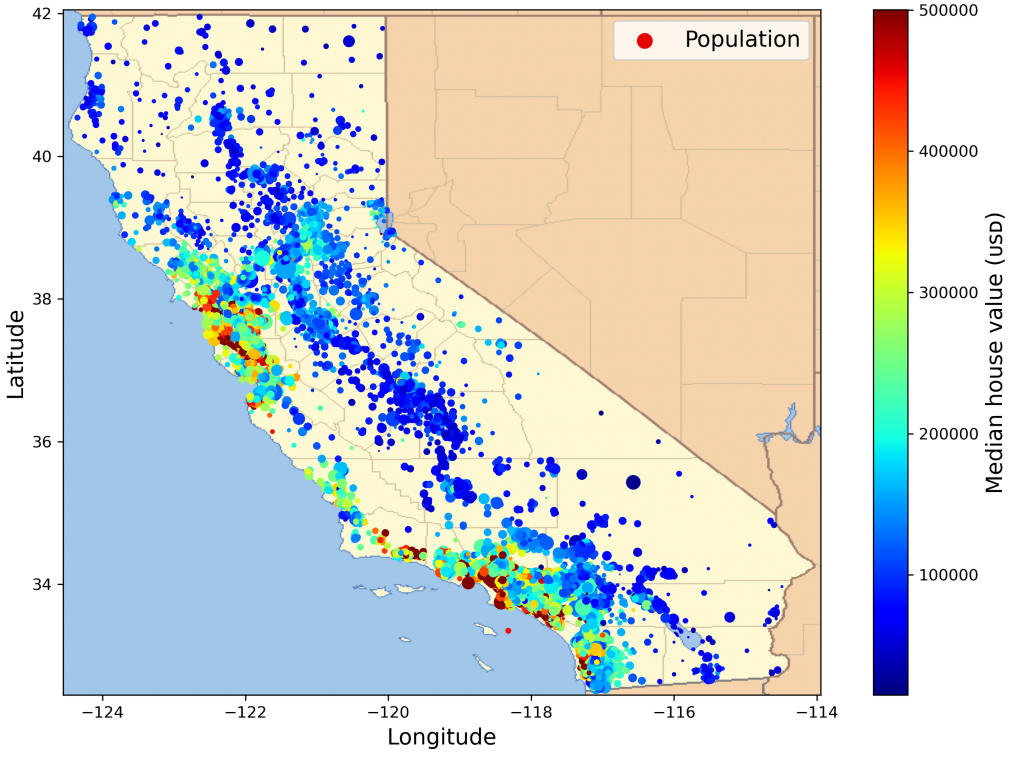
Làm việc với Dữ liệu Thực tế

Khi học về Machine Learning, cách tốt nhất là thử nghiệm trực tiếp với dữ liệu thực tế ngoài đời thực, chứ không phải các bộ dữ liệu nhân tạo. Rất may mắn là hiện nay có hàng nghìn bộ dữ liệu mở để bạn lựa chọn, trải dài trên đủ mọi lĩnh vực. Dưới đây là một số kho lưu trữ dữ liệu mở phổ biến mà bạn có thể sử dụng để thu thập dữ liệu:

- Google Datasets Search
- Hugging Face Datasets
- OpenML.org
- Kaggle.com
- UC Irvine Machine Learning Repository
- Stanford Large Network Dataset Collection
- Amazon's AWS datasets
- U.S. Government's Open Data
- DataPortals.org

- Danh sách các bộ dữ liệu Machine Learning trên Wikipedia

Trong chương này, chúng ta sẽ sử dụng bộ dữ liệu Giá nhà ở California (California Housing Prices) từ kho lưu trữ StatLib[1] (xem Hình 2-1). Tập dữ liệu này dựa trên dữ liệu từ cuộc điều tra dân số California năm 1990. Dữ liệu này không hẳn là gần đây (vào thời điểm đó, một ngôi nhà đẹp ở khu vực Vịnh San Francisco vẫn có mức giá phải chăng), nhưng nó hội tụ rất nhiều phẩm chất tuyệt vời phục vụ mục đích học tập, do đó chúng ta sẽ coi như đây là dữ liệu hiện tại. Vì mục đích giảng dạy, tôi đã thêm vào một thuộc tính phân loại và loại bỏ một số đặc trưng.



Hình 2-1. Giá nhà ở California (California housing prices)

Quan sát Bức tranh Tổng thể

Chào mừng bạn đến với Machine Learning Housing Corporation! Nhiệm vụ đầu tiên của bạn là sử dụng dữ liệu điều tra dân số California để xây dựng một mô hình dự đoán giá nhà trong toàn bang. Dữ liệu này bao gồm các chỉ số như dân số, thu nhập trung vị (median income), và giá nhà trung vị (median housing price) cho từng nhóm khối (block group) tại California. Nhóm khối là đơn vị địa lý nhỏ nhất mà Cục Thống kê Dân

[1] Tập dữ liệu gốc xuất hiện trong bài báo của R. Kelley Pace và Ronald Barry, "Sparse Spatial Autoregressions", *Statistics & Probability Letters* 33, no. 3 (1997): 291–297.

số Hoa Kỳ công bố dữ liệu mẫu (một nhóm khối thường có dân số từ 600 đến 3.000 người). Để cho ngắn gọn, tôi sẽ gọi chúng là các "quận" (districts).

Mô hình của bạn cần học từ dữ liệu này và có khả năng dự đoán giá nhà trung vị ở bất kỳ quận nào, dựa trên tất cả các chỉ số còn lại.



Vì bạn là một nhà khoa học dữ liệu làm việc có tổ chức, điều đầu tiên bạn nên làm là mở bản danh sách kiểm tra dự án Machine Learning (checklist) của mình ra. Bạn có thể bắt đầu với danh sách tại `https://homl.info/checklist`; nó hoạt động tương đối tốt cho hầu hết các dự án Machine Learning, nhưng hãy nhớ điều chỉnh nó sao cho phù hợp với nhu cầu cụ thể của bạn. Trong chương này, chúng ta sẽ đi qua nhiều mục trong checklist, nhưng cũng sẽ bỏ qua một vài mục, hoặc vì chúng quá hiển nhiên, hoặc vì chúng sẽ được thảo luận kỹ trong các chương sau.

Xác định Bài toán

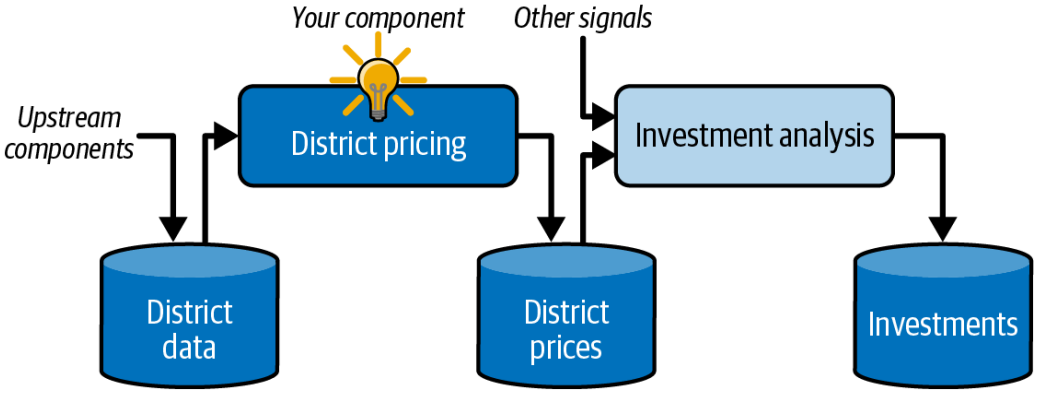
Câu hỏi đầu tiên cần đặt ra cho cấp trên của bạn là mục tiêu kinh doanh chính xác là gì. Việc xây dựng một mô hình bản thân nó có lẽ không phải là đích đến cuối cùng. Công ty dự định sử dụng và hưởng lợi từ mô hình này như thế nào? Biết được mục tiêu là vô cùng quan trọng vì nó sẽ quyết định cách bạn định hình bài toán (frame the problem), những thuật toán nào bạn sẽ lựa chọn, thước đo hiệu năng (performance measure) nào bạn sẽ dùng để đánh giá mô hình, và bạn sẽ dành bao nhiêu nỗ lực để tinh chỉnh nó.

Cấp trên của bạn trả lời rằng đầu ra của mô hình (dự đoán về giá nhà trung vị của một quận) sẽ là yếu tố cốt lõi để xác định xem liệu có đáng để đầu tư vào một khu vực nhất định hay không. Cụ thể hơn, đầu ra của mô hình sẽ được nạp vào một hệ thống Machine Learning khác (xem Hình 2-2), cùng với một số tín hiệu khác.[2] Vì vậy, điều quan trọng là phải làm cho mô hình dự đoán giá nhà của chúng ta đạt độ chính xác cao nhất có thể.

Câu hỏi tiếp theo bạn nên hỏi cấp trên là giải pháp hiện tại trông như thế nào (nếu có). Hiện trạng thực tế thường sẽ cho bạn một điểm chuẩn (baseline) tham chiếu về hiệu năng, cũng như những hiểu biết sâu sắc về cách giải quyết vấn đề. Cấp trên trả lời rằng giá nhà ở các quận hiện đang được ước tính thủ công bởi các chuyên gia: một nhóm thu thập thông tin cập nhật về một quận, và khi họ không thể có được giá nhà trung vị, họ ước tính nó bằng các quy tắc phức tạp.

Cách tiếp cận này rất tốn kém và mất thời gian, đồng thời các ước tính của họ cũng không thực sự xuất sắc; trong những trường hợp mà họ tìm ra được giá nhà trung vị thực tế, họ thường nhận ra rằng các ước tính của mình bị lệch hơn 30%. Đây là lý do tại sao công ty cho rằng việc huấn luyện một mô hình để dự đoán giá nhà trung vị của một quận dựa trên các dữ liệu khác về quận đó sẽ rất hữu ích. Dữ liệu điều tra dân số có vẻ là một tập dữ liệu tuyệt vời để khai thác cho mục đích này, vì nó bao gồm giá nhà trung vị của hàng nghìn quận, cùng với nhiều dữ liệu khác.

[2] Một mẩu thông tin được nạp vào hệ thống Machine Learning thường được gọi là một tín hiệu (*signal*), liên hệ tới lý thuyết thông tin của Claude Shannon mà ông đã phát triển tại Bell Labs để cải tiến viễn thông. Lý thuyết của ông: bạn muốn có tỷ số tín hiệu trên nhiễu (*signal-to-noise ratio*) cao.



Hình 2-2. Một Machine Learning Pipeline phục vụ đầu tư bất động sản

Pipelines

Một chuỗi các thành phần xử lý dữ liệu được gọi là một *data pipeline*. Pipelines rất phổ biến trong các hệ thống Machine Learning, vì có rất nhiều dữ liệu cần thao tác và nhiều bước biến đổi dữ liệu cần áp dụng.

Các thành phần thường chạy bất đồng bộ (*asynchronously*). Mỗi thành phần kéo vào một lượng lớn dữ liệu, xử lý dữ liệu đó, và nhả kết quả ra một kho lưu trữ dữ liệu khác. Sau đó, một lúc sau, thành phần tiếp theo trong pipeline sẽ kéo dữ liệu này vào và tạo ra đầu ra của riêng nó. Mỗi thành phần hoạt động khá độc lập: giao diện giữa các thành phần đơn giản chỉ là kho lưu trữ dữ liệu. Điều này giúp hệ thống trở nên dễ nắm bắt (với sự trợ giúp của đồ thị luồng dữ liệu - data flow graph), và các đội ngũ khác nhau có thể tập trung vào các thành phần khác nhau. Hơn nữa, nếu một thành phần bị hỏng, các thành phần ở phía hạ lưu thường vẫn có thể tiếp tục chạy bình thường (ít nhất là trong một khoảng thời gian) bằng cách chỉ sử dụng đầu ra gần nhất từ thành phần bị hỏng. Điều này làm cho kiến trúc trở nên rất mạnh mẽ và kiên cố (*robust*).

Mặt khác, một thành phần bị hỏng có thể không bị phát hiện trong một khoảng thời gian nếu không triển khai hệ thống giám sát thích hợp. Dữ liệu sẽ trở nên cũ kỹ và hiệu năng của toàn bộ hệ thống sẽ bị sụt giảm.

Với tất cả thông tin này, giờ đây bạn đã sẵn sàng bắt tay vào thiết kế hệ thống của mình. Trước tiên, hãy xác định xem mô hình sẽ cần loại hình giám sát huấn luyện nào: đây là bài toán học có giám sát (supervised), không giám sát (unsupervised), bán giám sát (semi-supervised), tự giám sát (self-supervised), hay học tăng cường (reinforcement learning)? Và đó là bài toán phân loại (classification), hồi quy (regression), hay cái gì khác? Bạn nên sử dụng kỹ thuật batch learning hay online learning? Trước khi đọc tiếp, hãy tạm dừng lại và cố gắng tự trả lời những câu hỏi này.

Bạn đã tìm ra câu trả lời chưa? Hãy cùng xem nhé. Rõ ràng đây là một bài toán học có giám sát điển hình, vì mô hình có thể được huấn luyện với các ví dụ có gắn nhãn (mỗi mẫu dữ liệu đi kèm với đầu ra mong đợi, tức là giá nhà trung vị của quận). Đây là một tác vụ hồi quy điển hình, vì mô hình sẽ được yêu cầu dự đoán một giá trị liên tục. Cụ thể hơn, đây là một bài toán hồi quy đa biến (multiple regression), vì hệ thống sẽ sử dụng nhiều đặc trưng để đưa ra dự đoán (dân số của quận, thu nhập trung vị, v.v.). Đây cũng là một bài toán hồi quy đơn mục tiêu (univariate regression), vì chúng ta chỉ

cố gắng dự đoán một giá trị duy nhất cho mỗi quận. Nếu chúng ta cố gắng dự đoán nhiều giá trị cho mỗi quận, thì đó sẽ là một bài toán hồi quy đa mục tiêu (multivariate regression). Cuối cùng, không có luồng dữ liệu liên tục nào đổ vào hệ thống, không có nhu cầu đặc biệt nào đòi hỏi phải thích ứng nhanh chóng với dữ liệu thay đổi, và dữ liệu đủ nhỏ để nằm gọn trong bộ nhớ, vì vậy batch learning thông thường là hoàn toàn phù hợp.



Nếu dữ liệu có kích thước khổng lồ, bạn có thể phân chia công việc batch learning qua nhiều máy chủ (sử dụng kỹ thuật MapReduce) hoặc sử dụng kỹ thuật online learning.

Lựa chọn Thước đo Hiệu năng

Bước tiếp theo của bạn là lựa chọn một thước đo hiệu năng (performance measure). Một thước đo hiệu năng điển hình cho các bài toán hồi quy là sai số toàn phương trung bình căn (Root Mean Squared Error - RMSE). Nó cho biết hệ thống thường mắc bao nhiêu sai số trong các dự đoán của mình, với trọng số lớn hơn được gán cho các lỗi sai lớn. Phương trình 2-1 thể hiện công thức toán học để tính RMSE.

$$\text{RMSE}(\mathbf{X}, \mathbf{y}, h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(h(\mathbf{x}^{(i)}) - y^{(i)}\right)^2} \tag{2-1}$$

Quy ước Ký hiệu (Notations)

Phương trình này giới thiệu một số ký hiệu Machine Learning rất phổ biến mà tôi sẽ sử dụng xuyên suốt cuốn sách này:

- m là số lượng mẫu (instances) trong tập dữ liệu mà bạn đang đo lường RMSE.
  - Ví dụ, nếu bạn đang đánh giá RMSE trên một tập kiểm định (validation set) gồm 2.000 quận, thì $m = 2.000$.
- $\mathbf{x}^{(i)}$ là một vector chứa tất cả các giá trị đặc trưng (không bao gồm nhãn) của mẫu thứ i trong tập dữ liệu, và $y^{(i)}$ là nhãn của nó (giá trị đầu ra mong muốn cho mẫu đó). $\mathbf{y}$ là một vector chứa nhãn của tất cả các mẫu trong tập dữ liệu.
  - Ví dụ, nếu quận đầu tiên trong tập dữ liệu nằm ở kinh độ $-118,29°$, vĩ độ $33,91°$, và nó có 1.416 cư dân với mức thu nhập trung vị là $38.372\,\$$, và giá trị nhà trung vị là $156.400\,\$$ (tạm thời bỏ qua các đặc trưng khác), thì:

$$\mathbf{x}^{(1)} = \begin{pmatrix} -118,29 \\ 33,91 \\ 1.416 \\ 38.372 \end{pmatrix}$$

và:

$$y^{(1)} = 156.400$$

- $\mathbf{X}$ là ma trận chứa tất cả các giá trị đặc trưng (không bao gồm nhãn) của tất cả các mẫu trong tập dữ liệu. Mỗi hàng tương ứng với một mẫu, và hàng thứ i bằng chuyển vị của vector $\mathbf{x}^{(i)}$, ký hiệu là $(\mathbf{x}^{(i)})^\top$.[3]
  - Ví dụ, nếu quận đầu tiên có các thuộc tính như vừa mô tả, thì ma trận $\mathbf{X}$ sẽ có dạng như sau:

$$\mathbf{X} = \begin{pmatrix} (\mathbf{x}^{(1)})^\top \\ (\mathbf{x}^{(2)})^\top \\ \vdots \\ (\mathbf{x}^{(1999)})^\top \\ (\mathbf{x}^{(2000)})^\top \end{pmatrix} = \begin{pmatrix} -118{,}29 & 33{,}91 & 1.416 & 38.372 \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

- h là hàm dự đoán của hệ thống của bạn, còn được gọi là giả thuyết (hypothesis). Khi hệ thống nhận được một vector đặc trưng của mẫu $\mathbf{x}^{(i)}$, nó sẽ xuất ra một giá trị dự đoán $\hat{y}^{(i)} = h(\mathbf{x}^{(i)})$ cho mẫu đó ($\hat{y}$ được phát âm là "y-hat").
  - Ví dụ, nếu hệ thống của bạn dự đoán giá nhà trung vị ở quận đầu tiên là $158.400\,\$$, thì $\hat{y}^{(1)} = h(\mathbf{x}^{(1)}) = 158.400$. Sai số dự đoán cho quận này là $\hat{y}^{(1)} - y^{(1)} = 2.000$.
- RMSE$(\mathbf{X}, \mathbf{y}, h)$ là hàm chi phí (cost function) được đo lường trên tập hợp các ví dụ bằng cách sử dụng giả thuyết h của bạn.

Chúng ta sử dụng chữ thường in nghiêng cho các giá trị vô hướng (scalar, chẳng hạn như m hoặc $y^{(i)}$) và tên hàm (chẳng hạn như h), chữ thường in đậm cho các vector (chẳng hạn như $\mathbf{x}^{(i)}$), và chữ hoa in đậm cho các ma trận (chẳng hạn như $\mathbf{X}$).

Mặc dù RMSE nhìn chung là thước đo hiệu năng được ưa chuộng nhất cho các tác vụ hồi quy, nhưng trong một số bối cảnh, bạn có thể muốn sử dụng một hàm khác, đặc biệt là khi có nhiều outlier (điểm ngoại lai) trong dữ liệu, vì RMSE khá nhạy cảm với chúng. Trong trường hợp đó, bạn có thể cân nhắc sử dụng sai số tuyệt đối trung bình (Mean Absolute Error - MAE, còn gọi là độ lệch tuyệt đối trung bình - average absolute deviation), được thể hiện trong Phương trình 2-2:

$$\text{MAE}(\mathbf{X}, \mathbf{y}, h) = \frac{1}{m} \sum_{i=1}^{m} \left| h(\mathbf{x}^{(i)}) - y^{(i)} \right| \tag{2-2}$$

Cả RMSE và MAE đều là những cách đo khoảng cách giữa hai vector: vector dự đoán và vector giá trị mục tiêu. Có nhiều thước đo khoảng cách khác nhau, hay chuẩn (norms), có thể sử dụng:

- Việc tính căn bậc hai của tổng các bình phương (RMSE) tương ứng với chuẩn Euclid (Euclidean norm): đây là khái niệm khoảng cách mà tất cả chúng ta đều quen thuộc. Nó còn được gọi là chuẩn ℓ_2, ký hiệu là $\|\cdot\|_2$ (hoặc chỉ đơn giản là $\|\cdot\|$).
- Việc tính tổng các giá trị tuyệt đối (MAE) tương ứng với chuẩn ℓ_1, ký hiệu là $\|\cdot\|_1$. Chuẩn này đôi khi được gọi là chuẩn Manhattan vì nó đo khoảng cách giữa hai điểm trong thành phố nếu bạn chỉ có thể di chuyển dọc theo các khối phố vuông góc.

[3] Nhắc lại rằng toán tử chuyển vị biến đổi một vector cột thành một vector hàng (và ngược lại).

- Tổng quát hơn, chuẩn ℓ_k của một vector **v** chứa n phần tử được định nghĩa là $\|\mathbf{v}\|_k = \left(|v_1|^k + |v_2|^k + \cdots + |v_n|^k\right)^{1/k}$. Chuẩn ℓ_0 cho biết số lượng phần tử khác không trong vector, và chuẩn ℓ_∞ cho biết giá trị tuyệt đối lớn nhất trong vector.

Chỉ số chuẩn càng cao thì nó càng tập trung nhiều vào các giá trị lớn và bỏ qua các giá trị nhỏ. Đây là lý do tại sao RMSE nhạy cảm với các outlier (điểm ngoại lai) hơn so với MAE. Nhưng khi các outlier cực kỳ hiếm gặp (chẳng hạn như trong phân phối hình chuông Gauss), RMSE hoạt động rất xuất sắc và nhìn chung được ưu tiên lựa chọn.

Kiểm tra các Giả định

Cuối cùng, một thói quen thực hành tốt là liệt kê và xác minh các giả định đã được đưa ra từ trước đến nay (bởi bạn hoặc những người khác); điều này có thể giúp bạn sớm phát hiện các vấn đề nghiêm trọng. Ví dụ, mức giá của các quận mà hệ thống của bạn xuất ra sẽ được đưa vào một hệ thống Machine Learning ở hạ nguồn, và bạn giả định rằng các mức giá này sẽ được sử dụng trực tiếp đúng như vậy. Nhưng chuyện gì sẽ xảy ra nếu hệ thống hạ nguồn chuyển đổi các mức giá thành các phân loại (chẳng hạn như "rẻ", "trung bình", hoặc "đắt") rồi sau đó sử dụng các phân loại đó thay vì chính các giá trị giá nhà? Trong trường hợp này, việc dự đoán mức giá hoàn toàn chính xác tuyệt đối không còn quan trọng nữa; hệ thống của bạn chỉ cần dự đoán đúng phân loại danh mục. Nếu đúng như vậy, bài toán lẽ ra phải được định hình như một tác vụ phân loại (classification), chứ không phải là một tác vụ hồi quy (regression). Bạn chắc chắn không muốn phát hiện ra điều này sau khi đã miệt mài xây dựng một hệ thống hồi quy suốt nhiều tháng trời!

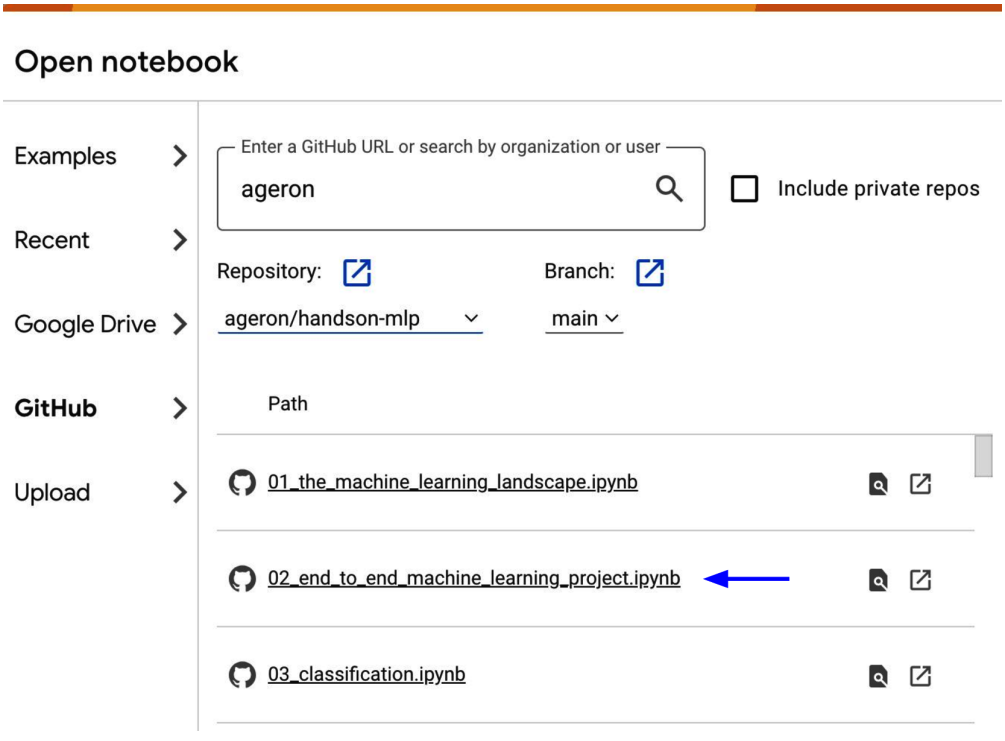
May mắn thay, sau khi trao đổi với đội ngũ phụ trách hệ thống hạ nguồn, bạn có thể hoàn toàn yên tâm rằng họ thực sự cần các mức giá cụ thể bằng số, chứ không chỉ là các danh mục phân loại. Tuyệt vời! Mọi thứ đã sẵn sàng, đèn xanh đã bật, và bạn có thể bắt đầu viết code ngay bây giờ!

Thu thập Dữ liệu (Get the Data)

Đã đến lúc bắt tay vào thực hành. Đừng ngần ngại mở máy tính xách tay của bạn lên và trực tiếp thực thi các ví dụ mã nguồn. Như tôi đã đề cập trong lời nói đầu, tất cả các ví dụ mã trong cuốn sách này đều là mã nguồn mở và có sẵn trực tuyến dưới dạng các Jupyter notebook – những tài liệu tương tác chứa văn bản, hình ảnh và các đoạn mã nguồn có thể thực thi (trong trường hợp của chúng ta là Python). Trong cuốn sách này, tôi mặc định rằng bạn đang chạy các notebook này trên Google Colab, một dịch vụ miễn phí cho phép bạn chạy bất kỳ Jupyter notebook nào trực tiếp trên trình duyệt mà không cần phải cài đặt bất cứ thứ gì trên máy tính cá nhân. Nếu bạn muốn sử dụng một nền tảng trực tuyến khác (ví dụ: Kaggle) hoặc nếu bạn muốn cài đặt mọi thứ cục bộ trên máy tính của mình, vui lòng xem hướng dẫn chi tiết trên trang GitHub của cuốn sách.

Chạy các Ví dụ Mã bằng Google Colab

Trước tiên, hãy mở trình duyệt web và truy cập `https://homl.info/colab-p`: liên kết này sẽ dẫn bạn đến Google Colab và hiển thị danh sách các Jupyter notebook của cuốn sách này (xem Hình 2-3). Bạn sẽ thấy một notebook cho mỗi chương, cùng với một vài notebook bổ sung và các bài hướng dẫn về NumPy, Matplotlib, Pandas, đại số tuyến tính và giải tích vi phân. Ví dụ: nếu bạn nhấp vào `02_end_to_end_machine_learning_project.ipynb`, notebook của Chương 2 sẽ được mở ra trong Google Colab (xem Hình 2-4).

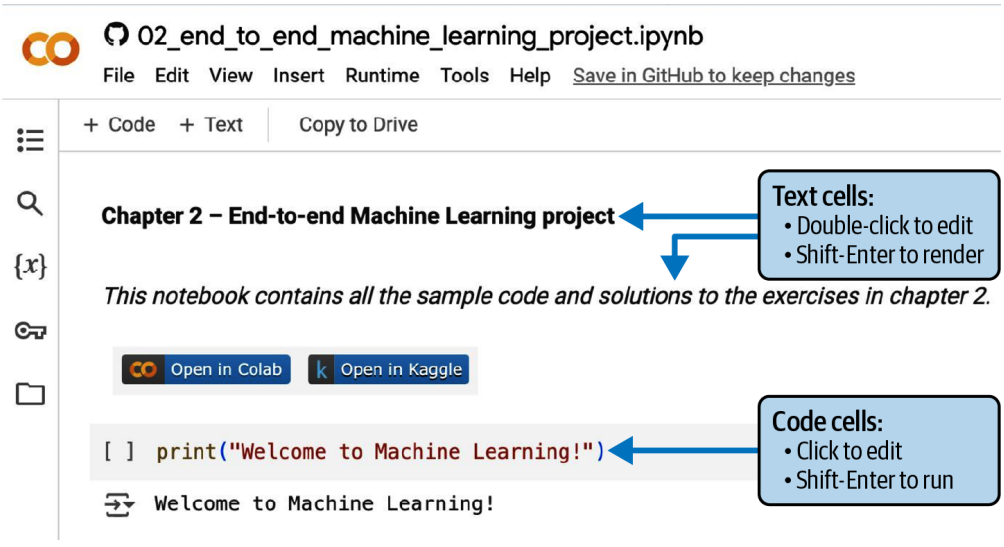


Hình 2-3. Danh sách các notebook trong Google Colab

Một Jupyter notebook được cấu thành từ một danh sách các ô (cells). Mỗi ô chứa mã nguồn thực thi được hoặc văn bản. Hãy thử nhấp đúp vào ô văn bản đầu tiên (chứa câu "Welcome to Machine Learning Housing Corp.!"). Thao tác này sẽ mở ô đó ở chế độ chỉnh sửa. Bạn sẽ nhận thấy rằng Jupyter notebook sử dụng cú pháp Markdown để định dạng văn bản (ví dụ: `**in đậm**`, `*in nghiêng*`, `# Tiêu đề`, `[url](văn bản liên kết)`, v.v.). Hãy thử sửa đổi văn bản này, sau đó nhấn tổ hợp phím Shift-Enter để xem kết quả hiển thị.

Tiếp theo, hãy tạo một ô mã mới bằng cách chọn Insert → "Code cell" từ thanh menu. Ngoài ra, bạn có thể nhấp vào nút + Code trên thanh công cụ, hoặc di chuột qua phần dưới cùng của một ô cho đến khi bạn thấy + Code và + Text xuất hiện, sau đó nhấp vào + Code.

Trong ô mã mới, hãy nhập một đoạn mã Python đơn giản, chẳng hạn như `print("Hello World")`, sau đó nhấn Shift-Enter để chạy đoạn mã này (hoặc nhấp vào



Hình 2-4. Notebook của bạn trong Google Colab

nút ▶ trên cạnh trái của ô). Nếu bạn chưa đăng nhập vào tài khoản Google, bạn sẽ được yêu cầu đăng nhập ngay bây giờ (nếu chưa có tài khoản Google, bạn sẽ cần tạo một tài khoản). Sau khi đăng nhập, khi bạn cố gắng chạy mã, bạn sẽ thấy một cảnh báo bảo mật cho biết rằng notebook này không phải do Google tạo ra. Một kẻ xấu có thể tạo ra một notebook nhằm lừa bạn nhập thông tin đăng nhập Google để truy cập vào dữ liệu cá nhân của bạn, vì vậy trước khi chạy một notebook, hãy luôn đảm bảo bạn tin tưởng tác giả của nó (hoặc kiểm tra kỹ từng ô mã sẽ làm gì trước khi chạy). Giả sử bạn tin tưởng tôi (hoặc bạn có kế hoạch kiểm tra kỹ từng ô mã), giờ đây bạn có thể nhấp vào "Run anyway".

Sau đó, Colab sẽ cấp phát một môi trường thực thi (*runtime*) mới cho bạn: đây là một máy ảo miễn phí đặt trên các máy chủ của Google, chứa sẵn một loạt các công cụ và thư viện Python, bao gồm tất cả những gì bạn cần cho hầu hết các chương (ở một số chương, bạn sẽ cần chạy một lệnh để cài đặt thêm các thư viện bổ sung). Quá trình này sẽ mất vài giây. Tiếp theo, Colab sẽ tự động kết nối với môi trường thực thi này và sử dụng nó để thực thi ô mã mới của bạn. Điều quan trọng cần lưu ý là mã nguồn chạy trên môi trường thực thi từ xa của máy chủ, chứ không phải trên máy tính cá nhân của bạn. Kết quả đầu ra của đoạn mã sẽ được hiển thị ngay bên dưới ô đó. Xin chúc mừng, bạn đã chạy thành công đoạn mã Python đầu tiên trên Colab!



Để chèn một ô mã mới, bạn cũng có thể nhấn Ctrl-M (hoặc Cmd-M trên macOS) rồi nhấn phím A (để chèn phía trên ô hiện tại) hoặc B (để chèn phía dưới). Còn rất nhiều phím tắt tiện lợi khác: bạn có thể xem và chỉnh sửa chúng bằng cách nhấn Ctrl-M (hoặc Cmd-M) rồi nhấn H. Nếu bạn chọn chạy notebook cục bộ trên máy tính của mình thay vì sử dụng Colab, bạn sẽ cần cài đặt Jupyter; vui lòng tham khảo hướng dẫn cài đặt tại `https://homl.info/install`.

Lưu lại các Thay đổi Mã và Dữ liệu của Bạn

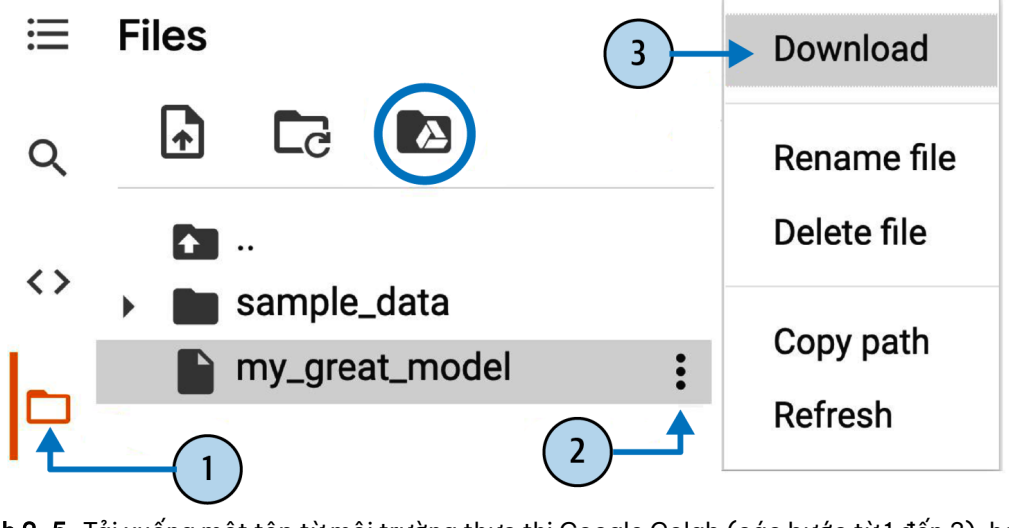
Mọi thay đổi bạn thực hiện đối với notebook Colab mặc định sẽ không được lưu vào bản gốc của cuốn sách. Nếu muốn lưu lại các chỉnh sửa của mình, bạn cần lưu một bản sao vào Google Drive cá nhân của bạn (bằng cách chọn File → "Save a copy in Drive" từ menu) hoặc lưu vào kho lưu trữ GitHub của bạn (File → "Save a copy in GitHub").

Bất kỳ tệp dữ liệu nào mà mã nguồn của bạn tải xuống hoặc tạo ra trong quá trình chạy sẽ được lưu trữ trên hệ thống tệp của môi trường thực thi Colab. Bạn có thể duyệt các tệp này bằng cách nhấp vào biểu tượng thư mục ở bảng điều khiển bên trái của giao diện Colab.



Google Colab chỉ dành cho mục đích sử dụng tương tác: bạn có thể thoải mái thử nghiệm trong các notebook và tinh chỉnh mã theo ý muốn, nhưng bạn không thể để các notebook chạy tự động không giám sát trong một thời gian dài, nếu không Colab sẽ tự động ngắt kết nối khỏi môi trường thực thi và xóa sạch nó! Mọi tệp dữ liệu được lưu trên môi trường thực thi đó sẽ bị mất vĩnh viễn. Vì vậy, hãy chắc chắn rằng bạn đã tải xuống các tệp quan trọng về máy tính của mình hoặc lưu chúng vào Google Drive.

Để tải một tệp từ môi trường thực thi Colab về máy tính của bạn, hãy mở bảng điều khiển Tệp (bước 1 trong Hình 2-5), tìm tệp bạn muốn tải xuống, nhấp vào dấu ba chấm dọc bên cạnh tệp đó (bước 2), rồi chọn "Download" (bước 3). Ngoài ra, bạn cũng có thể gắn kết (*mount*) Google Drive của mình vào môi trường thực thi bằng cách nhấp vào biểu tượng Google Drive (được khoanh tròn trong Hình 2-5) và làm theo hướng dẫn trên màn hình. Sau khi gắn kết, Google Drive của bạn sẽ xuất hiện dưới dạng một thư mục có tên `drive` bên trong thư mục `/content`, cho phép bạn đọc và ghi các tệp trực tiếp vào Google Drive từ mã nguồn Python của mình bằng các đường dẫn tệp tiêu chuẩn!



Hình 2-5. Tải xuống một tệp từ môi trường thực thi Google Colab (các bước từ 1 đến 3), hoặc gắn kết Google Drive (biểu tượng được khoanh tròn)

Theo mặc định, Google Drive của bạn sẽ được gắn kết tại `/content/drive/MyDrive`. Nếu bạn muốn sao lưu một tệp dữ liệu, chỉ cần sao chép nó vào thư mục này bằng cách chạy `!cp /content/my_great_model /content/drive/MyDrive`. Bất kỳ lệnh nào bắt đầu bằng dấu chấm than (`!`) đều được coi là một lệnh shell, chứ không phải mã nguồn Python: `cp` là lệnh shell của Linux để sao chép một tệp từ đường dẫn này sang đường dẫn khác. Lưu ý rằng môi trường thực thi Colab chạy trên nền tảng Linux (cụ thể là Ubuntu).

Sức mạnh và Sự Nguy hiểm của Tính Tương tác

Tính tương tác là một trong những điểm mạnh lớn nhất của Jupyter Notebook: bạn có thể chạy một ô mã, kiểm tra kết quả đầu ra, sau đó sửa đổi mã và chạy lại ô đó, hoặc chạy một ô khác để thử nghiệm một ý tưởng mới. Tuy nhiên, tính tương tác cũng tiềm ẩn một nguy cơ đáng kể: các ô mã có thể được thực thi theo bất kỳ thứ tự nào! Nếu bạn chạy các ô không theo đúng thứ tự từ trên xuống dưới, bạn có thể vô tình định nghĩa một biến trong một ô ở phía dưới, sau đó sử dụng nó trong một ô ở phía trên, và mọi thứ vẫn hoạt động bình thường trong phiên làm việc hiện tại của bạn. Nhưng khi bạn hoặc ai đó khởi động lại notebook và chạy lại từ đầu, mã nguồn sẽ báo lỗi vì biến đó chưa được định nghĩa tại thời điểm gọi!

Để tránh cạm bẫy này, bạn nên thường xuyên kiểm tra xem notebook của mình có thể chạy mượt mà từ đầu đến cuối hay không bằng cách chọn Runtime → "Restart and run all" từ menu. Thao tác này sẽ đặt lại môi trường thực thi về trạng thái ban đầu sạch sẽ và chạy tuần tự từng ô từ trên xuống dưới.



Nếu bạn từng gặp phải một lỗi kỳ lạ, hãy thử khởi động lại môi trường thực thi (bằng cách chọn Runtime → "Restart runtime" từ menu) và sau đó chạy lại tất cả các ô từ đầu notebook. Cách này thường giải quyết được vấn đề. Nếu không, rất có thể một trong những thay đổi mà bạn đã thực hiện đã làm hỏng notebook: chỉ cần quay trở lại notebook gốc và thử lại. Nếu nó vẫn tiếp tục báo lỗi, vui lòng nộp một báo cáo lỗi (issue) trên GitHub.

Mã Nguồn trong Sách so với Mã Nguồn trong Notebook

Đôi khi bạn có thể nhận thấy một vài khác biệt nhỏ giữa mã nguồn trong cuốn sách này và mã nguồn trong các notebook. Điều này có thể xảy ra vì một vài lý do:

- Một thư viện có thể đã thay đổi đôi chút tại thời điểm bạn đọc những dòng này, hoặc có thể dù tôi đã nỗ lực hết sức nhưng vẫn có sai sót trong sách. Thật đáng tiếc là tôi không thể sửa đổi một cách kỳ diệu mã nguồn trong bản in cuốn sách của bạn (trừ khi bạn đang đọc bản điện tử và có thể tải xuống phiên bản mới nhất), nhưng tôi có thể chỉnh sửa các notebook. Vì vậy, nếu bạn gặp lỗi sau khi sao chép mã từ cuốn sách này, vui lòng tìm mã đã được sửa trong các notebook: tôi sẽ luôn cố gắng giữ cho chúng không có lỗi và cập nhật với các phiên bản thư viện mới nhất.

- Các notebook chứa một số mã bổ sung để làm đẹp các hình vẽ (thêm nhãn, thiết lập cỡ chữ, v.v.) và để lưu chúng ở độ phân giải cao cho cuốn sách này. Bạn hoàn toàn có thể an tâm bỏ qua phần mã bổ sung này nếu muốn.
- Tôi đã tối ưu hóa mã nguồn cho khả năng đọc và sự đơn giản: tôi làm cho mã phẳng và tuyến tính nhất có thể, định nghĩa rất ít hàm hoặc lớp. Mục tiêu là đảm bảo rằng đoạn mã bạn đang chạy luôn hiện diện ngay trước mắt bạn, chứ không bị lồng sâu bên trong nhiều tầng trừu tượng mà bạn phải tốn công tìm kiếm. Điều này cũng giúp bạn dễ dàng thử nghiệm và vọc vạch với mã hơn. Để giữ sự đơn giản, khả năng xử lý lỗi (error handling) được giới hạn, và tôi đã đặt một số lệnh import ít phổ biến ngay tại nơi chúng cần dùng (thay vì đặt ở đầu tệp như khuyến nghị của hướng dẫn phong cách Python PEP 8). Dù vậy, mã nguồn trên môi trường sản xuất thực tế của bạn cũng sẽ không quá khác biệt: chỉ cần mô-đun hóa hơn một chút, cùng với các bài kiểm thử và cơ chế xử lý lỗi bổ sung.

Được rồi! Khi bạn đã cảm thấy thoải mái với Colab, bạn đã sẵn sàng để tải xuống dữ liệu.

Tải xuống Dữ liệu (Download the Data)

Trong môi trường làm việc thông thường, dữ liệu của bạn sẽ có sẵn trong một cơ sở dữ liệu quan hệ hoặc một kho lưu trữ dữ liệu phổ biến khác, và phân tán trên nhiều bảng/tài liệu/phân vùng. Để truy cập dữ liệu, trước tiên bạn cần phải có thông tin xác thực, được cấp quyền truy cập và làm quen với lược đồ dữ liệu (data schema).[4] Tuy nhiên, trong dự án này, mọi thứ đơn giản hơn nhiều: bạn chỉ cần tải xuống một tệp nén duy nhất (`housing.tgz`) chứa một tệp CSV (Comma-Separated Values) có tên là `housing.csv` với toàn bộ dữ liệu.

Bạn có thể tải xuống tệp này thủ công bằng trình duyệt web và giải nén nó, nhưng viết một hàm nhỏ để làm việc đó bằng Python sẽ chuyên nghiệp và linh hoạt hơn nhiều. Cách làm này đặc biệt hữu ích nếu dữ liệu được cập nhật định kỳ: bạn có thể viết một kịch bản nhỏ sử dụng hàm này để lấy dữ liệu mới nhất (hoặc thiết lập một tác vụ tự động chạy định kỳ). Việc tự động hóa quy trình thu thập dữ liệu cũng rất tiện lợi nếu bạn cần cài đặt bộ dữ liệu trên nhiều máy tính khác nhau.

Dưới đây là hàm dùng để tải xuống và nạp dữ liệu:

```
from pathlib import Path
import pandas as pd
import tarfile
import urllib.request

def load_housing_data():
    tarball_path = Path("datasets/housing.tgz")
    if not tarball_path.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/housing.tgz"
        urllib.request.urlretrieve(url, tarball_path)
        with tarfile.open(tarball_path) as housing_tarball:
```

[4] Bạn cũng có thể cần phải kiểm tra các ràng buộc pháp lý, chẳng hạn như các trường dữ liệu riêng tư không bao giờ được phép sao chép tới những nơi lưu trữ không an toàn.

```
            housing_tarball.extractall(path="datasets", filter="data")
    return pd.read_csv(Path("datasets/housing/housing.csv"))

housing_full = load_housing_data()
```



Nếu bạn gặp lỗi `SSL: CERTIFICATE_VERIFY_FAILED` trên hệ điều hành macOS, rất có thể bạn cần phải cài đặt gói `certifi`, như được giải thích chi tiết tại https://homl.info/sslerror.

Khi hàm `load_housing_data()` được gọi, nó sẽ tìm kiếm tệp `datasets/housing.tgz`. Nếu không tìm thấy, nó sẽ tạo thư mục `datasets` bên trong thư mục hiện tại (mặc định là `/content` trong Colab), tải tệp `housing.tgz` từ kho lưu trữ GitHub `ageron/data`, và giải nén nội dung của nó vào thư mục `datasets`; thao tác này sẽ tạo ra thư mục `datasets/housing` chứa tệp `housing.csv` bên trong. Cuối cùng, hàm nạp tệp CSV này vào một đối tượng Pandas DataFrame chứa toàn bộ dữ liệu và trả về đối tượng đó.



Nếu bạn đang sử dụng Python 3.12 hoặc 3.13 trở lên, bạn nên thêm đối số `filter='data'` vào phương thức `extractall()`: điều này giới hạn các quyền thao tác của thuật toán giải nén và cải thiện tính bảo mật (xem tài liệu chính thức của Python để biết thêm chi tiết).

Xem Nhanh Cấu trúc Dữ liệu (Take a Quick Look at the Data Structure)

Bạn bắt đầu bằng việc quan sát 5 hàng đầu tiên của dữ liệu bằng phương thức `head()` của DataFrame (xem Hình 2-6).

```
housing.head()
```

	longitude	latitude	housing_median_age	median_income	ocean_proximity	median_house_value
0	-122.23	37.88	41.0	8.3252	NEAR BAY	452600.0
1	-122.22	37.86	21.0	8.3014	NEAR BAY	358500.0
2	-122.24	37.85	52.0	7.2574	NEAR BAY	352100.0
3	-122.25	37.85	52.0	5.6431	NEAR BAY	341300.0
4	-122.25	37.85	52.0	3.8462	NEAR BAY	342200.0

Hình 2-6. Năm hàng đầu tiên trong tập dữ liệu

Mỗi hàng đại diện cho một quận. Có 10 thuộc tính (không phải tất cả đều hiển thị trong ảnh chụp màn hình): `longitude` (kinh độ), `latitude` (vĩ độ), `housing_median_age` (tuổi nhà trung vị), `total_rooms` (tổng số phòng), `total_bedrooms` (tổng số phòng ngủ), `population` (dân số), `households` (số hộ gia đình), `median_income` (thu nhập trung vị), `median_house_value` (giá trị nhà trung vị), và `ocean_proximity` (khoảng cách tới đại dương).

Phương thức `info()` rất hữu ích để có được mô tả ngắn gọn và nhanh chóng về dữ liệu, đặc biệt là tổng số hàng, kiểu dữ liệu của từng thuộc tính và số lượng giá trị không

bị thiếu (*non-null values*):

```
>>> housing_full.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```



Trong cuốn sách này, khi một ví dụ mã nguồn chứa sự kết hợp giữa mã lệnh và kết quả đầu ra, giống như trường hợp ở trên, nó được định dạng tương tự như trong trình thông dịch Python tương tác để tăng khả năng đọc: các dòng mã được mở đầu bằng `>>>` (hoặc `...` cho các khối thụt đầu dòng), còn kết quả đầu ra không có tiền tố nào.

Có tổng cộng 20.640 mẫu (instances) trong tập dữ liệu, có nghĩa là nó khá nhỏ theo tiêu chuẩn của Machine Learning hiện đại, nhưng hoàn hảo để bắt đầu học tập. Bạn sẽ nhận thấy rằng thuộc tính `total_bedrooms` chỉ có 20.433 giá trị không rỗng (non-null), nghĩa là có 207 quận bị thiếu đặc trưng này. Bạn sẽ cần phải xử lý vấn đề này ở các bước sau.

Tất cả các thuộc tính đều là dữ liệu dạng số, ngoại trừ `ocean_proximity`. Kiểu dữ liệu của nó là `object`, do đó về mặt lý thuyết nó có thể chứa bất kỳ loại đối tượng Python nào. Nhưng vì bạn đã nạp dữ liệu này từ tệp CSV, bạn biết chắc rằng nó phải là một thuộc tính dạng văn bản. Khi quan sát 5 hàng đầu tiên, bạn có thể đã nhận thấy các giá trị trong cột `ocean_proximity` lặp đi lặp lại nhiều lần, điều này cho thấy đây có khả năng là một thuộc tính phân loại (categorical attribute). Bạn có thể tìm hiểu xem có những danh mục nào tồn tại và có bao nhiêu quận thuộc về từng danh mục bằng cách sử dụng phương thức `value_counts()`:

```
>>> housing_full["ocean_proximity"].value_counts()
ocean_proximity
<1H OCEAN     9136
INLAND        6551
NEAR OCEAN    2658
NEAR BAY      2290
ISLAND           5
Name: count, dtype: int64
```

Bây giờ hãy xem xét các trường dữ liệu còn lại. Phương thức `describe()` hiển thị bảng tóm tắt thống kê của tất cả các thuộc tính dạng số (Hình 2-7).

```
housing.describe()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	median_house_value
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.385070	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	500001.000000

Hình 2-7. Tóm tắt thống kê của từng thuộc tính dạng số

Các hàng `count`, `mean`, `min`, và `max` đều mang ý nghĩa rất quen thuộc. Lưu ý rằng các giá trị rỗng (null values) sẽ bị bỏ qua (vì vậy, ví dụ, `count` của `total_bedrooms` là 20.433 chứ không phải 20.640). Hàng `std` thể hiện độ lệch chuẩn (standard deviation), đo lường mức độ phân tán của các giá trị dữ liệu.[5]

Các hàng `25%`, `50%`, và `75%` hiển thị các phân vị vị trí tương ứng (percentiles): một bách phân vị cho biết giá trị mà dưới đó có một tỷ lệ phần trăm quan sát nhất định trong một nhóm rơi vào. Ví dụ: 25% số quận có `housing_median_age` thấp hơn 18, trong khi 50% thấp hơn 29 và 75% thấp hơn 37. Những giá trị này thường được gọi là phân vị thứ 25 (hoặc tứ phân vị thứ nhất - first quartile), trung vị (median), và phân vị thứ 75 (hoặc tứ phân vị thứ ba - third quartile).

Một cách nhanh chóng khác để cảm nhận bản chất của loại dữ liệu bạn đang xử lý là vẽ biểu đồ tần suất (histogram) cho từng thuộc tính dạng số. Biểu đồ histogram hiển thị số lượng mẫu (trên trục tung) có dải giá trị nhất định (trên trục hoành). Bạn có thể vẽ biểu đồ này cho từng thuộc tính riêng lẻ, hoặc có thể gọi phương thức `hist()` trên toàn bộ tập dữ liệu (như thể hiện trong đoạn mã sau), và nó sẽ tự động vẽ một histogram cho từng thuộc tính dạng số (xem Hình 2-8).

Số lượng khoảng giá trị có thể được điều chỉnh bằng cách sử dụng tham số `bins` (hãy thử thay đổi giá trị này để quan sát biểu đồ thay đổi ra sao):

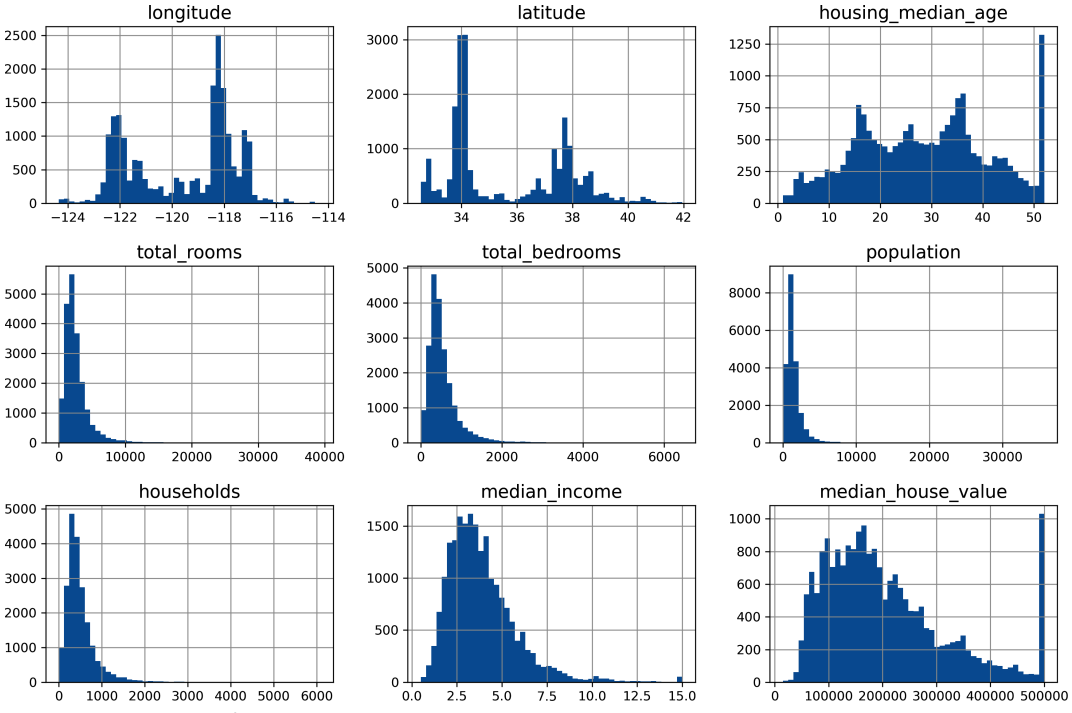
```
import matplotlib.pyplot as plt

housing_full.hist(bins=50, figsize=(12, 8))
plt.show()
```

Quan sát các biểu đồ histogram này, bạn sẽ nhận thấy một vài điểm đáng chú ý:

- Thứ nhất, thuộc tính thu nhập trung vị (`median_income`) trông có vẻ như không được

[5] Độ lệch chuẩn thường được ký hiệu là σ (chữ cái Hy Lạp sigma), và nó là căn bậc hai của phương sai (variance) – vốn là trung bình của bình phương độ lệch so với giá trị trung bình. Khi một đặc trưng có phân phối chuẩn hình chuông (còn gọi là phân phối Gauss), vốn rất phổ biến, quy tắc "68-95-99.7" sẽ được áp dụng: khoảng 68% giá trị nằm trong khoảng 1σ tính từ giá trị trung bình, 95% nằm trong khoảng 2σ, và 99,7% nằm trong khoảng 3σ.



Hình 2-8. Biểu đồ phân phối tần suất (histogram) cho từng thuộc tính dạng số

tính bằng đơn vị đô la Mỹ (USD). Sau khi hỏi lại nhóm thu thập dữ liệu, bạn được biết rằng dữ liệu đã được thu nhỏ tỷ lệ (scaled) và giới hạn chặn trên (capped) ở mức 15 (thực tế là 15,0001) đối với các mức thu nhập trung vị cao hơn, và ở mức 0,5 (thực tế là 0,4999) đối với các mức thu nhập trung vị thấp hơn. Các con số này đại diện cho khoảng chục nghìn đô la (ví dụ: số 3 thực chất có nghĩa là khoảng $30.000\,\$$). Làm việc với các thuộc tính đã qua tiền xử lý là điều rất phổ biến trong Machine Learning, và nó không hẳn là một vấn đề xấu, nhưng bạn nên luôn tìm hiểu kỹ xem dữ liệu đã được tính toán như thế nào.

- Tuổi nhà trung vị (`housing_median_age`) và giá trị nhà trung vị (`median_house_value`) cũng bị áp mức trần (capped). Điểm bị giới hạn sau có thể là một vấn đề cực kỳ nghiêm trọng vì đó chính là thuộc tính mục tiêu của bạn (các nhãn mục tiêu - labels). Các thuật toán Machine Learning của bạn có thể sẽ học được quy luật rằng giá nhà không bao giờ vượt quá giới hạn đó! Bạn cần kiểm tra lại với nhóm khách hàng của mình (đội ngũ sẽ sử dụng đầu ra của hệ thống) để xem liệu điều này có phải là vấn đề đối với họ hay không. Nếu họ trả lời rằng họ cần những dự đoán chính xác ngay cả khi mức giá vượt quá $500.000\,\$$, thì bạn sẽ có hai phương án:
  1. Thu thập nhãn chính xác cho các quận có giá trị bị áp trần.
  2. Loại bỏ hoàn toàn các quận đó khỏi tập huấn luyện (và cả khỏi tập kiểm thử, vì hệ thống của bạn không nên bị đánh giá kém chỉ vì nó dự đoán các giá trị vượt quá $500.000\,\$$).
- Các thuộc tính này có thang đo (scales) rất khác nhau. Chúng ta sẽ thảo luận chi

tiết về điều này ở phần sau của chương khi khám phá kỹ thuật co giãn đặc trưng (feature scaling).

- Cuối cùng, nhiều biểu đồ histogram bị lệch phải (skewed right): phần đuôi của chúng trải dài sang bên phải của giá trị trung vị nhiều hơn so với bên trái. Điều này có thể khiến một số thuật toán Machine Learning gặp khó khăn hơn trong việc phát hiện các quy luật mẫu. Ở phần sau, bạn sẽ thử áp dụng các phép biến đổi để đưa các thuộc tính này về phân phối đối xứng và dạng hình chuông hơn.

Giờ đây bạn đã có được sự hiểu biết rõ ràng hơn về bản chất của tập dữ liệu mà mình đang xử lý.

Tạo Tập Kiểm thử (Create a Test Set)

Trước khi đào sâu phân tích dữ liệu thêm nữa, bạn cần tạo ngay một tập kiểm thử (test set), gạt nó sang một bên, và tuyệt đối không bao giờ nhìn vào nó. Việc chủ động gạt một phần dữ liệu sang một bên ở giai đoạn đầu này thoạt nghe có vẻ kỳ lạ. Rốt cuộc, bạn mới chỉ lướt qua dữ liệu một cách rất sơ lược, và chắc chắn bạn nên tìm hiểu thêm thật nhiều về nó trước khi quyết định chọn thuật toán nào, đúng không? Điều đó đúng, nhưng bộ não con người là một hệ thống phát hiện quy luật (pattern detection) vô cùng kỳ diệu, và điều đó cũng đồng nghĩa với việc nó rất dễ bị rơi vào cạm bẫy quá khớp (overfitting): nếu bạn nhìn vào tập kiểm thử, bạn có thể tình cờ bắt gặp một mô thức trông có vẻ thú vị trong dữ liệu kiểm thử, khiến bạn bị dẫn dắt và lựa chọn một loại mô hình Machine Learning cụ thể nào đó. Khi bạn ước tính sai số khái quát hóa (generalization error) bằng tập kiểm thử đó, ước tính của bạn sẽ quá lạc quan, và bạn sẽ xuất bản một hệ thống không hoạt động tốt như kỳ vọng trong thực tế. Hiện tượng này được gọi là thiên lệch do nhìn trộm dữ liệu (*data snooping bias*).

Về mặt lý thuyết, việc tạo một tập kiểm thử rất đơn giản: chỉ cần chọn ngẫu nhiên một số mẫu, thường là 20% tập dữ liệu (hoặc ít hơn nếu tập dữ liệu của bạn có quy mô cực lớn), và để riêng chúng ra:

```
import numpy as np

def shuffle_and_split_data(data, test_ratio, rng):
    shuffled_indices = rng.permutation(len(data))
    test_set_size = int(len(data) * test_ratio)
    test_indices = shuffled_indices[:test_set_size]
    train_indices = shuffled_indices[test_set_size:]
    return data.iloc[train_indices], data.iloc[test_indices]
```

Sau đó bạn có thể sử dụng hàm này như sau:

```
>>> rng = np.random.default_rng()  # default random number generator
>>> train_set, test_set = shuffle_and_split_data(housing_full, 0.2, rng)
>>> len(train_set)
16512
>>> len(test_set)
4128
```

Cách làm này tuy hiệu quả nhưng chưa hoàn hảo: nếu bạn chạy lại chương trình một lần nữa, nó sẽ sinh ra một tập kiểm thử hoàn toàn khác! Theo thời gian, bạn (hoặc các thuật toán Machine Learning của bạn) sẽ dần thấy toàn bộ tập dữ liệu, và đây chính xác là điều bạn muốn tránh bằng mọi giá.

Một giải pháp là lưu lại tập kiểm thử ngay trong lần chạy đầu tiên và sau đó nạp lại nó trong các lần chạy tiếp theo. Một phương án khác là thiết lập giá trị mầm ngẫu nhiên (seed) cho bộ sinh số ngẫu nhiên (ví dụ: truyền `seed=42` vào hàm `default_rng()`)[6] để đảm bảo nó luôn sinh ra cùng một chuỗi số ngẫu nhiên mỗi khi bạn chạy chương trình.

Tuy nhiên, cả hai giải pháp này đều sẽ bị phá vỡ vào lần tới khi bạn tải về một tập dữ liệu đã được cập nhật. Để có được một phân chia Train/Test Split ổn định ngay cả sau khi cập nhật tập dữ liệu, một giải pháp phổ biến là sử dụng định danh duy nhất (identifier) của từng mẫu để quyết định xem nó có nên thuộc về tập kiểm thử hay không (giả sử các mẫu có mã định danh duy nhất và bất biến). Ví dụ: bạn có thể tính toán mã băm (hash) của định danh của từng mẫu và đưa mẫu đó vào tập kiểm thử nếu mã băm nhỏ hơn hoặc bằng 20% giá trị băm tối đa. Điều này đảm bảo rằng tập kiểm thử sẽ duy trì tính nhất quán qua nhiều lần chạy, ngay cả khi bạn làm mới hoặc bổ sung thêm dữ liệu. Tập kiểm thử mới sẽ chứa 20% số mẫu mới, nhưng sẽ tuyệt đối không chứa bất kỳ mẫu nào trước đó từng nằm trong tập huấn luyện.

Dưới đây là một cách triển khai khả thi:

```
from zlib import crc32

def is_id_in_test_set(identifier, test_ratio):
    return crc32(np.int64(identifier)) < test_ratio * 2**32

def split_data_with_id_hash(data, test_ratio, id_column):
    ids = data[id_column]
    in_test_set = ids.apply(lambda id_: is_id_in_test_set(id_, test_ratio))
    return data.loc[~in_test_set], data.loc[in_test_set]
```

Thật không may, tập dữ liệu nhà ở này không có sẵn một cột định danh. Giải pháp đơn giản nhất là sử dụng chính chỉ số hàng (row index) làm ID:

```
housing_with_id = housing_full.reset_index()  # adds an `index` column
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "index")
```

Nếu sử dụng chỉ số hàng làm định danh duy nhất, bạn cần đảm bảo chắc chắn rằng dữ liệu mới luôn được nối vào cuối tập dữ liệu và không bao giờ có bất kỳ hàng nào bị xóa bỏ. Nếu không thể đảm bảo điều này, bạn có thể thử sử dụng các đặc trưng ổn định nhất để xây dựng một mã định danh duy nhất. Ví dụ: kinh độ và vĩ độ của một quận được đảm bảo là hoàn toàn ổn định trong vài triệu năm tới, vì vậy bạn có thể kết hợp chúng lại thành một ID như sau:[7]

[6] Bạn sẽ thường xuyên thấy mọi người đặt hạt giống ngẫu nhiên (random seed) là 42. Con số này không có đặc tính toán học đặc biệt nào, ngoại trừ việc nó là Câu trả lời cho Câu hỏi Cuối cùng về Sự sống, Vũ trụ và Mọi thứ trong tác phẩm kinh điển của Douglas Adams.

[7] Thông tin vị trí thực chất khá thô, và kết quả là nhiều quận sẽ có ID hoàn toàn giống hệt nhau, do đó chúng sẽ cùng rơi vào một tập (kiểm thử hoặc huấn luyện). Điều này gây ra sai lệch lấy mẫu (*sampling bias*) không mong muốn.

```
housing_with_id["id"] = (housing_full["longitude"] * 1000
                         + housing_full["latitude"])
train_set, test_set = split_data_with_id_hash(housing_with_id, 0.2, "id")
```

Scikit-Learn cung cấp một số hàm hỗ trợ chia nhỏ tập dữ liệu thành nhiều tập con theo nhiều cách khác nhau. Hàm đơn giản nhất là `train_test_split()`, thực hiện chức năng gần như tương tự hàm `shuffle_and_split_data()` mà chúng ta đã tự định nghĩa trước đó, kèm theo một vài tính năng bổ sung. Thứ nhất, nó có tham số `random_state` cho phép bạn thiết lập hạt giống sinh số ngẫu nhiên. Thứ hai, bạn có thể truyền vào nhiều tập dữ liệu có số hàng giống hệt nhau, và hàm sẽ phân chia chúng trên cùng các vị trí chỉ số (điều này cực kỳ hữu ích nếu bạn có một DataFrame riêng biệt dành cho các nhãn mục tiêu):

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(housing_full, test_size=0.2,
                                       random_state=42)
```

Cho đến nay, chúng ta mới chỉ xem xét các phương pháp lấy mẫu hoàn toàn ngẫu nhiên (purely random sampling). Phương pháp này nhìn chung là tốt nếu tập dữ liệu của bạn đủ lớn (đặc biệt là so với số lượng thuộc tính), nhưng nếu tập dữ liệu không đủ lớn, bạn có nguy cơ gặp phải sai lệch lấy mẫu (*sampling bias*) nghiêm trọng. Khi nhân viên của một công ty khảo sát quyết định gọi điện cho 1.000 người để hỏi một số câu hỏi, họ không chỉ chọn bừa 1.000 người ngẫu nhiên trong danh bạ điện thoại. Họ cố gắng đảm bảo rằng 1.000 người này là đại diện trung thực cho toàn bộ dân số, xét theo các câu hỏi mà họ muốn hỏi. Ví dụ: theo Cục Thống kê Dân số Hoa Kỳ, 51,6% công dân trong độ tuổi bầu cử là nữ, trong khi 48,4% là nam, vì vậy một cuộc khảo sát được tiến hành bài bản tại Hoa Kỳ sẽ cố gắng duy trì tỷ lệ này trong mẫu khảo sát: 516 nữ và 484 nam (ít nhất là nếu câu trả lời có khả năng khác biệt theo giới tính).

Phương pháp này được gọi là lấy mẫu phân tầng (*Stratified Sampling*): tổng thể dân số được chia thành các nhóm con đồng nhất gọi là các tầng (strata), và số lượng mẫu phù hợp sẽ được lấy ra từ mỗi tầng để đảm bảo chắc chắn rằng tập kiểm thử mang tính đại diện cho toàn bộ quần thể dân số. Nếu những người thực hiện khảo sát chỉ sử dụng phương pháp lấy mẫu ngẫu nhiên thuần túy, sẽ có hơn 10% khả năng họ thu về một tập kiểm thử bị lệch với tỷ lệ nữ ít hơn 49% hoặc nhiều hơn 54%. Dù theo hướng nào, kết quả khảo sát chắc chắn sẽ bị thiên lệch rất lớn.

Giả sử bạn đã trao đổi với một số chuyên gia và họ cho bạn biết rằng thu nhập trung vị (`median_income`) là một thuộc tính vô cùng quan trọng để dự đoán giá nhà trung vị. Bạn có thể muốn đảm bảo rằng tập kiểm thử có tính đại diện cao cho các mức thu nhập khác nhau trong toàn bộ tập dữ liệu. Vì thu nhập trung vị là một thuộc tính số liên tục, trước tiên bạn cần phải tạo ra một thuộc tính phân loại thu nhập. Hãy cùng nhìn lại biểu đồ histogram của thu nhập trung vị một cách cẩn thận hơn (xem lại Hình 2-8): hầu hết các giá trị thu nhập trung vị đều tập trung xung quanh mức 1,5 đến 6 (tức là $15.000\,\$$ đến $60.000\,\$$), nhưng một số giá trị thu nhập vượt xa mốc 6.

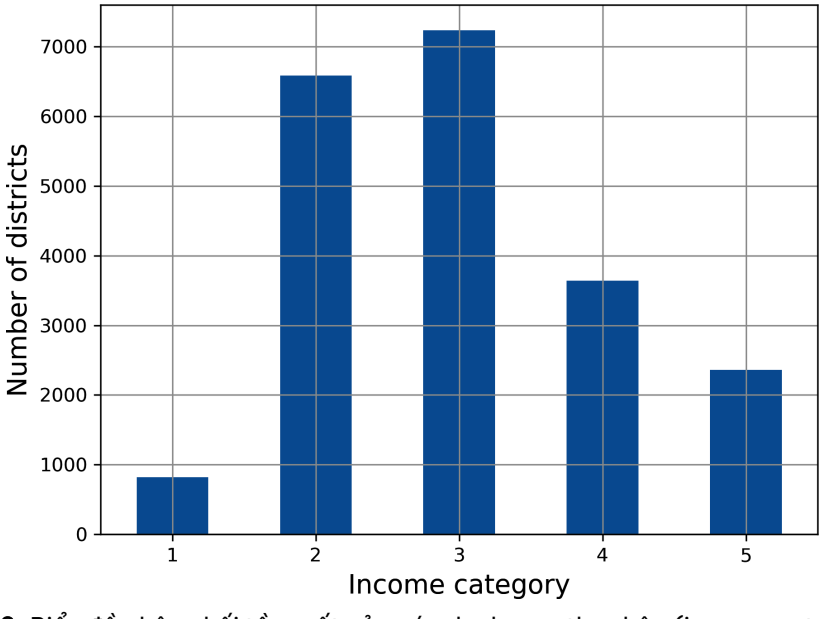
Điều quan trọng là phải có đủ số lượng mẫu trong tập dữ liệu cho từng tầng, nếu không ước tính về tầm quan trọng của một tầng có thể bị thiên lệch. Điều này có nghĩa là bạn không nên chia ra quá nhiều tầng, và mỗi tầng phải đủ lớn. Đoạn mã sau sử dụng hàm `pd.cut()` để tạo một thuộc tính phân loại thu nhập có 5 danh mục (được

gắn nhãn từ 1 đến 5); danh mục 1 nằm trong khoảng từ 0 đến 1,5 (tức là dưới $15.000\,\$$), danh mục 2 từ 1,5 đến 3, và cứ tiếp tục như vậy:

```
housing_full["income_cat"] = pd.cut(housing_full["median_income"],
                                    bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
                                    labels=[1, 2, 3, 4, 5])
```

Các danh mục thu nhập này được thể hiện trong Hình 2-9:

```
cat_counts = housing_full["income_cat"].value_counts().sort_index()
cat_counts.plot.bar(rot=0, grid=True)
plt.xlabel("Income category")
plt.ylabel("Number of districts")
plt.show()
```



Hình 2-9. Biểu đồ phân phối tần suất của các danh mục thu nhập (income categories)

Bây giờ bạn đã sẵn sàng thực hiện việc lấy mẫu phân tầng (Stratified Sampling) dựa trên danh mục thu nhập. Scikit-Learn cung cấp một số lớp phân chia (splitter classes) trong gói `sklearn.model_selection` để triển khai nhiều chiến lược phân tách tập dữ liệu thành tập huấn luyện và tập kiểm thử. Mỗi lớp chia đều có phương thức `split()` trả về một iterator lặp qua các phân chia Train/Test Split khác nhau của cùng một tập dữ liệu.

Chính xác hơn, phương thức `split()` sẽ sinh ra các chỉ số của tập huấn luyện và tập kiểm thử, chứ không trả về trực tiếp chính bản thân dữ liệu. Việc có nhiều phân chia có thể hữu ích nếu bạn muốn ước tính tốt hơn hiệu năng mô hình của mình, như bạn sẽ thấy khi chúng ta thảo luận về Cross-Validation ở phần sau của chương này. Ví dụ: đoạn mã sau tạo ra 10 phân chia phân tầng khác nhau của cùng một tập dữ liệu:

```
from sklearn.model_selection import StratifiedShuffleSplit
```

```
splitter = StratifiedShuffleSplit(n_splits=10, test_size=0.2, random_state=42)
strat_splits = []
for train_index, test_index in splitter.split(housing_full,
                                              housing_full["income_cat"]):
    strat_train_set_n = housing_full.iloc[train_index]
    strat_test_set_n = housing_full.iloc[test_index]
    strat_splits.append([strat_train_set_n, strat_test_set_n])
```

Hiện tại, bạn có thể chỉ cần sử dụng lần phân chia đầu tiên:

```
strat_train_set, strat_test_set = strat_splits[0]
```

Hoặc, vì phương pháp lấy mẫu phân tầng khá phổ biến, có một cách viết ngắn gọn hơn để lấy ra một phân chia duy nhất bằng cách sử dụng hàm `train_test_split()` kết hợp với tham số `stratify`:

```
strat_train_set, strat_test_set = train_test_split(
    housing_full, test_size=0.2, stratify=housing_full["income_cat"],
    random_state=42)
```

Hãy cùng kiểm tra xem cách làm này có hoạt động đúng như mong đợi hay không. Bạn có thể bắt đầu bằng cách xem xét tỷ lệ các danh mục thu nhập trong tập kiểm thử:

```
>>> strat_test_set["income_cat"].value_counts() / len(strat_test_set)
income_cat
3    0.350533
2    0.318798
4    0.176357
5    0.114341
1    0.039971
Name: count, dtype: float64
```

Với đoạn mã tương tự, bạn có thể đo lường tỷ lệ các danh mục thu nhập trong toàn bộ tập dữ liệu gốc. Hình 2-10 so sánh tỷ lệ các danh mục thu nhập trong tổng thể dữ liệu, trong tập kiểm thử được tạo ra bằng phương pháp lấy mẫu phân tầng, và trong tập kiểm thử được tạo bằng phương pháp lấy mẫu ngẫu nhiên thuần túy. Như bạn có thể thấy, tập kiểm thử được sinh ra bằng lấy mẫu phân tầng có tỷ lệ các danh mục thu nhập gần như đồng nhất tuyệt đối với tỷ lệ trong tập dữ liệu đầy đủ, trong khi tập kiểm thử tạo bằng phương pháp ngẫu nhiên thuần túy lại bị lệch đáng kể.

Bạn sẽ không cần sử dụng cột `income_cat` nữa, vì vậy tốt nhất bạn nên loại bỏ nó để hoàn trả dữ liệu về trạng thái ban đầu:

```
for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
```

Chúng ta đã dành khá nhiều thời gian cho việc tạo tập kiểm thử vì một lý do rất xác đáng: đây là một khâu quan trọng nhưng thường hay bị xem nhẹ trong một dự án Machine Learning. Hơn nữa, nhiều ý tưởng ở đây sẽ vô cùng hữu ích sau này khi chúng ta tìm hiểu về Cross-Validation. Bây giờ là lúc chuyển sang giai đoạn tiếp theo: khám phá dữ liệu.

Income Category	Overall %	Stratified %	Random %	Strat. Error %	Rand. Error %
1	3.98	4.00	4.24	0.36	6.45
2	31.88	31.88	30.74	-0.02	-3.59
3	35.06	35.05	34.52	-0.01	-1.53
4	17.63	17.64	18.41	0.03	4.42
5	11.44	11.43	12.09	-0.08	5.63

Hình 2-10. So sánh sai lệch lấy mẫu (sampling bias) giữa phương pháp lấy mẫu phân tầng (stratified sampling) và lấy mẫu ngẫu nhiên thuần túy

Khám phá và Trực quan hóa Dữ liệu để Thu thập Thông tin Chuyên sâu

Cho đến thời điểm này, bạn mới chỉ lướt qua dữ liệu một cách nhanh chóng để có được hiểu biết tổng quan về loại dữ liệu mà mình đang thao tác. Bây giờ, mục tiêu của bạn là đi sâu hơn vào từng chi tiết.

Trước tiên, hãy chắc chắn rằng bạn đã để riêng tập kiểm thử sang một bên và bạn chỉ thực hiện việc khám phá trên tập huấn luyện. Ngoài ra, nếu tập huấn luyện có quy mô rất lớn, bạn có thể muốn lấy mẫu một tập con phục vụ khám phá (exploration set), nhằm giúp các thao tác thử nghiệm diễn ra dễ dàng và nhanh chóng hơn trong giai đoạn này. Trong trường hợp dự án của chúng ta, tập huấn luyện khá nhỏ nên bạn có thể làm việc trực tiếp trên toàn bộ tập huấn luyện. Vì bạn sẽ thử nghiệm với nhiều phép biến đổi khác nhau trên tập huấn luyện, bạn nên tạo một bản sao của tập dữ liệu gốc để có thể dễ dàng hoàn tác hoặc quay lại bất cứ khi nào cần:

```
housing = strat_train_set.copy()
```

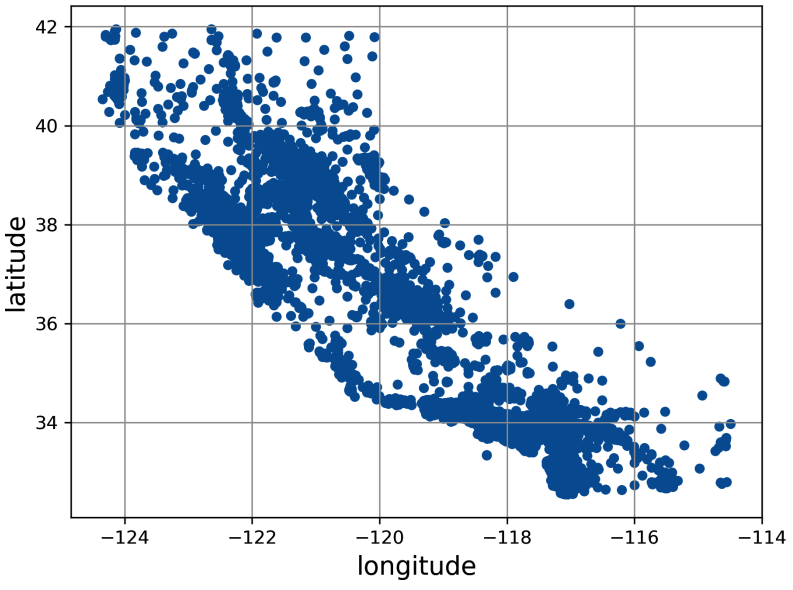
Trực quan hóa Dữ liệu Địa lý (Visualizing Geographical Data)

Vì tập dữ liệu chứa thông tin địa lý (vĩ độ và kinh độ), việc vẽ một biểu đồ phân tán (scatterplot) của tất cả các quận là một ý tưởng rất hay để trực quan hóa dữ liệu (Hình 2-11):

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True)
plt.show()
```

Hình dáng này trông đúng là bang California rồi, nhưng ngoài điều đó ra thì thật khó để nhận thấy bất kỳ mô thức quy luật đặc biệt nào. Việc thiết lập tùy chọn độ trong suốt `alpha=0.2` giúp bạn dễ dàng trực quan hóa hơn rất nhiều những khu vực có mật độ điểm dữ liệu tập trung cao (Hình 2-12):

```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True, alpha=0.2)
```



Hình 2-11. Biểu đồ phân tán địa lý của dữ liệu

```
plt.show()
```

Bây giờ biểu đồ trông đã tốt hơn rất nhiều: bạn có thể nhìn thấy rõ ràng các khu vực có mật độ tập trung cao, cụ thể là Vùng Vịnh (Bay Area), khu vực xung quanh Los Angeles và San Diego, cùng với một dải dài các khu vực mật độ khá cao ở Thung lũng Trung tâm (Central Valley, đặc biệt là quanh Sacramento và Fresno).

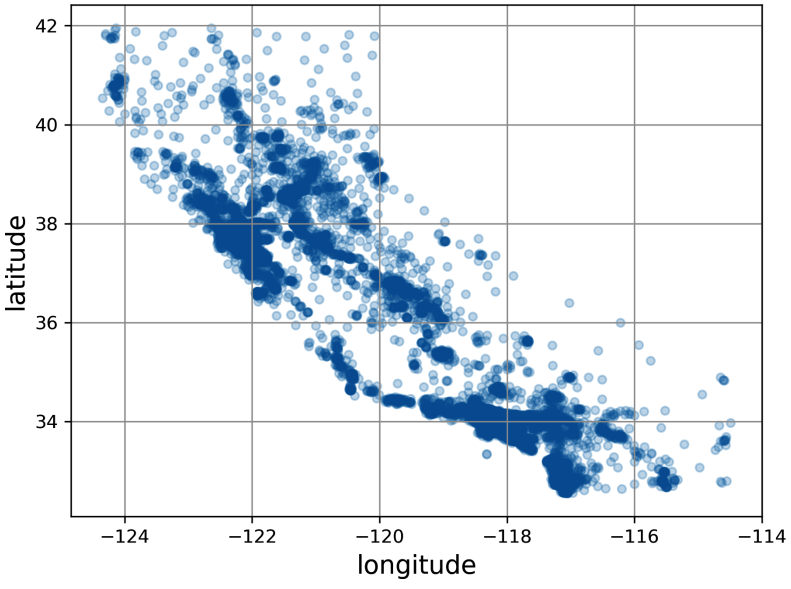
Bộ não của con người rất nhạy bén trong việc phát hiện các quy luật mẫu từ hình ảnh, nhưng bạn có thể cần phải tinh chỉnh linh hoạt các tham số trực quan hóa để làm cho các quy luật này nổi bật lên một cách rõ nét nhất.

Tiếp theo, bạn quan sát đến giá nhà (Hình 2-13). Bán kính của mỗi hình tròn đại diện cho dân số của quận đó (thông qua tùy chọn `s`), và màu sắc đại diện cho mức giá (tùy chọn `c`). Ở đây, bạn sử dụng một thang màu được xác định trước (tùy chọn `cmap`) có tên là `jet`, chuyển dải màu từ xanh lam (giá trị thấp) sang đỏ (giá trị cao):[8]

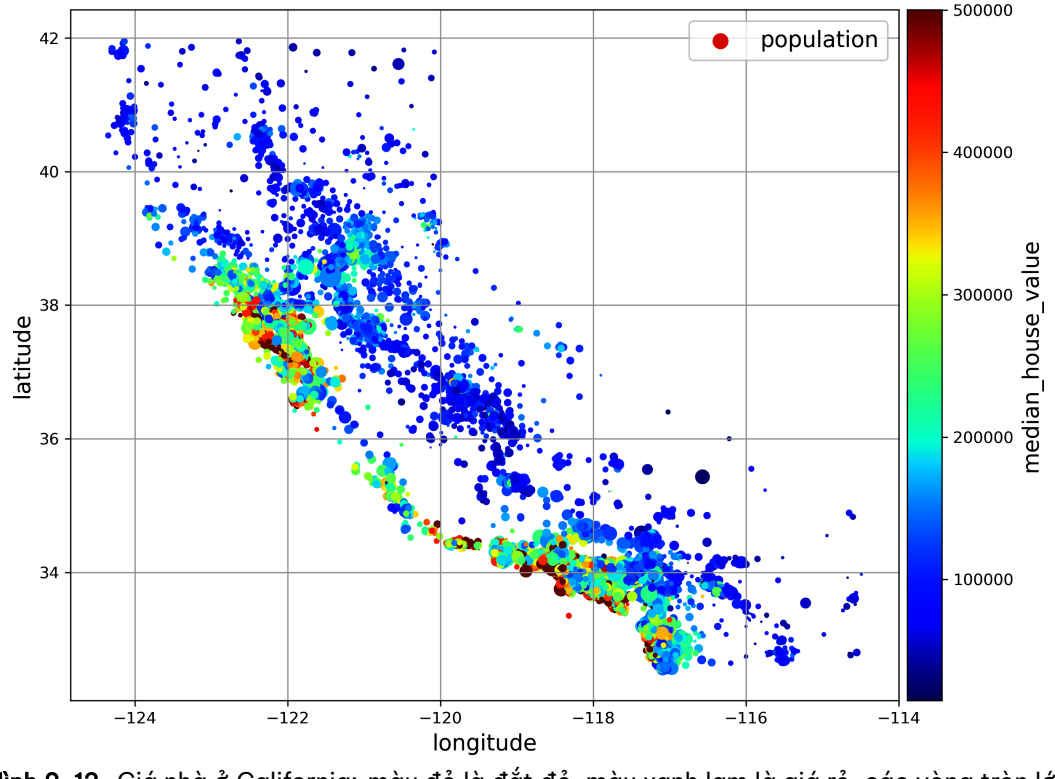
```
housing.plot(kind="scatter", x="longitude", y="latitude", grid=True,
             s=housing["population"] / 100, label="population",
             c="median_house_value", cmap="jet", colorbar=True,
             legend=True, sharex=False, figsize=(10, 7))
plt.show()
```

Bức tranh trực quan này cho bạn thấy rằng giá nhà có mối liên hệ rất chặt chẽ với vị trí địa lý (chẳng hạn như vị trí gần đại dương) và mật độ dân số, như có lẽ bạn cũng đã tự phán đoán từ trước. Một thuật toán phân cụm (clustering algorithm) sẽ rất hữu ích trong việc phát hiện cụm chính và thêm các đặc trưng mới đo lường khoảng cách tương đối tới các tâm cụm này. Thuộc tính khoảng cách tới đại dương (

[8] Nếu bạn đang đọc bản in đen trắng này, hãy cầm một cây bút màu đỏ và tô nhẹ dọc theo phần lớn bờ biển từ Vùng Vịnh kéo dài xuống tận San Diego (đúng như dự đoán của bạn). Bạn cũng có thể tô thêm một vệt màu vàng quanh khu vực Sacramento.



Hình 2-12. Một cách trực quan hóa tốt hơn làm nổi bật các khu vực có mật độ dân cư cao



Hình 2-13. Giá nhà ở California: màu đỏ là đắt đỏ, màu xanh lam là giá rẻ, các vòng tròn lớn hơn biểu thị những khu vực có dân số đông đúc hơn

`ocean_proximity`) cũng có thể mang lại nhiều thông tin giá trị, mặc dù ở phía Bắc California, giá nhà ở các quận ven biển không quá cao, do đó đây không phải là một quy tắc đơn giản tuyến tính.

Tìm kiếm Mối Tương quan (Look for Correlations)

Vì tập dữ liệu không quá lớn, bạn có thể dễ dàng tính toán hệ số tương quan tiêu chuẩn (standard correlation coefficient, còn gọi là hệ số r của Pearson) giữa từng cặp thuộc tính dạng số bằng phương thức `corr()`:

```
corr_matrix = housing.corr(numeric_only=True)
```

Bây giờ bạn có thể quan sát xem từng thuộc tính tương quan như thế nào với giá trị nhà trung vị:

```
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value    1.000000
median_income         0.688380
total_rooms           0.137455
housing_median_age    0.102175
households            0.071426
total_bedrooms        0.054635
population           -0.020153
longitude            -0.050859
latitude             -0.139584
Name: median_house_value, dtype: float64
```

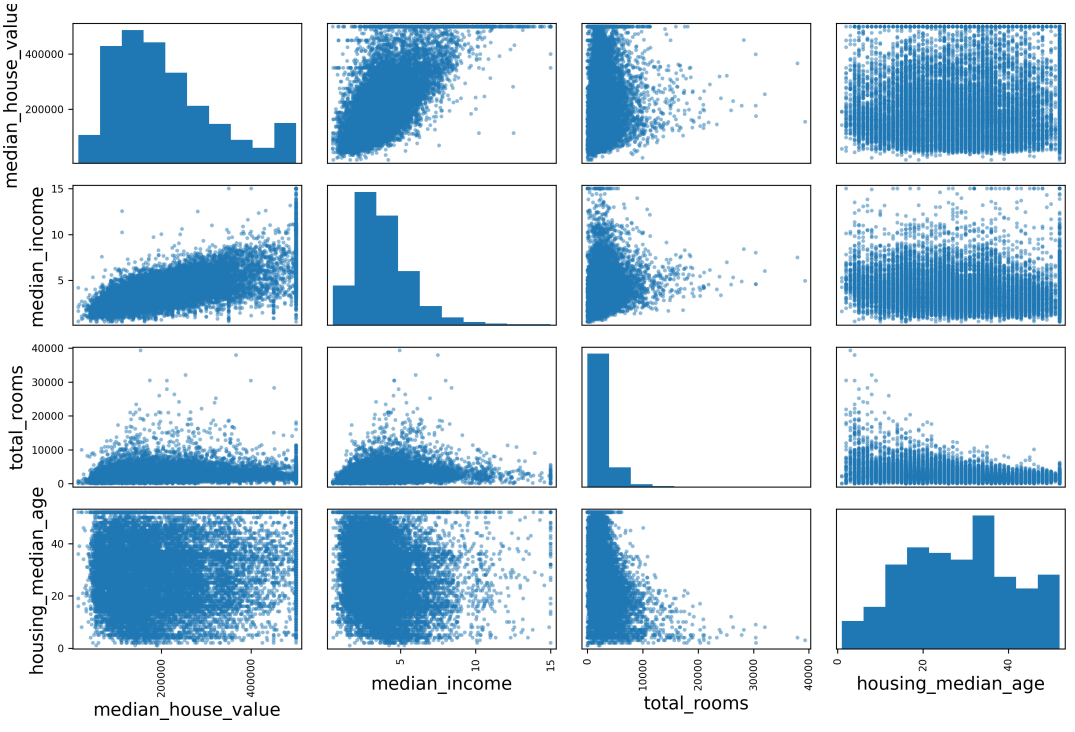
Hệ số tương quan dao động trong khoảng từ -1 đến 1. Khi giá trị gần bằng 1, điều đó có nghĩa là có một mối tương quan thuận mạnh mẽ; ví dụ: giá trị nhà trung vị có xu hướng tăng lên khi thu nhập trung vị tăng. Khi hệ số gần bằng -1, điều đó có nghĩa là có một tương quan nghịch mạnh; bạn có thể nhận thấy một tương quan nghịch nhỏ giữa vĩ độ (`latitude`) và giá trị nhà trung vị (nghĩa là giá nhà có xu hướng giảm nhẹ khi bạn đi dần về phía Bắc). Cuối cùng, các hệ số gần bằng 0 biểu thị rằng không có mối tương quan tuyến tính nào giữa hai biến số.

Một cách khác để kiểm tra mối tương quan giữa các thuộc tính là sử dụng hàm `scatter_matrix()` của Pandas, hàm này sẽ vẽ biểu đồ tương quan của từng thuộc tính số với tất cả các thuộc tính số khác. Vì hiện tại chúng ta có 9 thuộc tính số, bạn sẽ nhận được $9^2 = 81$ biểu đồ con, không thể nào vừa vặn trên một trang sách – do đó bạn quyết định tập trung vào một vài thuộc tính đầy hứa hẹn nhất có vẻ tương quan mạnh nhất với giá trị nhà trung vị (Hình 2-14):

```
from pandas.plotting import scatter_matrix

attributes = ["median_house_value", "median_income", "total_rooms",
              "housing_median_age"]
scatter_matrix(housing[attributes], figsize=(12, 8))
plt.show()
```

Đường chéo chính sẽ chỉ toàn các đường thẳng nếu Pandas vẽ từng biến với chính nó, điều này chẳng mang lại thông tin hữu ích nào. Vì vậy, thay vào đó, Pandas tự

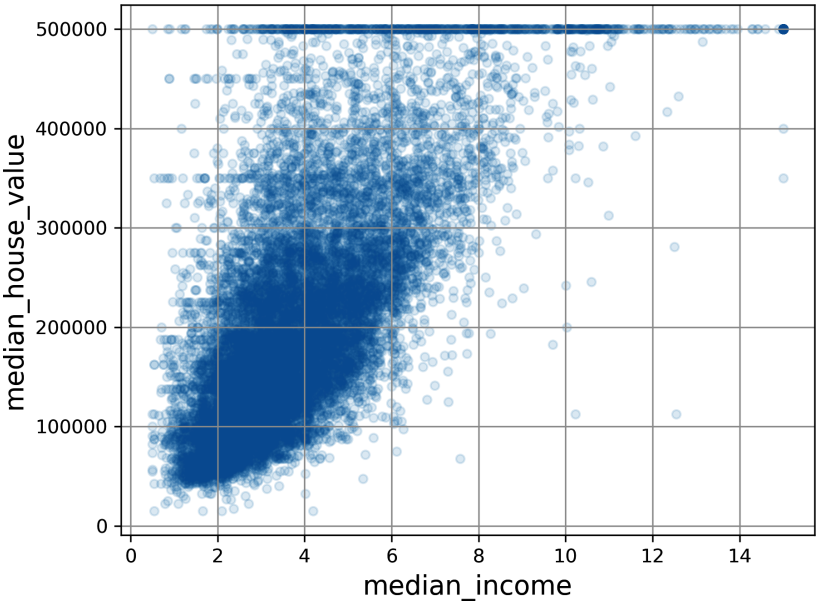


Hình 2-14. Ma trận biểu đồ phân tán này vẽ từng thuộc tính dạng số với mọi thuộc tính dạng số khác, kết hợp với biểu đồ histogram của từng thuộc tính trên đường chéo chính (từ góc trên cùng bên trái xuống góc dưới cùng bên phải)

động hiển thị biểu đồ phân phối tần suất (histogram) của từng thuộc tính (có các tùy chọn khác; xem tài liệu của Pandas để biết thêm chi tiết).

Nhìn vào các biểu đồ phân tán tương quan, có vẻ như thuộc tính triển vọng nhất để dự đoán giá trị nhà trung vị là thu nhập trung vị (`median_income`), vì vậy bạn phóng to vào biểu đồ phân tán đó (Hình 2-15):

```
housing.plot(kind="scatter", x="median_income", y="median_house_value",
             alpha=0.1, grid=True)
plt.show()
```

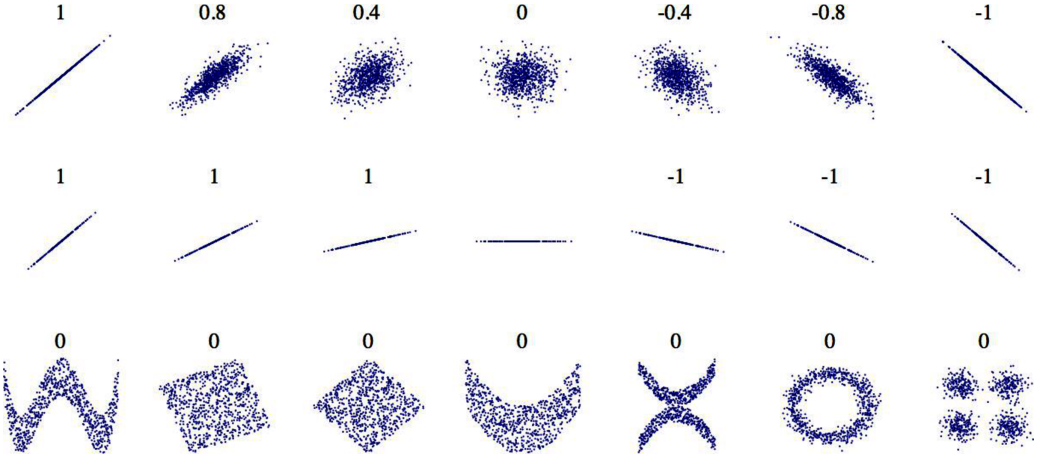


Hình 2-15. Thu nhập trung vị so với giá trị nhà trung vị

Biểu đồ này tiết lộ một vài điều thú vị. Thứ nhất, mối tương quan thực sự khá mạnh; bạn có thể thấy rõ xu hướng đi lên, mặc dù dữ liệu có độ nhiễu nhất định. Thứ hai, mức giá trần mà bạn nhận thấy trước đó hiện rõ mồn một dưới dạng một đường thẳng nằm ngang ở mức $500.000\,\$$. Tuy nhiên, biểu đồ này cũng để lộ các đường thẳng nằm ngang khác ít rõ ràng hơn: một đường nằm ngang ở khoảng $450.000\,\$$, một đường khác ở khoảng $350.000\,\$$, có lẽ một đường nữa quanh mức $280.000\,\$$, và một vài đường bên dưới mức đó. Bạn có thể muốn thử loại bỏ các quận tương ứng để ngăn chặn các thuật toán Machine Learning của mình học cách tái tạo lại những điểm bất thường kỳ dị này của dữ liệu.

Lưu ý rằng hệ số tương quan chỉ đo lường các mối tương quan tuyến tính ("khi x tăng, y nhìn chung tăng/giảm"). Nó có thể hoàn toàn bỏ sót các mối quan hệ phi tuyến tính (chẳng hạn như "khi x tiến dần về 0, y nhìn chung tăng"). Hình 2-16 minh họa một loạt các tập dữ liệu khác nhau cùng với hệ số tương quan tương ứng của chúng. Hãy chú ý cách tất cả các biểu đồ ở hàng dưới cùng đều có hệ số tương quan bằng 0, mặc dù các trục tọa độ của chúng rõ ràng không hề độc lập: đây là những ví dụ điển hình về các mối quan hệ phi tuyến. Ngoài ra, hàng thứ hai hiển thị các ví dụ mà hệ số tương quan bằng 1 hoặc -1; lưu ý rằng điều này hoàn toàn không liên quan gì đến độ dốc

của đường thẳng. Ví dụ: chiều cao của bạn tính bằng inch có hệ số tương quan bằng 1 với chiều cao tính bằng feet hoặc nanometer.



Hình 2-16. Hệ số tương quan tiêu chuẩn của nhiều bộ dữ liệu khác nhau (nguồn: Wikipedia; miền công cộng)

Thử nghiệm với các Tổ hợp Thuộc tính (Attribute Combinations)

Hy vọng rằng các phần trước đã gợi mở cho bạn một số ý tưởng về các cách thức bạn có thể khám phá dữ liệu và thu thập những hiểu biết sâu sắc. Bạn đã xác định được một số điểm kỳ lạ của dữ liệu mà bạn có thể muốn làm sạch trước khi đưa vào thuật toán Machine Learning, và bạn đã tìm thấy những mối tương quan thú vị giữa các thuộc tính, đặc biệt là với thuộc tính mục tiêu. Bạn cũng nhận thấy rằng một số thuộc tính có phân phối lệch phải, vì vậy bạn có thể muốn biến đổi chúng (ví dụ: bằng cách tính logarit hoặc căn bậc hai). Tất nhiên, trải nghiệm thực tế của bạn sẽ khác biệt đáng kể theo từng dự án cụ thể, nhưng các ý tưởng cốt lõi thì luôn tương tự nhau.

Một điều cuối cùng bạn có thể muốn làm trước khi chuẩn bị dữ liệu cho các thuật toán Machine Learning là thử nghiệm các tổ hợp thuộc tính khác nhau. Ví dụ: tổng số phòng trong một quận không mang lại nhiều ý nghĩa nếu bạn không biết có bao nhiêu hộ gia đình sinh sống ở đó. Điều bạn thực sự quan tâm là số phòng trên mỗi hộ gia đình (`rooms_per_house`). Tương tự như vậy, chỉ riêng tổng số phòng ngủ thì không mang lại nhiều thông tin: bạn có thể muốn so sánh nó với tổng số phòng (`bedrooms_ratio`). Và dân số trên mỗi hộ gia đình (`people_per_house`) dường như cũng là một tổ hợp thuộc tính rất đáng chú ý. Bạn tạo các thuộc tính mới này như sau:

```
housing["rooms_per_house"] = housing["total_rooms"] / housing["households"]
housing["bedrooms_ratio"] = housing["total_bedrooms"] / housing["total_rooms"]
housing["people_per_house"] = housing["population"] / housing["households"]
```

Và sau đó bạn kiểm tra lại ma trận tương quan một lần nữa:

```
>>> corr_matrix = housing.corr(numeric_only=True)
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value    1.000000
```

```
median_income          0.688380
rooms_per_house        0.143663
total_rooms            0.137455
housing_median_age     0.102175
households             0.071426
total_bedrooms         0.054635
population            -0.020153
people_per_house      -0.038224
longitude             -0.050859
latitude              -0.139584
bedrooms_ratio        -0.256397
Name: median_house_value, dtype: float64
```

Chà, không tệ chút nào! Thuộc tính mới `bedrooms_ratio` có mối tương quan mạnh mẽ hơn nhiều với giá trị nhà trung vị so với tổng số phòng hay tổng số phòng ngủ đơn lẻ. Đó là một tương quan nghịch mạnh, cho thấy những ngôi nhà có tỷ lệ phòng ngủ trên tổng số phòng thấp hơn thường có xu hướng đắt đỏ hơn. Số lượng phòng trên mỗi hộ gia đình cũng cung cấp nhiều thông tin hơn tổng số phòng của một quận – rõ ràng ngôi nhà càng lớn thì giá trị của nó càng đắt đỏ.



Khi tạo các đặc trưng kết hợp mới, hãy đảm bảo rằng chúng không có mối tương quan tuyến tính quá cao với các đặc trưng hiện có: hiện tượng cộng tuyến (collinearity) có thể gây ra sự cố cho một số mô hình, chẳng hạn như Hồi quy Tuyến tính (Linear Regression). Đặc biệt, hãy tránh các phép tính tổng có trọng số đơn giản từ các đặc trưng sẵn có.

Vòng khám phá dữ liệu này không nhất thiết phải đạt đến mức độ toàn diện tuyệt đối; mục đích ở đây là có được một sự khởi đầu thuận lợi và nhanh chóng thu thập những thông tin cần thiết giúp bạn xây dựng một bản nguyên mẫu (prototype) ban đầu tương đối tốt. Nhưng hãy nhớ rằng đây là một quy trình mang tính lặp đi lặp lại (*iterative process*): một khi bạn đã xây dựng và vận hành được một nguyên mẫu, bạn có thể phân tích đầu ra của nó để có thêm những hiểu biết sâu sắc mới và quay trở lại bước khám phá dữ liệu này bất cứ lúc nào.

Chuẩn bị Dữ liệu cho các Thuật toán Machine Learning

Đã đến lúc chuẩn bị dữ liệu cho các thuật toán Machine Learning của bạn. Thay vì thực hiện các thao tác này một cách thủ công, bạn nên viết các hàm chuyên biệt cho mục đích này, vì một số lý do rất xác đáng:

- Điều này sẽ cho phép bạn tái tạo lại các phép biến đổi này một cách dễ dàng trên bất kỳ tập dữ liệu nào (ví dụ: vào lần tới khi bạn nhận được một tập dữ liệu mới).
- Bạn sẽ dần dần xây dựng được một thư viện gồm các hàm biến đổi hữu ích mà bạn có thể tái sử dụng trong các dự án tương lai.
- Bạn có thể sử dụng các hàm này trong hệ thống đang hoạt động thực tế (live system) để biến đổi dữ liệu mới trước khi nạp vào các thuật toán của mình.

- Điều này sẽ giúp bạn dễ dàng thử nghiệm nhiều phép biến đổi khác nhau và quan sát xem tổ hợp biến đổi nào mang lại hiệu quả tốt nhất.

Nhưng trước tiên, hãy quay trở lại với một tập huấn luyện sạch ban đầu (bằng cách sao chép lại `strat_train_set` một lần nữa). Bạn cũng nên tách biệt hoàn toàn giữa các biến dự đoán (predictors) và các nhãn mục tiêu (labels), vì bạn không nhất thiết muốn áp dụng các phép biến đổi giống hệt nhau lên cả hai phần này (lưu ý rằng phương thức `drop()` tạo ra một bản sao của dữ liệu và không làm ảnh hưởng đến `strat_train_set`):

```
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()
```

Làm sạch Dữ liệu (Clean the Data)

Hầu hết các thuật toán Machine Learning không thể làm việc được với các đặc trưng bị thiếu giá trị, vì vậy bạn sẽ cần phải xử lý chúng một cách thấu đáo. Ví dụ: bạn đã nhận thấy trước đó rằng thuộc tính `total_bedrooms` có một số giá trị bị thiếu. Bạn có ba lựa chọn để giải quyết vấn đề này:

1. Loại bỏ các quận tương ứng có chứa giá trị bị thiếu.
2. Loại bỏ hoàn toàn thuộc tính đó khỏi tập dữ liệu.
3. Điền vào các giá trị bị thiếu bằng một giá trị nhất định (số 0, giá trị trung bình mean, giá trị trung vị median, v.v.). Thao tác này được gọi là phép gán giá trị khuyết (*imputation*).

Bạn có thể thực hiện các tùy chọn này một cách dễ dàng bằng cách sử dụng các phương thức `dropna()`, `drop()`, và `fillna()` của Pandas DataFrame:

```
housing.dropna(subset=["total_bedrooms"], inplace=True)  # option 1
housing.drop("total_bedrooms", axis=1, inplace=True)     # option 2
median = housing["total_bedrooms"].median()              # option 3
housing["total_bedrooms"] = housing["total_bedrooms"].fillna(median)
```

Bạn quyết định chọn tùy chọn 3 vì nó ít mang tính phá hủy dữ liệu nhất, nhưng thay vì sử dụng đoạn mã ở trên, bạn sẽ dùng một lớp rất tiện lợi của Scikit-Learn: `SimpleImputer`. Lợi ích của nó là nó sẽ lưu trữ lại giá trị trung vị của từng đặc trưng: điều này giúp bạn có thể điền giá trị khuyết không chỉ trên tập huấn luyện mà còn trên cả tập kiểm định (validation set), tập kiểm thử (test set) và bất kỳ dữ liệu mới nào được đưa vào mô hình sau này. Để sử dụng nó, trước tiên bạn cần tạo một thực thể `SimpleImputer`, chỉ định rằng bạn muốn thay thế các giá trị bị thiếu của mỗi thuộc tính bằng giá trị trung vị của thuộc tính đó:

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
```

Vì giá trị trung vị chỉ có thể được tính toán trên các thuộc tính dạng số, tiếp theo bạn cần tạo một bản sao của dữ liệu chỉ chứa các thuộc tính số (điều này sẽ loại trừ thuộc tính dạng văn bản `ocean_proximity`):

```
housing_num = housing.select_dtypes(include=[np.number])
```

Bây giờ bạn có thể khớp (fit) thực thể imputer này vào dữ liệu huấn luyện bằng phương thức `fit()`:

```
imputer.fit(housing_num)
```

Đối tượng `imputer` chỉ đơn giản là tính toán giá trị trung vị của từng thuộc tính số và lưu trữ kết quả trong biến thực thể `statistics_`. Chỉ có thuộc tính `total_bedrooms` là có giá trị bị thiếu, nhưng bạn không thể biết trước liệu có bất kỳ giá trị nào bị thiếu trong dữ liệu mới sau khi hệ thống được triển khai hay không, vì vậy việc áp dụng imputer cho tất cả các thuộc tính dạng số là một thói quen thực hành an toàn hơn:

```
>>> imputer.statistics_
array([-118.51 ,   34.26 ,   29.   , 2125.   ,  434.   , 1167.   ,
        408.   ,    3.5385])
```

Bạn có thể kiểm tra xem các giá trị này có hoàn toàn trùng khớp với việc tính toán thủ công trung vị của từng thuộc tính hay không:

```
>>> housing_num.median().values
array([-118.51 ,   34.26 ,   29.   , 2125.   ,  434.   , 1167.   ,
        408.   ,    3.5385])
```

Bây giờ bạn có thể sử dụng `imputer` đã được huấn luyện này để biến đổi tập huấn luyện bằng cách thay thế các giá trị bị thiếu bằng các giá trị trung vị đã học được:

```
X = imputer.transform(housing_num)
```

Các giá trị bị thiếu cũng có thể được thay thế bằng giá trị trung bình (`strategy="mean"`), hoặc bằng giá trị xuất hiện thường xuyên nhất (`strategy="most_frequent"`), hoặc bằng một giá trị hằng số (`strategy="constant", fill_value=…`). Hai chiến lược sau hỗ trợ dữ liệu phi số.



Ngoài ra còn có các imputer (bộ gán giá trị khuyết) mạnh mẽ hơn trong gói `sklearn.impute` (cả hai đều chỉ dành cho các đặc trưng số):

- `KNNImputer` thay thế từng giá trị bị thiếu bằng giá trị trung bình của k hàng xóm gần nhất cho đặc trưng đó. Khoảng cách được tính toán dựa trên tất cả các đặc trưng có sẵn.
- `IterativeImputer` huấn luyện một mô hình hồi quy cho từng đặc trưng để dự đoán các giá trị bị thiếu dựa trên tất cả các đặc trưng khác có sẵn. Sau đó, nó huấn luyện lại mô hình trên dữ liệu đã được cập nhật, và lặp lại quy trình này nhiều lần, liên tục cải thiện mô hình và các giá trị thay thế sau mỗi vòng lặp.

Triết lý Thiết kế của Scikit-Learn (Scikit-Learn Design)

Giao diện lập trình ứng dụng (API) của Scikit-Learn được thiết kế vô cùng tinh tế và nhất quán. Dưới đây là các nguyên tắc thiết kế chính:[9]

- **Tính nhất quán (Consistency):** Tất cả các đối tượng đều chia sẻ một giao diện chung và tối giản:
  - *Bộ ước lượng (Estimators):* Bất kỳ đối tượng nào có thể ước lượng một số tham số dựa trên tập dữ liệu đều được gọi là một estimator (ví dụ: `SimpleImputer` là một estimator). Quá trình ước lượng được thực hiện thông qua phương thức `fit()`, và nó chỉ nhận một tham số là tập dữ liệu (hoặc hai đối với các thuật toán học có giám sát; tập dữ liệu thứ hai chứa các nhãn). Bất kỳ tham số nào khác hướng dẫn quá trình ước lượng đều được coi là một siêu tham số (hyperparameter), và nó phải được thiết lập như một tham số hàm khởi tạo (thường có giá trị mặc định hợp lý).
  - *Bộ biến đổi (Transformers):* Một số estimator (chẳng hạn như `SimpleImputer`) cũng có thể biến đổi một tập dữ liệu; chúng được gọi là các transformer. Giao diện rất đơn giản: việc biến đổi được thực hiện bằng phương thức `transform()` với tập dữ liệu cần biến đổi được truyền vào như một tham số. Nó trả về tập dữ liệu đã được biến đổi. Tất cả các transformer cũng có một phương thức tiện lợi gọi là `fit_transform()`, tương đương với việc gọi `fit()` rồi sau đó gọi `transform()` (nhưng đôi khi `fit_transform()` được tối ưu hóa để chạy nhanh hơn đáng kể).
  - *Bộ dự đoán (Predictors):* Cuối cùng, một số estimator có khả năng đưa ra dự đoán dựa trên một tập dữ liệu; chúng được gọi là các predictor. Một predictor có một phương thức `predict()` nhận một tập dữ liệu gồm các mẫu mới và trả về một tập dữ liệu gồm các dự đoán tương ứng. Nó cũng có một phương thức `score()` đo lường chất lượng của các dự đoán trên một tập kiểm thử nhất định (kèm theo các nhãn tương ứng trong trường hợp học có giám sát).[10]
- **Khả năng kiểm tra nội tại (Inspection):** Tất cả các hyperparameter của một estimator đều có thể được truy cập trực tiếp thông qua các biến thực thể công khai (public instance variables, ví dụ: `imputer.strategy`), và tất cả các tham số đã học được của estimator đều có thể được truy cập thông qua các biến thực thể công khai có dấu gạch dưới ở cuối (ví dụ: `imputer.statistics_`).
- **Không nhân rộng các lớp (Nonproliferation of classes):** Các tập dữ liệu được biểu diễn dưới dạng mảng NumPy hoặc ma trận thưa SciPy, chứ không phải các lớp tự tạo. Các hyperparameter chỉ là các chuỗi ký tự Python thông thường hoặc các số.
- **Khả năng kết hợp (Composition):** Các khối xây dựng hiện có được tái sử dụng tối đa. Ví dụ: bạn có thể dễ dàng tạo một chuỗi pipeline gồm nhiều transformer theo sau là một estimator.
- **Các giá trị mặc định hợp lý (Sensible defaults):** Scikit-Learn cung cấp các giá trị mặc định hợp lý cho hầu hết các tham số, giúp bạn dễ dàng nhanh chóng tạo ra

[9] Để biết thêm chi tiết về các nguyên tắc thiết kế, xem Lars Buitinck và cộng sự, "API Design for Machine Learning Software: Experiences from the Scikit-Learn Project", arXiv preprint arXiv:1309.0238 (2013).

[10] Một số bộ dự đoán cũng cung cấp các phương thức để đo lường độ tin cậy của các dự đoán của chúng.

một hệ thống cơ sở hoạt động được.

Các transformer của Scikit-Learn xuất ra các mảng NumPy (hoặc đôi khi là ma trận thưa SciPy) ngay cả khi chúng nhận đầu vào là các Pandas DataFrame.[11] Vì vậy, đầu ra của `imputer.transform(housing_num)` là một mảng NumPy: `X` không có tên cột cũng như không có chỉ mục (index). Rất may mắn là việc bao bọc `X` trong một DataFrame và khôi phục tên cột cùng chỉ mục từ `housing_num` không quá khó khăn:

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,
                          index=housing_num.index)
```

Xử lý các Thuộc tính Dạng Văn bản và Phân loại

Cho đến nay, chúng ta mới chỉ xử lý các thuộc tính dạng số, nhưng dữ liệu của bạn cũng có thể chứa các thuộc tính dạng văn bản. Trong tập dữ liệu này, chỉ có duy nhất một thuộc tính như vậy: thuộc tính khoảng cách tới đại dương (`ocean_proximity`). Hãy cùng xem giá trị của nó ở một vài mẫu đầu tiên:

```
>>> housing_cat = housing[["ocean_proximity"]]
>>> housing_cat.head(8)
      ocean_proximity
13096        NEAR BAY
14973       <1H OCEAN
3785           INLAND
14689          INLAND
20507      NEAR OCEAN
1286           INLAND
18078       <1H OCEAN
4396         NEAR BAY
```

Đây không phải là văn bản tự do tùy ý: chỉ có một số lượng hữu hạn các giá trị có thể xuất hiện, mỗi giá trị đại diện cho một danh mục. Vì vậy, thuộc tính này là một thuộc tính phân loại (categorical attribute). Hầu hết các thuật toán Machine Learning đều thích làm việc với các con số hơn, vì vậy hãy chuyển đổi các danh mục này từ dạng văn bản sang dạng số. Để làm điều này, chúng ta có thể sử dụng lớp `OrdinalEncoder` của Scikit-Learn:

```
from sklearn.preprocessing import OrdinalEncoder

ordinal_encoder = OrdinalEncoder()
housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)
```

Dưới đây là một vài giá trị đã mã hóa đầu tiên trong `housing_cat_encoded`:

```
>>> housing_cat_encoded[:8]
array([[3.],
       [0.],
       [1.],
```

[11] Nếu bạn chạy `sklearn.set_config(transform_output="pandas")`, tất cả các transformer sẽ xuất ra Pandas DataFrame khi chúng nhận đầu vào là một DataFrame: Pandas in, Pandas out.

```
       [1.],
       [4.],
       [1.],
       [0.],
       [3.]])
```

Bạn có thể lấy danh sách các danh mục bằng cách sử dụng biến thực thể `categories_`:

```
>>> ordinal_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
       dtype=object)]
```

Một vấn đề lớn với biểu diễn này là các thuật toán Machine Learning sẽ mặc định giả định rằng hai giá trị gần nhau thì tương đồng hơn hai giá trị ở xa nhau. Điều này có thể ổn trong một số trường hợp (ví dụ: đối với các danh mục có thứ bậc như "kém", "trung bình", "tốt", và "xuất sắc"), nhưng rõ ràng không phải là trường hợp của cột `ocean_proximity` (ví dụ: danh mục 0 và 4 rõ ràng tương đồng với nhau hơn nhiều so với danh mục 0 và 1).

Để khắc phục nhược điểm này, một giải pháp phổ biến là tạo ra một thuộc tính nhị phân cho mỗi danh mục: một thuộc tính bằng 1 khi danh mục là "<1H OCEAN" (và bằng 0 trong các trường hợp khác), một thuộc tính khác bằng 1 khi danh mục là "INLAND" (và bằng 0 trong các trường hợp khác), và cứ tiếp tục như vậy. Kỹ thuật này được gọi là mã hóa One-Hot (One-Hot Encoding), bởi vì chỉ có duy nhất một thuộc tính bằng 1 (hot), trong khi tất cả các thuộc tính còn lại đều bằng 0 (cold). Các thuộc tính mới đôi khi được gọi là các thuộc tính giả (*dummy attributes*). Scikit-Learn cung cấp lớp `OneHotEncoder` để chuyển đổi các giá trị phân loại thành các vector one-hot:

```
from sklearn.preprocessing import OneHotEncoder

cat_encoder = OneHotEncoder()
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
```

Theo mặc định, đầu ra của `OneHotEncoder` là một ma trận thưa (*sparse matrix*) của SciPy, chứ không phải là một mảng NumPy:

```
>>> housing_cat_1hot
<Compressed Sparse Row sparse matrix of dtype 'float64'
 with 16512 stored elements and shape (16512, 5)>
```

Một ma trận thưa là một biểu diễn cực kỳ hiệu quả cho các ma trận chứa hầu hết toàn số 0. Thực chất, ở bên trong, nó chỉ lưu trữ các giá trị khác không và vị trí tương ứng của chúng. Khi một thuộc tính phân loại có hàng trăm hoặc hàng nghìn danh mục, việc mã hóa one-hot sẽ tạo ra một ma trận khổng lồ chứa toàn số 0 ngoại trừ một số 1 duy nhất trên mỗi hàng. Trong trường hợp này, ma trận thưa chính xác là những gì bạn cần: nó tiết kiệm rất nhiều bộ nhớ và tăng tốc độ tính toán. Bạn có thể sử dụng ma trận thưa gần giống như một mảng 2D thông thường,[12] nhưng nếu bạn muốn chuyển đổi nó thành một mảng NumPy dày đặc (dense array), bạn chỉ cần gọi phương thức `toarray()`:

[12] Xem tài liệu hướng dẫn của SciPy để biết thêm chi tiết.

```
>>> housing_cat_1hot.toarray()
array([[0., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [0., 0., 0., 0., 1.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.]])
```

Ngoài ra, bạn có thể đặt tham số `sparse_output=False` khi khởi tạo `OneHotEncoder`, trong trường hợp đó phương thức `transform()` sẽ trực tiếp trả về một mảng NumPy dày đặc thông thường:

```
cat_encoder = OneHotEncoder(sparse_output=False)
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
```

Tương tự như với `OrdinalEncoder`, bạn có thể lấy danh sách các danh mục thông qua biến thực thể `categories_` của bộ mã hóa:

```
>>> cat_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
       dtype=object)]
```

Pandas có một hàm gọi là `get_dummies()`, hàm này cũng chuyển đổi từng đặc trưng phân loại thành biểu diễn one-hot, với một đặc trưng nhị phân cho mỗi danh mục:

```
>>> df_test = pd.DataFrame({"ocean_proximity": ["INLAND", "NEAR BAY"]})
>>> pd.get_dummies(df_test)
   ocean_proximity_INLAND  ocean_proximity_NEAR BAY
0                    True                     False
1                   False                      True
```

Trông nó có vẻ rất gọn gàng và đơn giản, vậy tại sao không sử dụng nó thay cho `OneHotEncoder`? Lợi thế vượt trội của `OneHotEncoder` là nó ghi nhớ chính xác những danh mục nào mà nó đã được huấn luyện. Điều này cực kỳ quan trọng vì một khi mô hình của bạn đi vào môi trường sản xuất (production), nó phải được nạp chính xác các đặc trưng giống hệt như trong quá trình huấn luyện: không thừa, không thiếu. Hãy xem đối tượng `cat_encoder` đã qua huấn luyện của chúng ta xuất ra kết quả gì khi chúng ta yêu cầu nó biến đổi chính `df_test` đó (bằng phương thức `transform()`, chứ không phải `fit_transform()`):

```
>>> cat_encoder.transform(df_test)
array([[0., 1., 0., 0., 0.],
       [0., 0., 0., 1., 0.]])
```

Bạn có thấy sự khác biệt không? Hàm `get_dummies()` chỉ nhìn thấy hai danh mục nên nó chỉ xuất ra hai cột, trong khi `OneHotEncoder` xuất ra đúng một cột cho mỗi danh mục đã học được trong quá trình huấn luyện, theo đúng thứ tự đã định. Hơn nữa, nếu bạn nạp cho `get_dummies()` một DataFrame chứa một danh mục hoàn toàn mới chưa từng thấy (ví dụ: “<2H OCEAN”), nó sẽ tạo thêm một cột mới cho danh mục đó:

```
>>> df_test_unknown = pd.DataFrame({"ocean_proximity": ["<2H OCEAN", "ISLAND"]})
>>> pd.get_dummies(df_test_unknown)
   ocean_proximity_<2H OCEAN  ocean_proximity_ISLAND
0                       True                   False
1                      False                    True
```

Nhưng `OneHotEncoder` thông minh hơn nhiều: nó sẽ tự động phát hiện danh mục chưa từng biết và báo lỗi ngoại lệ. Nếu muốn, bạn có thể thiết lập hyperparameter `handle_unknown="ignore"`, trong trường hợp đó nó sẽ biểu diễn danh mục chưa biết đó bằng toàn các số 0:

```
>>> cat_encoder.handle_unknown = "ignore"
>>> cat_encoder.transform(df_test_unknown)
array([[0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0.]])
```



Nếu một thuộc tính phân loại có số lượng danh mục quá lớn (ví dụ: mã quốc gia, ngành nghề, loài sinh vật), thì việc mã hóa one-hot sẽ tạo ra một số lượng rất lớn các đặc trưng đầu vào. Điều này có thể làm chậm quá trình huấn luyện và làm suy giảm hiệu năng mô hình. Nếu điều này xảy ra, bạn có thể muốn thay thế đầu vào phân loại đó bằng các đặc trưng số hữu ích có liên quan: ví dụ: bạn có thể thay thế thuộc tính khoảng cách tới đại dương bằng khoảng cách địa lý thực tế tính bằng km (tương tự, mã quốc gia có thể được thay thế bằng dân số và GDP bình quân đầu người của quốc gia đó). Ngoài ra, bạn có thể sử dụng các bộ mã hóa trong gói `category_encoders` trên GitHub. Hoặc, khi làm việc với mạng nơ-ron nhân tạo, bạn có thể thay thế mỗi danh mục bằng một vector có số chiều thấp có thể học được gọi là vector nhúng (*embedding*, xem Chương 14). Đây là một ví dụ về học biểu diễn (*representation learning*).

Khi bạn khớp bất kỳ estimator nào của Scikit-Learn bằng một DataFrame, estimator đó sẽ lưu tên của các cột trong thuộc tính `feature_names_in_`. Sau đó, Scikit-Learn đảm bảo rằng bất kỳ DataFrame nào được đưa vào estimator này (ví dụ: gọi tới `transform()` hoặc `predict()`) đều phải có các tên cột giống hệt như vậy. Các transformer cũng cung cấp phương thức `get_feature_names_out()` mà bạn có thể sử dụng để tạo một DataFrame bao bọc lấy đầu ra của transformer:

```
>>> cat_encoder.feature_names_in_
array(['ocean_proximity'], dtype=object)
>>> cat_encoder.get_feature_names_out()
array(['ocean_proximity_<1H OCEAN', 'ocean_proximity_INLAND',
       'ocean_proximity_ISLAND', 'ocean_proximity_NEAR BAY',
       'ocean_proximity_NEAR OCEAN'], dtype=object)
>>> df_output = pd.DataFrame(cat_encoder.transform(df_test_unknown),
...                          columns=cat_encoder.get_feature_names_out(),
...                          index=df_test_unknown.index)
```

Tính năng này giúp tránh việc không khớp cột và đặc biệt hữu ích khi gỡ lỗi (debugging).

Co giãn và Biến đổi Đặc trưng (Feature Scaling and Transformation)

Một trong những phép biến đổi quan trọng nhất bạn cần áp dụng cho dữ liệu của mình là co giãn đặc trưng (Feature Scaling). Với rất ít ngoại lệ, các thuật toán Machine Learning thường hoạt động không tốt khi các thuộc tính số đầu vào có thang đo (scales) quá khác biệt nhau. Đây chính xác là trường hợp của dữ liệu nhà ở này: tổng số phòng dao động từ khoảng 6 đến 39.320, trong khi thu nhập trung vị chỉ dao động từ 0 đến 15. Nếu không co giãn đặc trưng, hầu hết các mô hình sẽ bị thiên lệch theo hướng bỏ qua thu nhập trung vị và tập trung quá mức vào số lượng phòng.

Có hai phương pháp phổ biến để đưa tất cả các thuộc tính về cùng một thang đo: co giãn min-max (min-max scaling) và chuẩn hóa z-score (standardization).



Cũng như với tất cả các estimator, điều quan trọng cốt tử là chỉ được khớp (fit) các bộ co giãn (scalers) trên dữ liệu huấn luyện: tuyệt đối không bao giờ sử dụng `fit()` hoặc `fit_transform()` cho bất kỳ tập dữ liệu nào khác ngoài tập huấn luyện! Sau khi đã có một scaler được huấn luyện xong, bạn có thể sử dụng nó để gọi `transform()` cho bất kỳ tập dữ liệu nào khác, bao gồm tập kiểm định, tập kiểm thử và dữ liệu mới ngoài đời thực. Lưu ý rằng trong khi các giá trị của tập huấn luyện sẽ luôn được co giãn về đúng dải giá trị đã chỉ định, nếu dữ liệu mới chứa các điểm ngoại lai (outliers), chúng có thể bị co giãn vượt ra ngoài phạm vi đó. Nếu muốn ngăn chặn điều này, bạn chỉ cần thiết lập hyperparameter `clip=True`.

Co giãn min-max (nhiều người gọi phương pháp này là chuẩn hóa min-max / normalization) là phương pháp đơn giản nhất: đối với mỗi thuộc tính, các giá trị được dịch chuyển và chia tỷ lệ lại để cuối cùng chúng nằm gọn trong khoảng từ 0 đến 1. Phép tính này được thực hiện bằng cách lấy giá trị trừ đi giá trị nhỏ nhất (min) rồi chia cho hiệu số giữa giá trị lớn nhất (max) và giá trị nhỏ nhất. Scikit-Learn cung cấp một transformer có tên là `MinMaxScaler` cho mục đích này. Nó có một hyperparameter `feature_range` cho phép bạn thay đổi dải giá trị nếu bạn không muốn dải từ 0 đến 1 (ví dụ: mạng nơ-ron thường thích dải từ -1 đến 1). Cách sử dụng nó khá dễ dàng:

```
from sklearn.preprocessing import MinMaxScaler

min_max_scaler = MinMaxScaler(feature_range=(-1, 1))
housing_num_min_max_scaled = min_max_scaler.fit_transform(housing_num)
```

Chuẩn hóa (Standardization) thì khác: đầu tiên nó trừ đi giá trị trung bình (vì vậy các giá trị được chuẩn hóa luôn có giá trị trung bình bằng 0), sau đó chia cho độ lệch chuẩn (vì vậy các giá trị kết quả luôn có phương sai và độ lệch chuẩn bằng 1). Không giống như co giãn min-max, chuẩn hóa không giới hạn các giá trị vào một khoảng cụ thể nào, điều này có thể là một vấn đề đối với một số thuật toán (ví dụ: mạng nơ-ron thường mong đợi các giá trị đầu vào nằm trong khoảng từ 0 đến 1). Tuy nhiên, chuẩn hóa ít bị ảnh hưởng bởi các điểm ngoại lai hơn rất nhiều. Ví dụ: giả sử một quận có thu nhập trung vị bằng 100 (do lỗi nhập liệu). Co giãn min-max sau đó sẽ thu nhỏ tất cả các giá trị thu nhập bình thường khác (từ 0 đến 15) xuống một dải cực hẹp từ 0 đến 0,15, trong khi chuẩn hóa sẽ không bị ảnh hưởng nặng nề đến như vậy. Scikit-Learn cung cấp một transformer có tên là `StandardScaler` để thực hiện việc chuẩn hóa này:

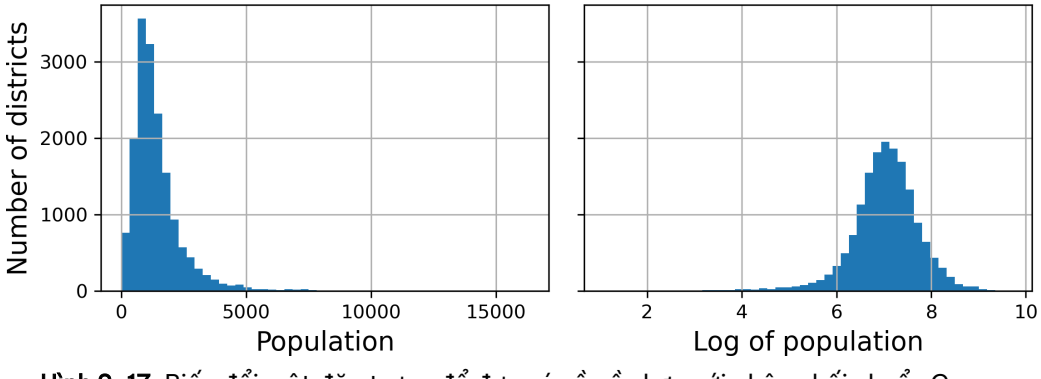
```
from sklearn.preprocessing import StandardScaler

std_scaler = StandardScaler()
housing_num_std_scaled = std_scaler.fit_transform(housing_num)
```



Nếu muốn co giãn một ma trận thưa mà không cần chuyển đổi nó thành một ma trận dày đặc trước, bạn có thể sử dụng `StandardScaler` với hyperparameter `with_mean=False`: nó sẽ chỉ chia dữ liệu cho độ lệch chuẩn mà không trừ đi giá trị trung bình (vì việc trừ giá trị trung bình sẽ phá vỡ tính chất thưa của ma trận).

Khi một đặc trưng có phân phối lệch đuôi dài (heavy-tailed distribution) – tức là các giá trị cách xa giá trị trung bình không hiếm một cách theo hàm mũ – cả min-max scaling và standardization đều sẽ ép hầu hết các giá trị vào một khoảng rất hẹp. Các mô hình Machine Learning nhìn chung hoàn toàn không thích điều này, như bạn sẽ thấy trong Chương 4. Vì vậy, trước khi co giãn đặc trưng, trước tiên bạn nên biến đổi nó để thu nhỏ phần đuôi dài lại, và nếu có thể, hãy đưa phân phối về dạng gần như đối xứng. Ví dụ: một cách phổ biến để làm điều này đối với các đặc trưng dương có phần đuôi dài lệch sang phải là thay thế đặc trưng bằng căn bậc hai của nó (hoặc nâng đặc trưng lên một lũy thừa từ 0 đến 1). Nếu đặc trưng có phần đuôi thực sự rất dài và nặng, chẳng hạn như phân phối theo luật lũy thừa (power law distribution), thì việc thay thế đặc trưng bằng hàm logarit của nó có thể sẽ giúp ích. Ví dụ: thuộc tính dân số (`population`) gần như tuân theo luật lũy thừa: các quận có 10.000 cư dân chỉ xuất hiện ít hơn 10 lần so với các quận có 1.000 cư dân, chứ không phải ít hơn theo hàm mũ. Hình 2-17 minh họa đặc trưng này trông đẹp hơn như thế nào khi bạn tính logarit của nó: nó rất gần với phân phối Gauss (tức là phân phối hình chuông).



Hình 2-17. Biến đổi một đặc trưng để đưa nó về gần hơn với phân phối chuẩn Gauss

Một cách tiếp cận khác để xử lý các đặc trưng có đuôi dài là gom cụm đặc trưng (Feature Bucketizing). Điều này có nghĩa là chia nhỏ phân phối của nó thành các khoảng (buckets) có kích thước xấp xỉ bằng nhau, và thay thế mỗi giá trị đặc trưng bằng chỉ số của khoảng mà nó thuộc về, tương tự như những gì chúng ta đã làm để tạo đặc trưng `income_cat` (mặc dù khi đó chúng ta chỉ sử dụng nó cho việc lấy mẫu phân tầng). Ví dụ: bạn có thể thay thế mỗi giá trị bằng giá trị phân vị (percentile)

của nó. Việc gom cụm với các khoảng có kích thước bằng nhau sẽ tạo ra một đặc trưng có phân phối gần như đồng đều (uniform distribution), do đó không cần phải co giãn thêm nữa, hoặc bạn chỉ cần chia cho số lượng khoảng để ép các giá trị về khoảng $0–1$.

Khi một đặc trưng có phân phối đa đỉnh (multimodal distribution) – tức là có từ hai đỉnh nhô cao rõ rệt trở lên (gọi là các mode) – chẳng hạn như thuộc tính `housing_median_age`, việc gom cụm cũng có thể rất hữu ích, nhưng lần này các ID của khoảng được xử lý như các giá trị phân loại (categories), thay vì là các giá trị số. Điều này đồng nghĩa với việc các chỉ số khoảng phải được mã hóa, ví dụ sử dụng `OneHotEncoder` (vì vậy thông thường bạn không nên sử dụng quá nhiều khoảng). Cách tiếp cận này sẽ cho phép mô hình hồi quy dễ dàng học được các quy tắc khác nhau cho các khoảng giá trị khác nhau của đặc trưng này. Ví dụ: có thể những ngôi nhà được xây dựng cách đây khoảng 35 năm có một phong cách kiến trúc đặc thù đã lỗi thời, và do đó chúng có giá rẻ hơn so với những gì mà chỉ riêng số tuổi của ngôi nhà thể hiện.

Một cách tiếp cận khác để biến đổi các phân phối đa đỉnh là thêm một đặc trưng cho mỗi đỉnh (ít nhất là các đỉnh chính), đại diện cho độ tương đồng giữa tuổi nhà trung vị và đỉnh cụ thể đó. Thước đo độ tương đồng thường được tính toán bằng hàm cơ sở bán kính (Radial Basis Function - RBF) – bất kỳ hàm nào chỉ phụ thuộc vào khoảng cách giữa giá trị đầu vào và một điểm cố định. Hàm RBF được sử dụng phổ biến nhất là Gaussian RBF: giá trị đầu ra của nó giảm dần theo hàm mũ khi giá trị đầu vào rời xa điểm mốc cố định. Ví dụ, độ tương đồng Gaussian RBF giữa tuổi nhà x và mốc 35 được xác định bởi phương trình $\exp\left(-\gamma(x-35)^2\right)$. Trong trường hợp tổng quát đa chiều, hàm này có dạng:

$$\phi_\gamma(\mathbf{x}, \boldsymbol{\ell}) = \exp\left(-\gamma\|\mathbf{x}-\boldsymbol{\ell}\|^2\right)$$

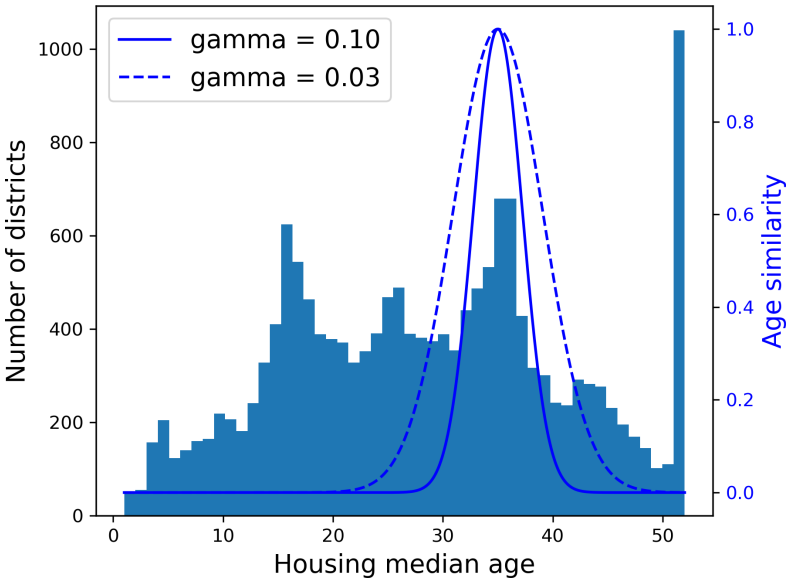
Siêu tham số (hyperparameter) γ (gamma) xác định mức độ suy giảm nhanh hay chậm của độ tương đồng khi x rời xa điểm mốc 35. Bằng cách sử dụng hàm `rbf_kernel()` của Scikit-Learn, bạn có thể tạo một đặc trưng Gaussian RBF mới đo độ tương đồng giữa tuổi nhà trung vị và mốc 35:

```
from sklearn.metrics.pairwise import rbf_kernel

age_simil_35 = rbf_kernel(housing[["housing_median_age"]], [[35]], gamma=0.1)
```

Hình 2-18 minh họa đặc trưng mới này như một hàm theo tuổi nhà trung vị (đường nét liền). Nó cũng cho thấy đặc trưng sẽ trông như thế nào nếu bạn sử dụng giá trị gamma nhỏ hơn. Như biểu đồ thể hiện, đặc trưng độ tương đồng tuổi mới đạt đỉnh tại 35, ngay xung quanh điểm tăng vọt trong phân phối tuổi nhà trung vị: nếu nhóm tuổi cụ thể này có mối tương quan chặt chẽ với mức giá thấp hơn, rất có thể đặc trưng mới này sẽ giúp ích cho mô hình.

Cho đến nay chúng ta mới chỉ xem xét các đặc trưng đầu vào, nhưng các giá trị mục tiêu (target values) cũng có thể cần phải được biến đổi. Ví dụ: nếu phân phối của mục tiêu có phần đuôi dài, bạn có thể chọn thay thế mục tiêu bằng hàm logarit của nó. Nhưng nếu làm như vậy, mô hình hồi quy giờ đây sẽ dự đoán logarit của giá trị nhà trung vị, chứ không phải bản thân giá trị nhà trung vị. Bạn sẽ cần phải tính hàm mũ của giá trị dự đoán của mô hình nếu muốn thu được giá trị nhà trung vị dự đoán



Hình 2-18. Đặc trưng Gaussian RBF đo độ tương đồng giữa tuổi nhà trung vị và giá trị mốc 35

thực tế.

Rất may mắn là hầu hết các transformer của Scikit-Learn đều có phương thức `inverse_transform()`, giúp dễ dàng tính toán phép biến đổi ngược của chúng. Ví dụ: đoạn mã dưới đây minh họa cách co giãn các nhãn bằng `StandardScaler` (giống hệt như chúng ta đã làm cho các đầu vào), sau đó huấn luyện một mô hình hồi quy tuyến tính đơn giản trên các nhãn đã co giãn và sử dụng nó để đưa ra dự đoán trên một số dữ liệu mới, rồi biến đổi ngược trở lại thang đo ban đầu bằng phương thức `inverse_transform()` của bộ co giãn đã học:

```
from sklearn.linear_model import LinearRegression

target_scaler = StandardScaler()
scaled_labels = target_scaler.fit_transform(housing_labels.to_frame())

model = LinearRegression()
model.fit(housing[["median_income"]], scaled_labels)
some_new_data = housing[["median_income"]].iloc[:5]  # pretend this is new data
scaled_predictions = model.predict(some_new_data)
predictions = target_scaler.inverse_transform(scaled_predictions)
```

Lưu ý rằng chúng ta chuyển đổi các nhãn từ một Pandas Series thành một DataFrame, vì `StandardScaler` yêu cầu đầu vào 2 chiều. Ngoài ra, trong ví dụ này, chúng ta chỉ huấn luyện mô hình trên một đặc trưng đầu vào thô duy nhất (`median_income`) để giữ cho ví dụ được đơn giản.

Cách làm này hoạt động tốt, nhưng sẽ đơn giản hơn và ít xảy ra sai sót hơn nếu sử dụng `TransformedTargetRegressor`, giúp tránh được những sự không khớp thang đo tiềm ẩn. Chúng ta chỉ cần khởi tạo nó, truyền vào mô hình hồi quy và transformer nhãn, sau đó khớp nó trên tập huấn luyện bằng các nhãn gốc chưa co giãn. Nó sẽ tự động sử dụng transformer để co giãn các nhãn và huấn luyện mô hình hồi quy trên các nhãn đã

co giãn thu được, tương tự như chúng ta đã làm thủ công ở trên. Sau đó, khi chúng ta muốn đưa ra dự đoán, nó sẽ gọi phương thức `predict()` của mô hình hồi quy và tự động sử dụng phương thức `inverse_transform()` của scaler để xuất ra dự đoán cuối cùng:

```
from sklearn.compose import TransformedTargetRegressor

model = TransformedTargetRegressor(LinearRegression(),
                                   transformer=StandardScaler())
model.fit(housing[["median_income"]], housing_labels)
predictions = model.predict(some_new_data)
```

Các Transformer Tùy chỉnh (Custom Transformers)

Mặc dù Scikit-Learn cung cấp sẵn rất nhiều transformer mạnh mẽ, bạn sẽ thường xuyên cần phải tự viết các transformer của riêng mình để thực hiện các tác vụ dọn dẹp dữ liệu cụ thể hoặc kết hợp các đặc trưng.

Nếu phép biến đổi của bạn không cần học bất kỳ tham số nào trong quá trình huấn luyện (tức là không cần phương thức `fit()`), bạn chỉ cần sử dụng lớp `FunctionTransformer`. Ví dụ: dưới đây là cách tạo một transformer tính logarit:

```
from sklearn.preprocessing import FunctionTransformer

log_transformer = FunctionTransformer(np.log, inverse_func=np.exp)
log_pop = log_transformer.transform(housing[["population"]])
```

Hàm biến đổi của bạn cũng có thể nhận các hyperparameter dưới dạng các đối số bổ sung thông qua tham số `kw_args`:

```
rbf_transformer = FunctionTransformer(rbf_kernel,
                                      kw_args=dict(Y=[[35.]], gamma=0.1))
age_simil_35 = rbf_transformer.transform(housing[["housing_median_age"]])
```

Lưu ý rằng không có hàm ngược nào cho hàm nhân RBF, vì luôn có hai giá trị ở cùng một khoảng cách nhất định tới một điểm cố định (ngoại trừ tại khoảng cách bằng 0). Cũng lưu ý rằng `rbf_kernel()` không xử lý các đặc trưng một cách riêng rẽ. Nếu bạn truyền cho nó một mảng gồm hai đặc trưng, nó sẽ đo khoảng cách 2 chiều (Euclidean) để đo lường độ tương đồng. Ví dụ: dưới đây là cách thêm một đặc trưng đo lường độ tương đồng địa lý giữa mỗi quận và San Francisco:

```
sf_coords = 37.7749, -122.41
sf_transformer = FunctionTransformer(rbf_kernel,
                                     kw_args=dict(Y=[sf_coords], gamma=0.1))
sf_simil = sf_transformer.transform(housing[["latitude", "longitude"]])
```

Các transformer tùy chỉnh cũng rất hữu ích để kết hợp các đặc trưng. Ví dụ: dưới đây là một `FunctionTransformer` tính tỷ lệ giữa đặc trưng đầu vào 0 và đặc trưng đầu vào 1:

```
>>> ratio_transformer = FunctionTransformer(lambda X: X[:, [0]] / X[:, [1]])
>>> ratio_transformer.transform(np.array([[1., 2.], [3., 4.]]))
```

```
array([[0.5 ],
       [0.75]])
```

`FunctionTransformer` rất tiện dụng, nhưng nếu bạn muốn transformer của mình có khả năng huấn luyện (trainable) – tức là học được một số tham số trong phương thức `fit()` và sau đó sử dụng chúng trong phương thức `transform()` thì sao? Đối với trường hợp này, bạn cần viết một lớp tùy chỉnh (custom class).



Phần còn lại của mục này sẽ trình bày cách định nghĩa các lớp transformer tùy chỉnh. Cụ thể, nó định nghĩa một transformer tùy chỉnh gom cụm các quận thành 10 cụm địa lý, sau đó đo khoảng cách giữa mỗi quận và tâm của từng cụm, bổ sung 10 đặc trưng độ tương đồng RBF tương ứng vào dữ liệu. Vì việc định nghĩa các lớp transformer tùy chỉnh tương đối nâng cao, bạn hoàn toàn có thể bỏ qua để chuyển sang mục tiếp theo và quay lại bất cứ khi nào cần.

Scikit-Learn dựa trên cơ chế duck typing,[13] vì vậy các lớp transformer tùy chỉnh không nhất thiết phải kế thừa từ một lớp cơ sở cụ thể nào. Tất cả những gì chúng cần là ba phương thức: `fit()` (phương thức này bắt buộc phải trả về `self`), `transform()`, và `fit_transform()`. Bạn có thể nhận được phương thức `fit_transform()` hoàn toàn miễn phí bằng cách thêm `TransformerMixin` làm lớp cơ sở: triển khai mặc định của nó chỉ đơn giản là gọi `fit()` rồi gọi `transform()`. Nếu bạn thêm `BaseEstimator` làm lớp cơ sở (và tránh sử dụng `*args` và `**kwargs` trong hàm khởi tạo của mình), bạn cũng sẽ nhận được thêm hai phương thức bổ sung: `get_params()` và `set_params()`. Hai phương thức này sẽ cực kỳ hữu ích cho việc tự động tinh chỉnh hyperparameter sau này.

Ví dụ: dưới đây là một transformer tùy chỉnh hoạt động gần giống như `StandardScaler`:

```
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.validation import check_array, check_is_fitted

class StandardScalerClone(BaseEstimator, TransformerMixin):
    def __init__(self, with_mean=True):  # no *args or **kwargs!
        self.with_mean = with_mean

    def fit(self, X, y=None):  # y is required even though we don't use it
        X = check_array(X)  # checks that X is an array with finite float values
        self.mean_ = X.mean(axis=0)
        self.scale_ = X.std(axis=0)
        self.n_features_in_ = X.shape[1]  # every estimator stores this in fit()
        return self  # always return self!

    def transform(self, X):
        check_is_fitted(self)  # checks learned attributes (with a trailing _)
        X = check_array(X)
        assert self.n_features_in_ == X.shape[1]
        if self.with_mean:
```

[13] Với duck typing, các phương thức và hành vi của một đối tượng mới là điều quan trọng, chứ không phải kiểu dữ liệu cụ thể của nó: "nếu nó trông giống một con vịt và kêu quác quác như một con vịt, thì nó hẳn phải là một con vịt".

```
            X = X - self.mean_
        return X / self.scale_
```

Dưới đây là một vài điểm quan trọng cần lưu ý:

- Gói `sklearn.utils.validation` chứa một số hàm mà chúng ta có thể sử dụng để kiểm tra tính hợp lệ của đầu vào. Để đơn giản hóa, chúng ta sẽ bỏ qua các kiểm tra này trong phần còn lại của cuốn sách, nhưng mã nguồn trên môi trường thực tế (production) nên có chúng.
- Các pipeline của Scikit-Learn yêu cầu phương thức `fit()` phải nhận hai tham số `X` và `y`, đó là lý do tại sao chúng ta cần tham số `y=None` ngay cả khi chúng ta không sử dụng `y`.
- Tất cả các estimator của Scikit-Learn đều thiết lập thuộc tính `n_features_in_` trong phương thức `fit()`, và chúng đảm bảo rằng dữ liệu được truyền vào `transform()` hoặc `predict()` có đúng số lượng đặc trưng này.
- Phương thức `fit()` bắt buộc phải trả về `self`.
- Triển khai này chưa hoàn chỉnh 100%: tất cả các estimator nên thiết lập thuộc tính `feature_names_in_` trong phương thức `fit()` khi chúng được truyền vào một DataFrame. Hơn nữa, tất cả các transformer nên cung cấp một phương thức `get_feature_names_out()`, cũng như phương thức `inverse_transform()` khi phép biến đổi của chúng có thể đảo ngược được. Xem Bài tập 6 ở phần cuối của chương này để biết thêm chi tiết.

Một transformer tùy chỉnh có thể (và thường) sử dụng các estimator khác trong quá trình triển khai của nó. Ví dụ: đoạn mã sau minh họa một bộ biến đổi tùy chỉnh sử dụng thuật toán phân cụm k-means trong phương thức `fit()` để xác định các cụm chính trong dữ liệu huấn luyện, sau đó sử dụng `rbf_kernel()` trong phương thức `transform()` để đo mức độ tương đồng giữa mỗi mẫu và từng tâm cụm:

```
from sklearn.cluster import KMeans

class ClusterSimilarity(BaseEstimator, TransformerMixin):
    def __init__(self, n_clusters=10, gamma=1.0, random_state=None):
        self.n_clusters = n_clusters
        self.gamma = gamma
        self.random_state = random_state

    def fit(self, X, y=None, sample_weight=None):
        self.kmeans_ = KMeans(self.n_clusters, random_state=self.random_state)
        self.kmeans_.fit(X, sample_weight=sample_weight)
        return self  # always return self!

    def transform(self, X):
        return rbf_kernel(X, self.kmeans_.cluster_centers_, gamma=self.gamma)

    def get_feature_names_out(self, names=None):
        return [f"Cluster {i} similarity" for i in range(self.n_clusters)]
```



Bạn có thể kiểm tra xem estimator tùy chỉnh của mình có tuân thủ đầy đủ API của Scikit-Learn hay không bằng cách truyền một thực thể vào hàm `check_estimator()` từ gói `sklearn.utils.estimator_checks`. Để tìm hiểu toàn bộ API, hãy truy cập https://scikit-learn.org/stable/developers.

Như bạn sẽ thấy trong Chương 8, k-means là một thuật toán phân cụm định vị các cụm trong dữ liệu. Ví dụ: chúng ta có thể sử dụng nó để tìm các vùng tập trung đông dân cư nhất ở California. Số lượng cụm mà k-means tìm kiếm được điều khiển bởi hyperparameter `n_clusters`. Phương thức `fit()` của `KMeans` hỗ trợ một đối số tùy chọn `sample_weight`, cho phép người dùng chỉ định trọng số tương đối của các mẫu. Ví dụ: chúng ta có thể truyền cho nó thu nhập trung vị nếu muốn các cụm bị thiên lệch về phía các quận giàu có. Sau khi huấn luyện, các tâm cụm có sẵn thông qua thuộc tính `cluster_centers_`. k-means là một thuật toán ngẫu nhiên (stochastic algorithm), nghĩa là nó dựa vào tính ngẫu nhiên để xác định vị trí các cụm, vì vậy nếu muốn có kết quả có thể tái lập, bạn phải đặt tham số `random_state`. Có thể thấy rằng, dù tác vụ khá phức tạp, đoạn mã triển khai lại tương đối gọn gàng. Bây giờ hãy sử dụng transformer tùy chỉnh này:

```
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
similarities = cluster_simil.fit_transform(housing[["latitude", "longitude"]])
```

Đoạn mã này tạo một transformer `ClusterSimilarity`, thiết lập số cụm là 10. Sau đó nó gọi `fit_transform()` với vĩ độ và kinh độ của từng quận trong tập huấn luyện (bạn có thể thử gán trọng số cho từng quận theo thu nhập trung vị của nó để xem điều đó ảnh hưởng đến các cụm như thế nào). Transformer này sử dụng k-means để xác định các cụm, sau đó đo lường độ tương đồng Gaussian RBF giữa mỗi quận và cả 10 tâm cụm. Kết quả là một ma trận với một hàng cho mỗi quận và một cột cho mỗi cụm. Hãy cùng xem 3 hàng đầu tiên, làm tròn đến hai chữ số thập phân:

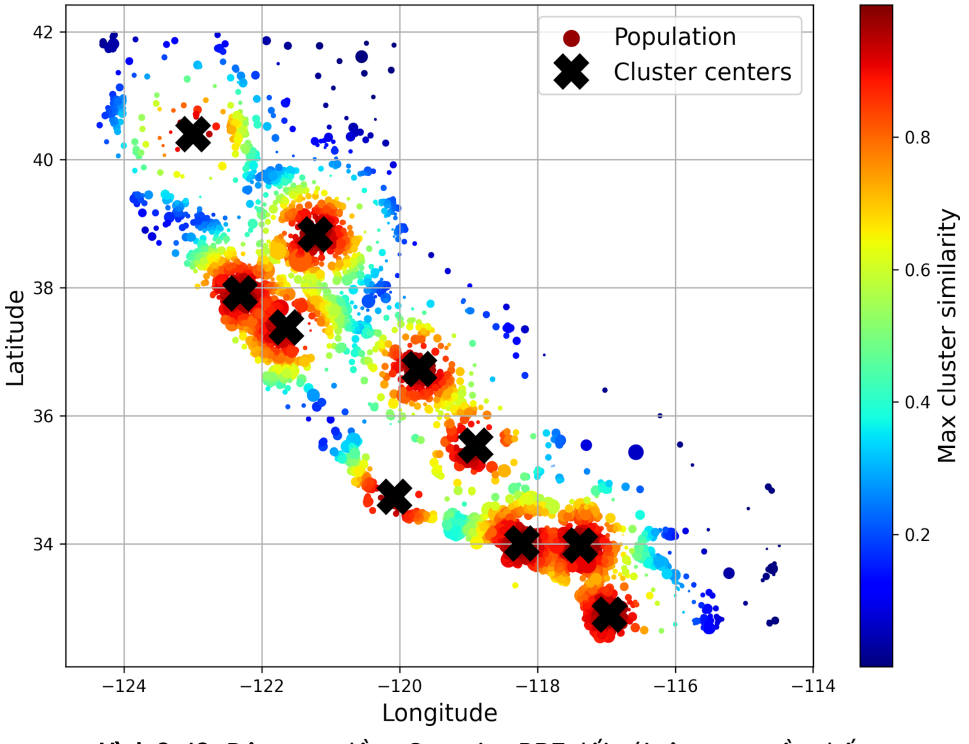
```
>>> similarities[:3].round(2)
array([[0.46, 0.  , 0.08, 0.  , 0.  , 0.  , 0.  , 0.98, 0.  , 0.  ],
       [0.  , 0.96, 0.  , 0.03, 0.04, 0.  , 0.  , 0.  , 0.11, 0.35],
       [0.34, 0.  , 0.45, 0.  , 0.  , 0.  , 0.01, 0.73, 0.  , 0.  ]])
```

Hình 2-19 thể hiện 10 tâm cụm được tìm thấy bởi k-means. Các quận được tô màu theo độ tương đồng địa lý của chúng đối với tâm cụm gần nhất. Hãy chú ý rằng hầu hết các tâm cụm đều nằm ở các khu vực tập trung đông dân cư.

Các Pipeline Biến đổi (Transformation Pipelines)

Như bạn có thể thấy, có rất nhiều bước biến đổi dữ liệu cần phải được thực hiện theo đúng trình tự nghiêm ngặt. Rất may mắn là Scikit-Learn cung cấp lớp `Pipeline` để hỗ trợ quản lý các chuỗi biến đổi tuần tự như vậy. Dưới đây là một pipeline nhỏ dành cho các thuộc tính dạng số, thực hiện việc điền giá trị khuyết trước rồi sau đó co giãn chuẩn hóa các đặc trưng:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```



Hình 2-19. Độ tương đồng Gaussian RBF đối với tâm cụm gần nhất

```
num_pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("standardize", StandardScaler()),
])
```

Hàm dựng của `Pipeline` nhận một danh sách các cặp tên/estimator (bộ 2 phần tử - 2-tuples) xác định một chuỗi các bước. Tên có thể là bất cứ thứ gì bạn thích, miễn là chúng là duy nhất và không chứa hai dấu gạch dưới liên tiếp (__). Chúng sẽ rất hữu ích sau này, khi chúng ta thảo luận về việc tinh chỉnh hyperparameter. Tất cả các estimator ngoại trừ estimator cuối cùng đều bắt buộc phải là các transformer (nghĩa là chúng phải có phương thức `fit_transform()`), trong khi estimator cuối cùng có thể là bất cứ thứ gì: transformer, predictor, hoặc bất kỳ loại estimator nào khác.



Trong Jupyter notebook, nếu bạn import thư viện `sklearn` và thực thi `sklearn.set_config(display="diagram")`, tất cả các estimator của Scikit-Learn sẽ được hiển thị dưới dạng các sơ đồ tương tác. Tính năng này đặc biệt hữu ích để trực quan hóa các pipeline. Để xem trực quan `num_pipeline`, hãy chạy một ô với `num_pipeline` ở dòng cuối cùng. Nhấp vào một estimator sẽ hiển thị thêm chi tiết.

Nếu không muốn phải tự đặt tên cho các transformer, bạn có thể sử dụng hàm tiện ích `make_pipeline()`; nó nhận các transformer làm đối số vị trí và tạo ra một `Pipeline` sử dụng tên các lớp của transformer, viết thường và không có dấu gạch dưới (ví dụ: `"simpleimputer"`):

```
from sklearn.pipeline import make_pipeline

num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
```

Nếu nhiều transformer có cùng tên, một chỉ số sẽ được nối vào tên của chúng (ví dụ: `"foo-1"`, `"foo-2"`, v.v.).

Khi bạn gọi phương thức `fit()` của pipeline, nó sẽ gọi `fit_transform()` tuần tự trên tất cả các transformer, truyền đầu ra của mỗi lệnh gọi làm tham số cho lệnh gọi tiếp theo cho đến khi chạm tới estimator cuối cùng, đối với estimator này nó chỉ gọi phương thức `fit()`.

Pipeline hiển thị các phương thức giống như estimator cuối cùng. Trong ví dụ này, estimator cuối cùng là một `StandardScaler`, vốn là một transformer, vì vậy pipeline cũng đóng vai trò như một transformer. Nếu bạn gọi phương thức `transform()` của pipeline, nó sẽ áp dụng tuần tự tất cả các phép biến đổi lên dữ liệu. Nếu estimator cuối cùng là một predictor thay vì một transformer, thì pipeline sẽ có phương thức `predict()` thay vì phương thức `transform()`. Khi gọi nó, pipeline sẽ áp dụng tuần tự tất cả các phép biến đổi lên dữ liệu và truyền kết quả cho phương thức `predict()` của predictor.

Hãy gọi phương thức `fit_transform()` của pipeline và quan sát hai hàng đầu tiên của kết quả đầu ra, được làm tròn đến hai chữ số thập phân:

```
>>> housing_num_prepared = num_pipeline.fit_transform(housing_num)
>>> housing_num_prepared[:2].round(2)
```

```
array([[-1.42,  1.01,  1.86,  0.31,  1.37,  0.14,  1.39, -0.94],
       [ 0.6 , -0.7 ,  0.91, -0.31, -0.44, -0.69, -0.37,  1.17]])
```

Như bạn đã thấy trước đó, nếu muốn khôi phục lại một DataFrame đẹp mắt, bạn có thể sử dụng phương thức `get_feature_names_out()` của pipeline:

```
df_housing_num_prepared = pd.DataFrame(
    housing_num_prepared, columns=num_pipeline.get_feature_names_out(),
    index=housing_num.index)
```

Pipelines hỗ trợ lập chỉ mục; ví dụ: `pipeline[1]` trả về estimator thứ hai trong pipeline, và `pipeline[:-1]` trả về một đối tượng `Pipeline` chứa tất cả các estimator ngoại trừ estimator cuối cùng. Bạn cũng có thể truy cập các estimator thông qua thuộc tính `steps`, đây là một danh sách các cặp tên/estimator, hoặc thông qua thuộc tính từ điển `named_steps`, ánh xạ các tên tới các estimator. Ví dụ: `num_pipeline["simpleimputer"]` trả về estimator có tên `"simpleimputer"`.

Cho đến nay, chúng ta đã xử lý các cột phân loại và cột số một cách riêng rẽ. Sẽ thuận tiện hơn nhiều nếu có một transformer duy nhất có khả năng xử lý tất cả các cột, áp dụng các phép biến đổi thích hợp cho từng cột. Đối với điều này, bạn có thể sử dụng một `ColumnTransformer`. Ví dụ: `ColumnTransformer` sau đây sẽ áp dụng `num_pipeline` (pipeline chúng ta vừa định nghĩa) cho các thuộc tính số, và `cat_pipeline` cho thuộc tính phân loại:

```
from sklearn.compose import ColumnTransformer

num_attribs = ["longitude", "latitude", "housing_median_age", "total_rooms",
               "total_bedrooms", "population", "households", "median_income"]
cat_attribs = ["ocean_proximity"]

cat_pipeline = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore"))

preprocessing = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", cat_pipeline, cat_attribs),
])
```

Đầu tiên chúng ta import lớp `ColumnTransformer`, sau đó định nghĩa danh sách tên các cột số và cột phân loại rồi xây dựng một pipeline đơn giản cho các thuộc tính phân loại. Cuối cùng, chúng ta xây dựng một `ColumnTransformer`. Hàm dựng của nó yêu cầu một danh sách các bộ 3 phần tử (3-tuples), mỗi bộ chứa một tên (phải là duy nhất và không chứa hai dấu gạch dưới liên tiếp), một transformer, và một danh sách tên (hoặc chỉ số) các cột mà transformer sẽ được áp dụng lên.



Thay vì sử dụng một transformer, bạn có thể chỉ định chuỗi ký tự `"drop"` nếu bạn muốn các cột đó bị loại bỏ, hoặc chỉ định `"passthrough"` nếu muốn các cột đó được giữ nguyên không thay đổi. Theo mặc định, các cột còn lại (tức là những cột không được liệt kê) sẽ bị loại bỏ, nhưng bạn có thể thiết

lập hyperparameter `remainder` thành bất kỳ transformer nào (hoặc thành `"passthrough"`) nếu muốn các cột này được xử lý theo cách khác.

Vì việc liệt kê thủ công tất cả tên cột không thuận tiện lắm, Scikit-Learn cung cấp lớp `make_column_selector` mà bạn có thể sử dụng để tự động chọn tất cả các đặc trưng thuộc một kiểu dữ liệu nhất định, chẳng hạn như số hoặc phân loại. Bạn có thể truyền một selector vào `ColumnTransformer` thay cho tên cột hoặc chỉ số. Hơn nữa, nếu bạn không bận tâm đến việc đặt tên cho các transformer, bạn có thể sử dụng `make_column_transformer()`, hàm này sẽ tự động chọn tên cho bạn, tương tự như những gì `make_pipeline()` thực hiện. Ví dụ: đoạn mã sau tạo ra cùng một `ColumnTransformer` như trước, ngoại trừ các transformer được tự động đặt tên là `"pipeline-1"` và `"pipeline-2"` thay vì `"num"` và `"cat"`:

```
from sklearn.compose import make_column_selector, make_column_transformer

preprocessing = make_column_transformer(
    (num_pipeline, make_column_selector(dtype_include=np.number)),
    (cat_pipeline, make_column_selector(dtype_include=object)),
)
```

Bây giờ chúng ta đã sẵn sàng để áp dụng `ColumnTransformer` này lên dữ liệu nhà ở:

```
housing_prepared = preprocessing.fit_transform(housing)
```

Tuyệt vời! Chúng ta đã có một tiền xử lý pipeline nhận toàn bộ tập dữ liệu huấn luyện và áp dụng từng transformer lên các cột phù hợp, sau đó nối các cột đã biến đổi theo chiều ngang (các transformer tuyệt đối không bao giờ được thay đổi số lượng hàng). Một lần nữa, kết quả trả về là một mảng NumPy, nhưng bạn có thể lấy tên các cột bằng `preprocessing.get_feature_names_out()` và bao bọc dữ liệu trong một DataFrame đẹp mắt như chúng ta đã làm trước đó.



`OneHotEncoder` trả về một ma trận thưa trong khi `num_pipeline` trả về một ma trận dày đặc. Khi có sự kết hợp giữa ma trận thưa và ma trận dày đặc như vậy, `ColumnTransformer` sẽ ước tính mật độ của ma trận cuối cùng (tức là tỷ lệ các ô khác không), và nó sẽ trả về một ma trận thưa nếu mật độ thấp hơn một ngưỡng nhất định (theo mặc định là `sparse_threshold=0.3`). Trong ví dụ này, nó trả về một ma trận dày đặc.

Dự án của bạn đang tiến triển cực kỳ thuận lợi và bạn gần như đã sẵn sàng để huấn luyện một số mô hình! Bây giờ bạn muốn tạo ra một pipeline hoàn chỉnh duy nhất thực hiện tất cả các phép biến đổi mà bạn đã thử nghiệm từ đầu đến giờ. Hãy cùng điểm lại những gì pipeline này sẽ thực hiện và lý do tại sao:

- Các giá trị bị thiếu trong các đặc trưng số sẽ được gán bằng giá trị trung vị, vì hầu hết các thuật toán ML không chấp nhận giá trị khuyết. Trong các đặc trưng phân loại, các giá trị bị thiếu sẽ được thay thế bằng danh mục xuất hiện thường xuyên nhất.
- Đặc trưng phân loại sẽ được mã hóa one-hot, vì hầu hết các thuật toán ML chỉ chấp nhận đầu vào dạng số.

- Một vài đặc trưng tỷ lệ sẽ được tính toán và bổ sung: `bedrooms_ratio`, `rooms_per_house`, và `people_per_house`. Hy vọng rằng các đặc trưng này sẽ có tương quan tốt hơn với giá trị nhà trung vị, qua đó hỗ trợ các mô hình ML.
- Một vài đặc trưng độ tương đồng cụm cũng sẽ được thêm vào. Các đặc trưng này có khả năng sẽ hữu ích hơn cho mô hình so với vĩ độ và kinh độ thuần túy.
- Các đặc trưng có đuôi dài sẽ được thay thế bằng logarit của chúng, vì hầu hết các mô hình ưa chuộng các đặc trưng có phân phối gần đồng đều hoặc phân phối Gauss.
- Tất cả các đặc trưng số sẽ được chuẩn hóa (standardized), vì hầu hết các thuật toán ML hoạt động tốt nhất khi tất cả các đặc trưng có thang đo xấp xỉ bằng nhau.

Đoạn mã xây dựng pipeline để thực hiện tất cả những điều này bây giờ hẳn đã rất quen thuộc với bạn:

```
from sklearn.compose import make_column_selector

def column_ratio(X):
    return X[:, [0]] / X[:, [1]]

def ratio_name(function_transformer, feature_names_in):
    return ["ratio"]

def ratio_pipeline():
    return make_pipeline(
        SimpleImputer(strategy="median"),
        FunctionTransformer(column_ratio, feature_names_out=ratio_name),
        StandardScaler())

log_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    FunctionTransformer(np.log, feature_names_out="one-to-one"),
    StandardScaler())

cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                                     StandardScaler())

preprocessing = ColumnTransformer([
        ("bedrooms", ratio_pipeline(), ["total_bedrooms", "total_rooms"]),
        ("rooms_per_house", ratio_pipeline(), ["total_rooms", "households"]),
        ("people_per_house", ratio_pipeline(), ["population", "households"]),
        ("log", log_pipeline, ["total_bedrooms", "total_rooms", "population",
                               "households", "median_income"]),
        ("geo", cluster_simil, ["latitude", "longitude"]),
        ("cat", cat_pipeline, make_column_selector(dtype_include=object)),
    ],
    remainder=default_num_pipeline)
```

Nếu bạn chạy `ColumnTransformer` này trên tập dữ liệu nhà ở ban đầu, nó sẽ thực hiện toàn bộ các phép biến đổi và xuất ra một mảng NumPy với 24 đặc trưng:

```
>>> housing_prepared = preprocessing.fit_transform(housing)
>>> housing_prepared.shape
(16512, 24)
>>> preprocessing.get_feature_names_out()
array(['bedrooms__ratio', 'rooms_per_house__ratio',
       'people_per_house__ratio', 'log__total_bedrooms',
       'log__total_rooms', 'log__population', 'log__households',
       'log__median_income', 'geo__Cluster 0 similarity',
       'geo__Cluster 1 similarity', 'geo__Cluster 2 similarity',
       'geo__Cluster 3 similarity', 'geo__Cluster 4 similarity',
       'geo__Cluster 5 similarity', 'geo__Cluster 6 similarity',
       'geo__Cluster 7 similarity', 'geo__Cluster 8 similarity',
       'geo__Cluster 9 similarity', 'cat__ocean_proximity_<1H OCEAN',
       'cat__ocean_proximity_INLAND', 'cat__ocean_proximity_ISLAND',
       'cat__ocean_proximity_NEAR BAY', 'cat__ocean_proximity_NEAR OCEAN',
       'remainder__housing_median_age'], dtype=object)
```

Lựa chọn và Huấn luyện Mô hình (Select and Train a Model)

Cuối cùng thì thời khắc quan trọng cũng đã tới! Bạn đã xác định rõ bài toán, thu thập và khám phá dữ liệu, phân chia tập huấn luyện và tập kiểm thử, đồng thời viết một pipeline tiền xử lý để tự động dọn dẹp và chuẩn bị dữ liệu cho các thuật toán Machine Learning. Giờ đây bạn đã sẵn sàng để lựa chọn và huấn luyện một mô hình Machine Learning.

Huấn luyện và Đánh giá trên Tập Huấn luyện

Tin vui là nhờ tất cả các bước chuẩn bị kỹ lưỡng trước đó, mọi thứ bây giờ sẽ trở nên vô cùng đơn giản! Bạn quyết định bắt đầu bằng việc huấn luyện một mô hình Hồi quy Tuyến tính (Linear Regression) cơ bản:

```
from sklearn.linear_model import LinearRegression

lin_reg = make_pipeline(preprocessing, LinearRegression())
lin_reg.fit(housing, housing_labels)
```

Xong! Bây giờ bạn đã có một mô hình Hồi quy Tuyến tính hoạt động hoàn chỉnh. Bạn thử nghiệm nó ngay trên tập huấn luyện, quan sát 5 dự đoán đầu tiên và so sánh chúng với các nhãn thực tế:

```
>>> housing_predictions = lin_reg.predict(housing)
>>> housing_predictions[:5].round(-2)  # -2 = round to nearest hundred
array([246000., 372700., 135700.,  91400., 330900.])
>>> housing_labels.iloc[:5].values
array([458300., 483800., 101700.,  96100., 361800.])
```

Chà, nó hoạt động, nhưng không phải lúc nào cũng tốt: dự đoán đầu tiên bị lệch rất xa (hơn $200.000\,\$$!), trong khi các dự đoán khác tốt hơn: hai dự đoán lệch khoảng 25%, và hai dự đoán lệch dưới 10%. Hãy nhớ lại rằng bạn đã chọn RMSE làm thước đo hiệu năng, vì vậy bạn muốn đo lường RMSE của mô hình hồi quy này trên toàn bộ tập huấn luyện bằng hàm `root_mean_squared_error()` của Scikit-Learn:

```
>>> from sklearn.metrics import root_mean_squared_error
>>> lin_rmse = root_mean_squared_error(housing_labels, housing_predictions)
>>> lin_rmse
68972.88910758484
```



Chúng ta không sử dụng phương thức `score()` ở đây vì nó trả về hệ số xác định R^2 thay vì RMSE. Hệ số này đại diện cho tỷ lệ phương sai trong dữ liệu mà mô hình có thể giải thích được: càng gần 1 (giá trị tối đa), mô hình càng tốt. Nếu mô hình luôn luôn chỉ dự đoán giá trị trung bình, nó sẽ không giải thích được phần phương sai nào, do đó điểm R^2 của nó bằng 0. Và nếu mô hình hoạt động thậm chí còn tệ hơn thế, thì điểm R^2 của nó có thể bị âm, thậm chí âm tùy ý.

Kết quả này tốt hơn là không có gì, nhưng rõ ràng không phải là một điểm số ấn tượng: giá trị nhà trung vị của hầu hết các quận dao động trong khoảng $120.000\,\$$ đến $265.000\,\$$, vì vậy mức sai số dự đoán điển hình lên tới $68.973\,\$$ thực sự không làm chúng ta hài lòng. Đây là một ví dụ điển hình về việc mô hình bị thiếu khớp (Underfitting) dữ liệu huấn luyện. Khi điều này xảy ra, nó có thể có nghĩa là các đặc trưng không cung cấp đủ thông tin hữu ích để đưa ra dự đoán tốt, hoặc mô hình chưa đủ mạnh mẽ.

Như chúng ta đã tìm hiểu trong chương trước, các cách chính để khắc phục hiện tượng Underfitting là: lựa chọn một mô hình mạnh mẽ hơn, cung cấp cho thuật toán huấn luyện những đặc trưng tốt hơn, hoặc giảm bớt các ràng buộc trên mô hình. Mô hình tuyến tính này không hề bị điều chuẩn hóa (regularized), do đó loại trừ phương án cuối cùng. Bạn có thể thử thêm các đặc trưng mới, nhưng trước tiên bạn muốn thử nghiệm một mô hình phức tạp hơn để xem nó thể hiện ra sao.

Bạn quyết định thử nghiệm một mô hình Cây Quyết định Hồi quy (`DecisionTreeRegressor`), vì đây là một mô hình khá mạnh có khả năng tìm ra các mối quan hệ phi tuyến phức tạp trong dữ liệu (cây quyết định sẽ được trình bày chi tiết hơn trong Chương 5):

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = make_pipeline(preprocessing, DecisionTreeRegressor(random_state=42))
tree_reg.fit(housing, housing_labels)
```

Bây giờ mô hình đã được huấn luyện, bạn hãy đánh giá nó trên tập huấn luyện:

```
>>> housing_predictions = tree_reg.predict(housing)
>>> tree_rmse = root_mean_squared_error(housing_labels, housing_predictions)
>>> tree_rmse
0.0
```

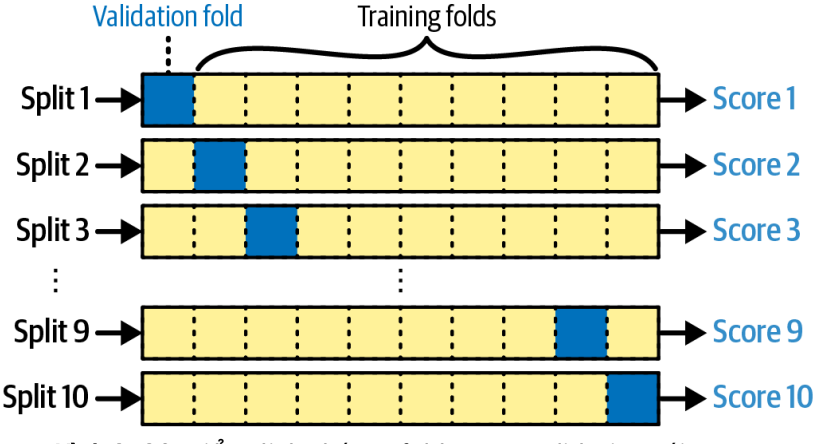
Đợi một chút! Sai số hoàn toàn bằng 0 ư? Không một chút lỗi sai nào sao? Liệu mô

hình này có thực sự hoàn hảo đến mức tuyệt đối như vậy không? Tất nhiên là không rồi. Khả năng cao hơn rất nhiều là mô hình đã bị quá khớp nghiêm trọng (Overfitting) dữ liệu huấn luyện. Làm thế nào bạn có thể chắc chắn về điều này? Như chúng ta đã biết, bạn không được phép chạm vào tập kiểm thử cho đến khi bạn hoàn toàn tự tin về mô hình của mình, vì vậy bạn cần sử dụng một phần của tập huấn luyện để huấn luyện và một phần để kiểm định mô hình.

Đánh giá Tốt hơn bằng Phương pháp Cross-Validation

Một cách để đánh giá mô hình cây quyết định là sử dụng hàm `train_test_split()` để chia tập huấn luyện thành một tập huấn luyện nhỏ hơn và một tập kiểm định (validation set), sau đó huấn luyện mô hình của bạn trên tập huấn luyện nhỏ hơn và đánh giá nó trên tập kiểm định. Cách làm này không quá khó và nó hoạt động khá tốt.

Một giải pháp thay thế tuyệt vời là sử dụng tính năng kiểm định chéo k-lần (*k-fold Cross-Validation*) của Scikit-Learn. Đoạn mã sau sẽ chia ngẫu nhiên tập huấn luyện thành 10 tập con không trùng lặp gọi là các nếp gấp (folds), sau đó nó huấn luyện và đánh giá mô hình cây quyết định 10 lần, mỗi lần chọn ra một nếp gấp khác nhau để đánh giá và huấn luyện trên 9 nếp gấp còn lại. Kết quả là một mảng gồm 10 điểm số đánh giá (xem Hình 2-20).



Hình 2-20. Kiểm định chéo k-fold cross-validation với $k = 10$

Hãy cùng sử dụng nó để đánh giá mô hình hồi quy cây quyết định của chúng ta với $k = 10$:

```
from sklearn.model_selection import cross_val_score

tree_rmses = -cross_val_score(tree_reg, housing, housing_labels,
                              scoring="neg_root_mean_squared_error", cv=10)
```



Các tính năng Cross-Validation của Scikit-Learn mong đợi một hàm thỏa dụng (utility function, giá trị càng cao càng tốt) hơn là một hàm chi phí (cost function, giá trị càng thấp càng tốt), vì vậy hàm tính điểm thực chất

là giá trị đối nghịch của RMSE. Nó là một giá trị âm, vì vậy bạn cần phải đổi dấu đầu ra (thêm dấu trừ ở đằng trước) để thu được điểm số RMSE thực tế.

Hãy cùng xem kết quả:

```
>>> pd.Series(tree_rmses).describe()
count       10.000000
mean     66573.734600
std       1103.402323
min      64607.896046
25%      66204.731788
50%      66388.272499
75%      66826.257468
max      68532.210664
dtype: float64
```

Giờ đây, mô hình cây quyết định trông không còn hoàn hảo như trước nữa. Trên thực tế, nó hoạt động kém gần như ngang ngửa với mô hình hồi quy tuyến tính! Hãy lưu ý rằng Cross-Validation không chỉ cho bạn một ước tính về hiệu năng của mô hình, mà còn cung cấp một thước đo về mức độ chuẩn xác của ước tính này (tức là độ lệch chuẩn của nó). Mô hình cây quyết định có RMSE trung bình khoảng 66.574, với độ lệch chuẩn khoảng 1.103. Bạn sẽ không thể có được thông tin giá trị này nếu chỉ sử dụng một tập kiểm định duy nhất. Tuy nhiên, việc áp dụng Cross-Validation phải trả giá bằng việc phải huấn luyện mô hình nhiều lần, vì vậy nó không phải lúc nào cũng khả thi khi tập dữ liệu quá lớn.

Nếu bạn tính toán chỉ số tương tự cho mô hình hồi quy tuyến tính, bạn sẽ thấy rằng RMSE trung bình là 70.003 và độ lệch chuẩn là 4.182. Như vậy, mô hình cây quyết định có vẻ hoạt động tốt hơn một chút so với mô hình tuyến tính, nhưng sự khác biệt là rất nhỏ do hiện tượng Overfitting nghiêm trọng. Chúng ta biết chắc chắn có vấn đề Overfitting vì sai số trên tập huấn luyện cực thấp (bằng 0), trong khi sai số kiểm định lại rất cao.

Bây giờ hãy thử nghiệm một mô hình cuối cùng: `RandomForestRegressor` (Rừng ngẫu nhiên). Như bạn sẽ thấy trong Chương 6, Random Forest hoạt động bằng cách huấn luyện nhiều cây quyết định trên các tập con ngẫu nhiên của các đặc trưng, sau đó lấy trung bình các dự đoán của chúng. Các mô hình được cấu thành từ nhiều mô hình khác như vậy được gọi là các mô hình Ensemble (Ensemble Learning): nếu các mô hình cơ sở bên dưới đa dạng, các sai số của chúng sẽ ít tương quan với nhau, và do đó việc lấy trung bình các dự đoán sẽ làm triệt tiêu các sai số ngẫu nhiên, giảm bớt hiện tượng Overfitting và cải thiện đáng kể hiệu năng tổng thể.

Đoạn mã triển khai tương tự như trước:

```
from sklearn.ensemble import RandomForestRegressor

forest_reg = make_pipeline(preprocessing,
                           RandomForestRegressor(random_state=42))
forest_rmses = -cross_val_score(forest_reg, housing, housing_labels,
                                scoring="neg_root_mean_squared_error", cv=10)
```

Hãy xem xét các điểm số thu được:

```
>>> pd.Series(forest_rmses).describe()
count       10.000000
mean     47038.092799
std       1021.491757
min      45495.976649
25%      46510.418013
50%      47118.719249
75%      47480.519175
max      49140.832210
dtype: float64
```

Chà, kết quả này tốt hơn rất nhiều: Random Forest thực sự rất triển vọng cho bài toán này! Tuy nhiên, nếu bạn huấn luyện một `RandomForestRegressor` và đo lường RMSE trên tập huấn luyện, bạn sẽ thu được kết quả khoảng 17.551: con số này thấp hơn đáng kể so với điểm kiểm định, có nghĩa là mô hình vẫn còn bị Overfitting khá nhiều. Các giải pháp khả thi là đơn giản hóa mô hình, áp đặt thêm ràng buộc (tức là điều chuẩn hóa nó), hoặc thu thập thêm nhiều dữ liệu huấn luyện hơn.

Tuy nhiên, trước khi đi sâu hơn vào việc tối ưu Random Forest, bạn nên thử nghiệm thêm nhiều mô hình khác từ các danh mục thuật toán Machine Learning đa dạng (ví dụ: máy vector hỗ trợ - Support Vector Machines với các kernel khác nhau, và có thể là một mạng nơ-ron nhân tạo), mà không cần tốn quá nhiều thời gian tinh chỉnh siêu tham số ở giai đoạn này. Mục tiêu là chọn ra một danh sách rút gọn gồm một vài (từ 2 đến 5) mô hình triển vọng nhất.

Tinh chỉnh Mô hình của Bạn (Fine-Tune Your Model)

Giả sử hiện tại bạn đã có một danh sách rút gọn gồm một vài mô hình triển vọng. Bây giờ bạn cần phải tinh chỉnh chúng (fine-tune). Hãy cùng xem xét một số cách để thực hiện điều này.

Grid Search (Tìm kiếm theo Lưới)

Một phương án là tinh chỉnh thủ công các hyperparameter (siêu tham số), cho đến khi bạn tìm ra được một tổ hợp các giá trị hyperparameter tuyệt vời. Đây sẽ là một công việc rất tẻ nhạt, mất thời gian và bạn có thể sẽ không có đủ thời gian để khám phá nhiều tổ hợp.

Thay vào đó, bạn có thể sử dụng lớp `GridSearchCV` của Scikit-Learn để thực hiện việc tìm kiếm tự động cho bạn. Tất cả những gì bạn cần làm là cho nó biết bạn muốn thử nghiệm những hyperparameter nào và những giá trị nào cần thử, và nó sẽ sử dụng Cross-Validation để đánh giá tất cả các tổ hợp giá trị hyperparameter có thể có. Ví dụ: đoạn mã sau tìm kiếm tổ hợp hyperparameter tốt nhất cho `RandomForestRegressor`:

```
from sklearn.model_selection import GridSearchCV

full_pipeline = Pipeline([
    ("preprocessing", preprocessing),
```

```
    ("random_forest", RandomForestRegressor(random_state=42)),
])

param_grid = [
    {'preprocessing__geo__n_clusters': [5, 8, 10],
     'random_forest__max_features': [4, 6, 8]},
    {'preprocessing__geo__n_clusters': [10, 15],
     'random_forest__max_features': [6, 8, 10]},
]

grid_search = GridSearchCV(full_pipeline, param_grid, cv=3,
                           scoring='neg_root_mean_squared_error')
grid_search.fit(housing, housing_labels)
```

Lưu ý rằng bạn có thể tham chiếu đến bất kỳ hyperparameter nào của bất kỳ estimator nào trong một pipeline, ngay cả khi estimator này nằm sâu bên trong nhiều pipeline và column transformer lồng nhau. Ví dụ: khi Scikit-Learn nhìn thấy chuỗi ký tự `"preprocessing__geo__n_clusters"`, nó sẽ tách chuỗi này tại các dấu gạch dưới kép (`__`), sau đó nó tìm kiếm một estimator có tên là `"preprocessing"` trong pipeline và tìm thấy ColumnTransformer tiền xử lý. Tiếp theo, nó tìm kiếm một transformer có tên là `"geo"` bên trong ColumnTransformer này và tìm thấy transformer `ClusterSimilarity` mà chúng ta đã sử dụng trên các thuộc tính vĩ độ và kinh độ. Sau đó, nó tìm thấy hyperparameter `n_clusters` của transformer này. Tương tự như vậy, `random_forest__max_features` tham chiếu đến hyperparameter `max_features` của estimator có tên là `"random_forest"`, chính là mô hình `RandomForestRegressor`.



Việc bao bọc các bước tiền xử lý trong một Scikit-Learn pipeline cho phép bạn tinh chỉnh các hyperparameter tiền xử lý cùng lúc với các hyperparameter của mô hình. Đây là một điều rất tốt vì chúng thường tương tác qua lại lẫn nhau. Ví dụ: có lẽ việc tăng `n_clusters` cũng đòi hỏi phải tăng cả `max_features`. Nếu việc khớp các transformer của pipeline tốn nhiều chi phí tính toán, bạn có thể đặt tham số `memory` của pipeline thành đường dẫn của một thư mục lưu bộ nhớ đệm (caching directory): khi bạn khớp pipeline lần đầu tiên, Scikit-Learn sẽ lưu các transformer đã khớp vào thư mục này. Nếu sau đó bạn khớp lại pipeline với các hyperparameter tương tự, Scikit-Learn sẽ chỉ việc nạp lại các transformer đã lưu từ bộ đệm.

Có hai từ điển trong `param_grid` này, vì vậy `GridSearchCV` trước tiên sẽ đánh giá tất cả $3 \times 3 = 9$ tổ hợp giá trị hyperparameter `n_clusters` và `max_features` được chỉ định trong từ điển thứ nhất, sau đó nó sẽ thử tất cả $2 \times 3 = 6$ tổ hợp trong từ điển thứ hai. Như vậy, tổng cộng Grid Search sẽ khám phá $9 + 6 = 15$ tổ hợp giá trị hyperparameter, và nó sẽ huấn luyện pipeline 3 lần cho mỗi tổ hợp (vì chúng ta đang sử dụng kiểm định chéo 3-fold CV). Điều này có nghĩa là sẽ có tổng cộng $15 \times 3 = 45$ vòng huấn luyện! Quá trình này có thể mất một khoảng thời gian, nhưng khi hoàn tất, bạn có thể lấy ra tổ hợp tham số tốt nhất như sau:

```
>>> grid_search.best_params_
{'preprocessing__geo__n_clusters': 15, 'random_forest__max_features': 6}
```



Vì 15 là giá trị tối đa được đánh giá cho `n_clusters`, bạn nên thử tìm kiếm lại với các giá trị cao hơn; điểm số có thể sẽ tiếp tục được cải thiện.

Bạn có thể truy cập mô hình ước lượng tốt nhất thông qua `grid_search.best_estimator_`. Nếu `GridSearchCV` được khởi tạo với `refit=True` (đây là giá trị mặc định), thì một khi nó tìm thấy mô hình tốt nhất bằng kiểm định chéo, nó sẽ tự động huấn luyện lại mô hình đó trên toàn bộ tập huấn luyện. Đây thường là một ý tưởng tuyệt vời, vì việc cung cấp cho mô hình nhiều dữ liệu hơn sẽ giúp cải thiện hiệu năng của nó.

Điểm số đánh giá chi tiết có sẵn thông qua `grid_search.cv_results_`. Đây là một từ điển, nhưng nếu bạn bọc nó trong một DataFrame, bạn sẽ có một danh sách rõ ràng về tất cả các điểm kiểm thử cho từng tổ hợp hyperparameter và cho từng phân chia Cross-Validation, cũng như điểm kiểm thử trung bình trên tất cả các phân chia:

```
>>> cv_res = pd.DataFrame(grid_search.cv_results_)
>>> cv_res.sort_values(by="mean_test_score", ascending=False, inplace=True)
>>> cv_res.head()
    n_clusters  max_features  split0  split1  split2  mean_test_rmse
12          15             6   42725   43708   44335           43590
13          15             8   43486   43820   44900           44069
6           10             4   43798   44036   44961           44265
9           10             6   43710   44163   44967           44280
7           10             6   43710   44163   44967           44280
```

Điểm số RMSE kiểm thử trung bình cho mô hình tốt nhất là 43.590, tốt hơn mức điểm bạn thu được trước đó khi sử dụng các giá trị hyperparameter mặc định (47.038). Xin chúc mừng, bạn đã tinh chỉnh thành công mô hình tốt nhất của mình!

Randomized Search (Tìm kiếm Ngẫu nhiên)

Cách tiếp cận Grid Search là ổn khi bạn khám phá tương đối ít tổ hợp, giống như trong ví dụ trước, nhưng `RandomizedSearchCV` thường được ưu tiên lựa chọn hơn, đặc biệt là khi không gian tìm kiếm hyperparameter lớn. Lớp này có thể được sử dụng gần như tương tự như lớp `GridSearchCV`, nhưng thay vì thử nghiệm tất cả các tổ hợp có thể, nó đánh giá một số lượng tổ hợp cố định bằng cách chọn một giá trị ngẫu nhiên cho mỗi hyperparameter ở mỗi lần lặp. Phương pháp này mang lại nhiều lợi ích vượt trội:

- Nếu một số hyperparameter của bạn là liên tục (hoặc rời rạc nhưng có rất nhiều giá trị khả dĩ), và bạn cho phép tìm kiếm ngẫu nhiên chạy trong 1.000 lần lặp, nó sẽ khám phá 1.000 giá trị khác nhau cho mỗi hyperparameter này, trong khi Grid Search sẽ chỉ khám phá một số ít giá trị mà bạn đã liệt kê thủ công cho từng tham số.
- Giả sử một hyperparameter thực chất không tạo ra nhiều sự khác biệt, nhưng bạn chưa biết điều đó. Nếu nó có 10 giá trị khả dĩ và bạn thêm nó vào Grid Search, thì quá trình huấn luyện sẽ mất thời gian lâu hơn gấp 10 lần. Nhưng nếu bạn thêm nó vào Randomized Search, nó hoàn toàn không làm tăng thêm chi phí tính toán.
- Nếu có 6 hyperparameter cần khám phá, mỗi hyperparameter có 10 giá trị khả dĩ, thì Grid Search buộc bạn phải huấn luyện mô hình $10^6 = 1.000.000$ lần, trong khi

Randomized Search luôn có thể chạy với bất kỳ số lần lặp nào do bạn tùy ý lựa chọn.

Đối với mỗi hyperparameter, bạn phải cung cấp một danh sách các giá trị khả dĩ hoặc một phân phối xác suất:

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_distribs = {'preprocessing__geo__n_clusters': randint(low=3, high=50),
                  'random_forest__max_features': randint(low=2, high=20)}

rnd_search = RandomizedSearchCV(
    full_pipeline, param_distributions=param_distribs, n_iter=10, cv=3,
    scoring='neg_root_mean_squared_error', random_state=42)
rnd_search.fit(housing, housing_labels)
```

Scikit-Learn cũng cung cấp các lớp tìm kiếm hyperparameter `HalvingRandomSearchCV` và `HalvingGridSearchCV`. Mục tiêu của chúng là sử dụng tài nguyên tính toán hiệu quả hơn, nhằm huấn luyện nhanh hơn hoặc khám phá một không gian hyperparameter rộng lớn hơn bằng cơ chế loại trừ dần các ứng viên yếu qua từng vòng lặp.

Các Phương pháp Kết hợp Mô hình (Ensemble Methods)

Một cách khác để tinh chỉnh hệ thống của bạn là cố gắng kết hợp các mô hình hoạt động tốt nhất lại với nhau. Nhóm mô hình (hoặc "ensemble") thường sẽ hoạt động tốt hơn so với mô hình đơn lẻ tốt nhất – giống như cách Random Forest hoạt động tốt hơn các cây quyết định riêng lẻ mà nó dựa vào – đặc biệt nếu các mô hình đơn lẻ mắc phải các loại sai số rất khác nhau. Chúng ta sẽ tìm hiểu sâu hơn về chủ đề này trong Chương 6.

Phân tích các Mô hình Tốt nhất và Sai số của Chúng

Bạn sẽ thường thu thập được nhiều hiểu biết sâu sắc giá trị về bài toán bằng cách kiểm tra các mô hình tốt nhất. Ví dụ: `RandomForestRegressor` có thể chỉ ra tầm quan trọng tương đối của từng thuộc tính trong việc đưa ra các dự đoán chính xác:

```
>>> final_model = rnd_search.best_estimator_
>>> feature_importances = final_model["random_forest"].feature_importances_
>>> sorted(zip(feature_importances,
...            final_model["preprocessing"].get_feature_names_out()),
...        reverse=True)
[(np.float64(0.18599734460509476), 'log__median_income'),
 (np.float64(0.07338850855844489), 'cat__ocean_proximity_INLAND'),
 (np.float64(0.06556941990883976), 'bedrooms__ratio'),
 (np.float64(0.053648710076725316), 'rooms_per_house__ratio'),
 (np.float64(0.04598870861894749), 'people_per_house__ratio'),
 (np.float64(0.04175269214442519), 'geo__Cluster 30 similarity'),
 (np.float64(0.025976797232869678), 'geo__Cluster 25 similarity'),
```

```
 (np.float64(0.023595895886342255), 'geo__Cluster 36 similarity'),
 [...]
 (np.float64(0.0004325970342247361), 'cat__ocean_proximity_NEAR BAY'),
 (np.float64(3.0190221102670295e-05), 'cat__ocean_proximity_ISLAND')]
```

Với thông tin này, bạn có thể muốn thử loại bỏ một số đặc trưng ít hữu ích nhất (ví dụ: rõ ràng chỉ có một danh mục `ocean_proximity` là thực sự hữu ích, vì vậy bạn có thể thử loại bỏ các danh mục khác).



Transformer `sklearn.feature_selection.SelectFromModel` có thể tự động loại bỏ các đặc trưng ít hữu ích nhất cho bạn: khi bạn khớp nó, nó huấn luyện một mô hình (thường là random forest), xem xét thuộc tính `feature_importances_` của mô hình và chọn ra các đặc trưng hữu ích nhất. Sau đó, khi bạn gọi `transform()`, nó sẽ tự động loại bỏ các đặc trưng còn lại.

Bạn cũng nên xem xét các lỗi cụ thể mà hệ thống của bạn mắc phải, sau đó cố gắng tìm hiểu lý do tại sao nó lại mắc lỗi đó và điều gì có thể khắc phục được vấn đề: thêm các đặc trưng bổ sung hoặc loại bỏ những đặc trưng không mang lại thông tin, làm sạch các outlier (điểm ngoại lai), v.v.

Bây giờ cũng là thời điểm thích hợp để kiểm tra tính công bằng của mô hình (model fairness): mô hình không chỉ cần hoạt động tốt trên mức trung bình, mà còn phải hoạt động tốt trên các phân nhóm quận khác nhau, cho dù chúng là nông thôn hay thành thị, giàu hay nghèo, miền bắc hay miền nam, khu vực thiểu số hay không, v.v. Điều này đòi hỏi một phân tích sai lệch chi tiết (bias analysis): tạo các tập con của tập kiểm định cho từng phân nhóm, và phân tích hiệu năng của mô hình trên từng tập con đó. Đó là một khối lượng công việc rất lớn, nhưng nó vô cùng quan trọng: nếu mô hình của bạn hoạt động kém cỏi trên toàn bộ một phân nhóm quận nào đó, thì có lẽ nó không nên được triển khai cho đến khi vấn đề được giải quyết triệt để, hoặc ít nhất nó không nên được dùng để đưa ra dự đoán cho nhóm quận đó, vì nó có thể gây hại nhiều hơn là mang lại lợi ích.

Đánh giá Hệ thống trên Tập Kiểm thử (Evaluate Your System on the Test Set)

Sau khi tinh chỉnh các mô hình của mình trong một thời gian, cuối cùng bạn cũng có được một hệ thống hoạt động đủ tốt. Bạn đã sẵn sàng để đánh giá mô hình cuối cùng trên tập kiểm thử. Không có gì quá phức tạp về quy trình này; chỉ cần lấy các predictor (biến đặc trưng) và nhãn từ tập kiểm thử của bạn, chạy `final_model` để biến đổi dữ liệu và đưa ra dự đoán, sau đó đánh giá các dự đoán này:

```
X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

final_predictions = final_model.predict(X_test)
final_rmse = root_mean_squared_error(y_test, final_predictions)
print(final_rmse)  # prints 41445.533268606625
```

Trong một số trường hợp, một ước tính điểm (point estimate) như vậy về sai số khái quát hóa sẽ không đủ để thuyết phục bạn hoặc các bên liên quan xuất bản mô hình: điều gì sẽ xảy ra nếu nó chỉ tốt hơn 0,1% so với mô hình hiện đang hoạt động trên hệ thống sản xuất? Bạn có thể muốn biết ước tính này có độ chuẩn xác như thế nào. Đối với điều này, bạn có thể tính khoảng tin cậy 95% cho sai số khái quát hóa bằng cách sử dụng kỹ thuật bootstrap với hàm `scipy.stats.bootstrap()`:

```
from scipy import stats

def rmse(squared_errors):
    return np.sqrt(np.mean(squared_errors))

confidence = 0.95
squared_errors = (final_predictions - y_test) ** 2
boot_result = stats.bootstrap([squared_errors], rmse,
                              confidence_level=confidence, random_state=42)
rmse_lower, rmse_upper = boot_result.confidence_interval
```

Bạn thu được một khoảng tin cậy tương đối rộng từ 39.521 đến 43.702, và ước tính điểm trước đó của bạn là 41.445 nằm xấp xỉ ở ngay chính giữa khoảng này.

Nếu bạn thực hiện nhiều tinh chỉnh hyperparameter, hiệu năng thực tế thường sẽ kém hơn một chút so với những gì bạn đo lường bằng kiểm định chéo (cross-validation). Đó là vì hệ thống của bạn cuối cùng đã được tinh chỉnh để hoạt động tốt trên dữ liệu kiểm định và có thể sẽ không khái quát hóa tốt bằng trên các bộ dữ liệu hoàn toàn chưa từng biết. Đó không phải là trường hợp trong ví dụ này vì RMSE kiểm thử thấp hơn RMSE kiểm định, nhưng khi điều đó xảy ra, bạn phải tuyệt đối chống lại sự cám dỗ tinh chỉnh các hyperparameter để làm cho các con số trông đẹp hơn trên tập kiểm thử; những cải tiến đó hầu như chắc chắn sẽ không thể khái quát hóa trên dữ liệu thực tế mới.

Bây giờ là giai đoạn tiền xuất bản dự án (prelaunch phase). Trình bày giải pháp một cách hiệu quả chính là điều tạo nên sự khác biệt giữa các nhà khoa học dữ liệu xuất sắc và những người bình thường. Bạn nên tạo các báo cáo súc tích (Markdown, PDF, slide thuyết trình), trực quan hóa các phát hiện cốt lõi (ví dụ: sử dụng Matplotlib hoặc các công cụ khác như Seaborn hay Tableau), và điều chỉnh thông điệp của bạn cho phù hợp với từng đối tượng người nghe: chi tiết kỹ thuật cho các đồng nghiệp kỹ sư, và góc nhìn chiến lược cấp cao cho các bên liên quan (stakeholders). Hãy đưa ra những khẳng định ấn tượng và dễ nhớ (chẳng hạn như: "thu nhập trung vị là yếu tố dự đoán số một của giá nhà"). Làm nổi bật những gì bạn đã học được, những gì hiệu quả và những gì không, những giả định nào đã được đưa ra và những hạn chế của hệ thống là gì.

Kết quả của bạn cần phải có khả năng tái lập (reproducible) tối đa: chia sẻ mã nguồn cho nhóm của bạn (ví dụ: thông qua GitHub), thêm tệp `README.md` có cấu trúc rõ ràng để hướng dẫn một kỹ sư thực hiện từng bước cài đặt. Cung cấp các notebook rõ ràng (như Jupyter) có đầy đủ mã lệnh, giải thích và kết quả, viết mã sạch sẽ và có chú thích cẩn thận. Tạo tệp `requirements.txt` hoặc `environment.yml` chứa tất cả các thư viện cần thiết cùng với phiên bản chính xác của chúng (hoặc tạo một Docker image). Thiết lập hạt giống mầm ngẫu nhiên (seed) cho các bộ sinh số ngẫu nhiên và loại bỏ bất kỳ nguồn gây biến thiên nào khác.

Trong ví dụ về dữ liệu nhà ở California này, hiệu năng cuối cùng của hệ thống không

vượt trội hơn quá nhiều so với ước tính giá của các chuyên gia (vốn thường bị lệch 30%), nhưng việc triển khai nó vẫn là một ý tưởng rất tốt, đặc biệt là nếu điều này giúp giải phóng thời gian cho các chuyên gia để họ có thể tập trung vào những nhiệm vụ thú vị và năng suất hơn.

Triển khai, Giám sát và Bảo trì Hệ thống (Launch, Monitor, and Maintain Your System)

Tuyệt vời, bạn đã nhận được sự chấp thuận từ các bên liên quan để đưa mô hình vào hoạt động! Bây giờ bạn cần chuẩn bị giải pháp của mình sẵn sàng cho môi trường sản xuất (ví dụ: hoàn thiện và dọn dẹp mã nguồn, viết tài liệu hướng dẫn và các bài kiểm thử tự động, v.v.). Sau đó, bạn có thể triển khai mô hình của mình vào môi trường sản xuất. Cách cơ bản nhất để thực hiện điều này là lưu lại mô hình tốt nhất mà bạn đã huấn luyện, chuyển tệp đó sang môi trường sản xuất và nạp lại nó. Để lưu mô hình, bạn có thể sử dụng thư viện `joblib` như sau:

```
import joblib

joblib.dump(final_model, "my_california_housing_model.pkl")
```



Việc lưu lại mọi mô hình bạn từng thử nghiệm thường là một thói quen tốt để bạn có thể dễ dàng quay lại bất kỳ mô hình nào khi cần. Bạn cũng có thể lưu các điểm số Cross-Validation và thậm chí là các dự đoán thực tế trên tập kiểm định. Điều này cho phép bạn dễ dàng so sánh điểm số giữa các loại mô hình khác nhau và phân tích dạng sai số mà từng mô hình mắc phải.

Khi mô hình đã được chuyển sang môi trường sản xuất, bạn có thể nạp nó lên và bắt đầu sử dụng. Để làm được điều này, trước tiên bạn phải import bất kỳ lớp tùy chỉnh và hàm nào mà mô hình phụ thuộc vào (nghĩa là bạn cũng phải chuyển mã nguồn tương ứng lên môi trường sản xuất), sau đó nạp mô hình bằng `joblib` và sử dụng nó để đưa ra dự đoán:

```
import joblib

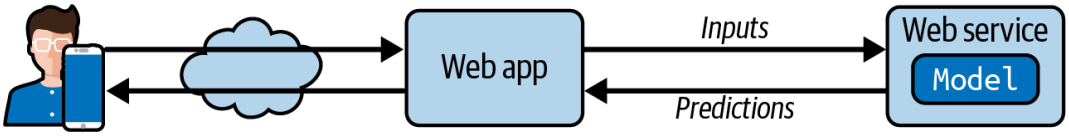
# Import custom classes and functions: KMeans, BaseEstimator, TransformerMixin, ...
def column_ratio(X): ...
def ratio_name(function_transformer, feature_names_in): ...
class ClusterSimilarity(BaseEstimator, TransformerMixin): ...

final_model_reloaded = joblib.load("my_california_housing_model.pkl")

new_data = [...]  # new districts to predict prices for
predictions = final_model_reloaded.predict(new_data)
```

Ví dụ: có thể mô hình sẽ được sử dụng bên trong một trang web: người dùng nhập thông tin về một quận mới và nhấp vào nút “Ước tính Giá”. Thao tác này sẽ gửi một

yêu cầu chứa dữ liệu tới máy chủ web, máy chủ web sẽ chuyển tiếp yêu cầu đó tới ứng dụng web của bạn, và cuối cùng mã nguồn của bạn chỉ việc gọi phương thức `predict()` của mô hình (bạn nên nạp sẵn mô hình ngay khi máy chủ khởi động, thay vì nạp lại mô hình mỗi khi có yêu cầu mới). Ngoài ra, bạn có thể đóng gói mô hình bên trong một dịch vụ web chuyên dụng mà ứng dụng web của bạn có thể truy vấn thông qua một REST API[14] (xem Hình 2-21). Cách làm này giúp việc nâng cấp mô hình lên các phiên bản mới diễn ra dễ dàng hơn mà không làm gián đoạn ứng dụng chính. Nó cũng đơn giản hóa việc mở rộng quy mô (scaling), vì bạn có thể khởi chạy bao nhiêu dịch vụ web tùy thích và phân bổ cân bằng tải các yêu cầu từ ứng dụng web trên các dịch vụ web này. Hơn nữa, nó cho phép ứng dụng web sử dụng bất kỳ ngôn ngữ lập trình nào, không nhất thiết phải là Python.



Hình 2-21. Mô hình được triển khai dưới dạng một dịch vụ web và được sử dụng bởi một ứng dụng web

Một chiến lược phổ biến khác là triển khai mô hình của bạn lên điện toán đám mây, ví dụ như trên nền tảng Vertex AI của Google (trước đây là Google Cloud AI Platform và Google Cloud ML Engine): bạn chỉ cần lưu mô hình của mình bằng `joblib` và tải nó lên Google Cloud Storage (GCS), sau đó đi tới Vertex AI và tạo một phiên bản mô hình mới trỏ tới tệp trên GCS. Thế là xong! Thao tác này sẽ cung cấp cho bạn một dịch vụ web hoàn chỉnh tự động xử lý việc cân bằng tải và co giãn tài nguyên cho bạn. Nó tiếp nhận các yêu cầu JSON chứa dữ liệu đầu vào và trả về các phản hồi JSON chứa kết quả dự đoán.

Tuy nhiên, việc triển khai chưa phải là hồi kết của câu chuyện. Bạn cũng cần phải viết mã giám sát (monitoring) để kiểm tra định kỳ hiệu năng hoạt động thực tế của hệ thống và kích hoạt cảnh báo khi hiệu năng bị sụt giảm. Hiệu năng có thể sụt giảm rất nhanh, ví dụ như khi một thành phần trong cơ sở hạ tầng bị hỏng hóc, nhưng hãy lưu ý rằng nó cũng có thể suy giảm rất từ từ theo thời gian, điều này rất dễ bị bỏ qua trong một khoảng thời gian dài nếu không theo dõi sát sao. Hiện tượng này xảy ra rất phổ biến do sự trôi dạt dữ liệu (*Data Drift*): nếu mô hình được huấn luyện bằng dữ liệu của năm ngoái, nó có thể không còn thích ứng tốt với dữ liệu của ngày hôm nay. Hoặc có thể có sự trôi dạt khái niệm (*Concept Drift*), khi mối quan hệ giữa các đặc trưng và nhãn mục tiêu thực tế đã bị biến đổi.[15]

Vì vậy, bạn cần phải chủ động giám sát hiệu năng thực tế của mô hình. Nhưng bạn làm điều đó như thế nào? Điều này còn phụ thuộc vào bài toán cụ thể. Trong một số trường hợp, hiệu năng của mô hình có thể được suy ra từ các chỉ số kinh doanh ở hạ nguồn. Ví dụ: nếu mô hình của bạn là một phần của hệ thống gợi ý sản phẩm cho người dùng, thì việc theo dõi số lượng sản phẩm được gợi ý bán ra mỗi ngày là rất dễ

[14] Nói một cách ngắn gọn, REST (hoặc RESTful) API là một giao diện lập trình dựa trên giao thức HTTP tuân theo các quy ước tiêu chuẩn, chẳng hạn như sử dụng các động từ HTTP cơ bản để đọc, cập nhật, tạo mới hoặc xóa tài nguyên (GET, POST, PUT, DELETE) và sử dụng định dạng JSON cho dữ liệu đầu vào và đầu ra.

[15] Ghi chú của người dịch: Mặc dù nguyên tác sách chỉ nhấn mạnh Data Drift, trong thực tế Concept Drift (sự thay đổi của mối quan hệ giữa đặc trưng và nhãn mục tiêu theo thời gian) cũng là một nguyên nhân cốt lõi làm suy giảm hiệu năng mô hình.

dàng. Nếu con số này giảm xuống (so với các sản phẩm không được gợi ý), thì đối tượng nghi vấn số một chính là mô hình. Điều này có thể do pipeline dữ liệu bị gián đoạn, hoặc có thể mô hình cần được huấn luyện lại trên dữ liệu mới.

Tuy nhiên, bạn cũng có thể cần đến sự phân tích của con người để đánh giá chất lượng mô hình. Ví dụ: giả sử bạn đã huấn luyện một mô hình phân loại hình ảnh để phát hiện các lỗi sản phẩm trên dây chuyền sản xuất. Làm thế nào bạn có thể nhận được cảnh báo nếu hiệu năng của mô hình bị giảm sút, trước khi hàng nghìn sản phẩm lỗi bị chuyển giao tới khách hàng? Một giải pháp là gửi một mẫu ngẫu nhiên các bức ảnh mà mô hình đã phân loại cho các chuyên viên đánh giá là con người (đặc biệt là những bức ảnh mà mô hình không thực sự chắc chắn). Tùy thuộc vào từng tác vụ, những người đánh giá có thể là các chuyên gia, hoặc họ có thể là những người không chuyên trên các nền tảng crowdsourcing (như Amazon Mechanical Turk). Trong một số ứng dụng, người đánh giá thậm chí có thể chính là người dùng cuối, phản hồi thông qua các cuộc khảo sát hoặc các bài kiểm tra captcha được tái sử dụng.[16]

Dù theo cách nào, bạn cũng cần thiết lập một hệ thống giám sát toàn diện, cũng như tất cả các quy trình liên quan để xác định phải làm gì khi xảy ra sự cố và cách chuẩn bị ứng phó với chúng. Thật không may, điều này có thể đòi hỏi rất nhiều công sức. Trên thực tế, khối lượng công việc này thường lớn hơn nhiều so với việc xây dựng và huấn luyện mô hình ban đầu.

Nếu dữ liệu liên tục thay đổi, bạn sẽ cần cập nhật các tập dữ liệu của mình và huấn luyện lại mô hình một cách định kỳ. Bạn nên tự động hóa toàn bộ quy trình này càng nhiều càng tốt. Dưới đây là một số khâu bạn có thể tự động hóa:

- Thu thập dữ liệu mới định kỳ và gán nhãn cho nó (ví dụ: thông qua chuyên viên đánh giá).
- Viết một kịch bản để huấn luyện lại mô hình và tinh chỉnh hyperparameter tự động. Kịch bản này có thể chạy tự động hàng ngày hoặc hàng tuần, tùy thuộc vào nhu cầu thực tế.
- Viết một kịch bản khác để đánh giá cả mô hình mới và mô hình cũ trên tập kiểm thử đã được cập nhật, và chỉ triển khai mô hình mới lên môi trường sản xuất nếu hiệu năng của nó không bị giảm sút (nếu bị giảm, hãy chắc chắn bạn tìm hiểu kỹ lý do). Kịch bản này nên kiểm tra hiệu năng của mô hình trên nhiều tập con khác nhau của tập kiểm thử, chẳng hạn như các quận nghèo hoặc giàu, các quận nông thôn hoặc thành thị, v.v.

Bạn cũng cần đảm bảo đánh giá chất lượng dữ liệu đầu vào của mô hình. Đôi khi hiệu năng bị giảm sút nhẹ chỉ vì một tín hiệu kém chất lượng (ví dụ: cảm biến bị trục trặc gửi về các giá trị ngẫu nhiên, hoặc dữ liệu đầu ra từ một nhóm khác bị cũ kỹ), nhưng có thể mất một khoảng thời gian dài trước khi hiệu năng của hệ thống sụt giảm đủ mức để kích hoạt cảnh báo. Nếu bạn trực tiếp giám sát dữ liệu đầu vào của mô hình, bạn có thể phát hiện sự cố sớm hơn rất nhiều. Ví dụ: bạn có thể kích hoạt cảnh báo nếu ngày càng có nhiều mẫu bị thiếu một đặc trưng nào đó, hoặc nếu giá trị trung bình hay độ lệch chuẩn trôi dạt quá xa so với tập huấn luyện, hoặc nếu một đặc trưng phân loại bắt đầu xuất hiện những danh mục mới chưa từng thấy.

Cuối cùng, hãy chắc chắn rằng bạn luôn sao lưu dự phòng (backups) cho mọi phiên bản mô hình mà bạn tạo ra và có sẵn các quy trình cũng như công cụ để nhanh chóng

[16] Captcha là một bài kiểm tra để đảm bảo người dùng không phải là robot. Các bài kiểm tra này thường được sử dụng như một cách thức tiết kiệm chi phí để gán nhãn dữ liệu huấn luyện.

quay ngược trở lại (roll back) phiên bản mô hình trước đó, trong trường hợp mô hình mới gặp sự cố nghiêm trọng. Việc có các bản sao lưu cũng giúp bạn dễ dàng so sánh các mô hình mới với các mô hình cũ. Tương tự như vậy, bạn nên lưu các bản sao lưu của từng phiên bản tập dữ liệu của mình để có thể khôi phục lại tập dữ liệu trước đó nếu dữ liệu mới bị hỏng (ví dụ: nếu dữ liệu mới thu thập chứa quá nhiều outlier (điểm ngoại lai)).

Như bạn có thể thấy, Machine Learning liên quan đến rất nhiều cơ sở hạ tầng. Đây là một chủ đề vô cùng rộng lớn được gọi là ML Operations (MLOps). Vì vậy, đừng ngạc nhiên nếu dự án Machine Learning đầu tiên của bạn đòi hỏi rất nhiều nỗ lực và thời gian để xây dựng và đưa vào vận hành thực tế. Rất may mắn là một khi toàn bộ cơ sở hạ tầng đã được thiết lập vững chắc, con đường đi từ ý tưởng đến môi trường sản xuất sẽ diễn ra nhanh hơn rất nhiều.

Hãy Thử nghiệm Thực tế! (Try It Out!)

Hy vọng rằng chương này đã mang đến cho bạn một cái nhìn toàn diện và rõ ràng về quy trình triển khai một dự án Machine Learning hoàn chỉnh ngoài đời thực, cũng như giới thiệu một số công cụ mạnh mẽ mà bạn có thể sử dụng để xây dựng một hệ thống xuất sắc. Như bạn có thể thấy, phần lớn khối lượng công việc thực chất nằm ở bước chuẩn bị dữ liệu: xây dựng các công cụ giám sát, thiết lập các pipeline đánh giá và tự động hóa việc huấn luyện mô hình định kỳ. Các thuật toán Machine Learning tất nhiên là rất quan trọng, nhưng việc nắm vững toàn bộ quy trình tổng thể và hiểu sâu sắc 3 hoặc 4 thuật toán cốt lõi sẽ mang lại lợi ích thực tế lớn hơn nhiều so với việc dành toàn bộ thời gian chỉ để chạy theo những thuật toán phức tạp xa vời.

Vì vậy, nếu bạn chưa làm điều đó, bây giờ là thời điểm tuyệt vời nhất để bạn mở máy tính xách tay lên, chọn một bộ dữ liệu mà bạn cảm thấy hứng thú và tự mình trải nghiệm toàn bộ quy trình này từ A đến Z. Một nơi lý tưởng để bắt đầu là các trang web thi đấu trực tuyến như Kaggle: bạn sẽ có sẵn một tập dữ liệu thực tế, một mục tiêu rõ ràng và một cộng đồng lớn để cùng nhau chia sẻ kinh nghiệm. Chúc bạn có những trải nghiệm thật thú vị!

Bài tập Thực hành (Exercises)

Các bài tập thực hành sau đây được xây dựng dựa trên bộ dữ liệu nhà ở California của chương này:

1. Thử nghiệm một mô hình hồi quy máy vector hỗ trợ (`sklearn.svm.SVR`) với nhiều hyperparameter khác nhau, chẳng hạn như `kernel="linear"` (với nhiều giá trị khác nhau cho hyperparameter `C`) hoặc `kernel="rbf"` (với nhiều giá trị khác nhau cho các hyperparameter `C` và `gamma`). Lưu ý rằng máy vector hỗ trợ không mở rộng tốt trên các bộ dữ liệu lớn, vì vậy bạn có thể chỉ nên huấn luyện mô hình của mình trên 5.000 mẫu đầu tiên của tập huấn luyện và chỉ sử dụng 3-fold cross-validation, nếu không quá trình chạy sẽ mất hàng giờ đồng hồ. Đừng quá lo lắng về ý nghĩa của các hyperparameter này vào lúc này; chúng sẽ được giải thích cặn kẽ trong

chương trực tuyến về SVM tại https://homl.info/. Mô hình dự đoán SVR tốt nhất đạt được hiệu năng như thế nào?

2. Thử thay thế `GridSearchCV` bằng một `RandomizedSearchCV`.

3. Thử thêm một transformer `SelectFromModel` vào pipeline chuẩn bị dữ liệu để chỉ chọn ra những thuộc tính quan trọng nhất.

4. Thử tạo một transformer tùy chỉnh thực hiện việc huấn luyện một mô hình hồi quy k-hàng xóm gần nhất (`sklearn.neighbors.KNeighborsRegressor`) trong phương thức `fit()` của nó, và xuất ra các dự đoán của mô hình trong phương thức `transform()`. Mô hình hồi quy KNN này chỉ nên sử dụng vĩ độ và kinh độ làm đầu vào và dự đoán giá trị thu nhập trung vị. Tiếp theo, hãy thêm transformer mới này vào pipeline tiền xử lý. Thao tác này sẽ bổ sung thêm một đặc trưng mới đại diện cho thu nhập trung vị được làm mượt trên các quận lân cận.

5. Tự động khám phá một số tùy chọn chuẩn bị dữ liệu bằng cách sử dụng `RandomizedSearchCV`.

6. Thử tự triển khai lại lớp `StandardScalerClone` từ đầu, sau đó thêm hỗ trợ cho phương thức `inverse_transform()`: việc thực thi lệnh `scaler.inverse_transform(scaler.fit_transform(X))` sẽ phải trả về một mảng có giá trị xấp xỉ gần như tuyệt đối với `X`. Tiếp theo, hãy thêm hỗ trợ cho tên đặc trưng: thiết lập thuộc tính `feature_names_in_` trong phương thức `fit()` nếu dữ liệu đầu vào là một DataFrame. Thuộc tính này phải là một mảng NumPy chứa tên các cột. Cuối cùng, hãy triển khai phương thức `get_feature_names_out()`: phương thức này nên nhận một đối số tùy chọn `input_features=None`. Nếu đối số này được truyền vào, phương thức nên kiểm tra xem độ dài của nó có khớp với `n_features_in_` hay không, và phải khớp với `feature_names_in_` nếu thuộc tính này đã được định nghĩa; sau đó trả về `input_features`. Nếu `input_features` là `None`, phương thức nên trả về `feature_names_in_` (nếu đã định nghĩa) hoặc mảng `np.array(["x0", "x1", ...])` có độ dài bằng `n_features_in_` trong trường hợp ngược lại.

7. Giải quyết một bài toán hồi quy tự chọn theo quy trình chuẩn bạn đã học trong chương này. Ví dụ: bạn có thể thử giải quyết bộ dữ liệu Phương tiện giao thông (Vehicle dataset), với mục tiêu là dự đoán giá bán của một chiếc xe ô tô đã qua sử dụng dựa trên số tuổi của xe, số kilômét đã đi, hãng xe, dòng xe và nhiều yếu tố khác. Một bộ dữ liệu thú vị khác để thử sức là bộ dữ liệu Chia sẻ xe đạp (Bike Sharing dataset): mục tiêu là dự đoán số lượng xe đạp được thuê trong một khoảng thời gian (cột `cnt`), dựa trên ngày trong tuần, thời gian và điều kiện thời tiết.

Lời giải chi tiết cho các bài tập này có sẵn ở phần cuối của Jupyter notebook đi kèm chương này tại https://homl.info/colab-p.